

Human review packet: GHKR22BiasBounties

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: GHKR22BiasBounties

Paper id: GHKR22BiasBounties

Source version: arXiv:2201.10408v4, cross-checked against FAcCT 2022 DOI 10.1145/3531146.3533172

Source record: audit/paper_statement_map.json

Rows: 15

Generated: 2026-09-06

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper GHKR22BiasBounties --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

Contents

- [Governing Lean declarations \(137; supporting context\)](#)
- [Semantic library prerequisites \(1; not paper claims\)](#)
 - [AppliedModelingLib.Learning.Prediction.BoundedLoss](#)
- [Paper-specific semantic prerequisites \(26; not paper claims\)](#)
 - [GHKR22BiasBounties.AEBayesOptimalForRisk](#)
 - [GHKR22BiasBounties.BoundedLoss](#)
 - [GHKR22BiasBounties.Group](#)
 - [GHKR22BiasBounties.Model](#)
 - [GHKR22BiasBounties.MeasureApproxBayesOptimal](#)
 - [GHKR22BiasBounties.groupIndicator](#)
 - [GHKR22BiasBounties.measureGroupLossNumerator](#)
 - [GHKR22BiasBounties.measureGroupMass](#)
 - [GHKR22BiasBounties.measureGroupLoss](#)
 - [GHKR22BiasBounties.MeasureCertificateOfSuboptimality](#)
 - [GHKR22BiasBounties.MeasureCostSensitiveMinimizer](#)
 - [GHKR22BiasBounties.adaptiveFalsifyAndUpdate](#)
 - [GHKR22BiasBounties.certificateCheckerDecision](#)
 - [GHKR22BiasBounties.certificateCheckerRun](#)
 - [GHKR22BiasBounties.checkerAcceptanceBudget](#)
 - [GHKR22BiasBounties.derivedGroup](#)
 - [GHKR22BiasBounties.derivedModel](#)
 - [GHKR22BiasBounties.listUpdate](#)
 - [GHKR22BiasBounties.falsifyAndUpdateStep](#)
 - [GHKR22BiasBounties.inducedCost](#)
 - [GHKR22BiasBounties.kernelConditionalRisk](#)
 - [GHKR22BiasBounties.measureCoordinateAscentStep](#)
 - [GHKR22BiasBounties.measureCoordinateAscentLoop](#)
 - [GHKR22BiasBounties.measureModelLoss](#)

- GHKR22BiasBounties.monotoneBountyOutput
- GHKR22BiasBounties.trainByOptRun
- Source claims (15)
 - source_observation4Spec
 - source_theorem8Spec
 - source_theorem9Spec
 - source_theorem10Spec
 - source_theorem11Spec
 - source_theorem12Spec
 - source_remark13Spec
 - source_theorem14Spec
 - certificateOptimizationObjectiveSpec
 - source_lemma15Spec
 - source_theorem16Spec
 - source_theorem20Spec
 - source_lemma21Spec
 - source_lemma22Spec
 - source_theorem23Spec

Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

AppliedModelingLib.HeterogeneousBatchTrace

Declaration location: reusable library

Lean declaration body

```
type:
( $\mathbb{N} \rightarrow \text{Type } u_1 \rightarrow \text{Type } u_2 \rightarrow \mathbb{N} \rightarrow \text{Type } (\max u_1 u_2)$ )

value:
fun (BatchIndex :  $\mathbb{N} \rightarrow \text{Type } u_1$ ) (Data :  $\text{Type } u_2$ ) (iteration :  $\mathbb{N}$ ) => (round : Fin iteration)  $\rightarrow$  BatchIndex  $\uparrow$  round  $\rightarrow$  Data
```

AppliedModelingLib.HeterogeneousBatchTraceCoordinate

Declaration location: reusable library

Lean declaration body

```
type:
( $\mathbb{N} \rightarrow \text{Type } u_1 \rightarrow \text{Type } u_2 \rightarrow \mathbb{N} \rightarrow \text{Type } (\max u_1 u_2)$ )

value:
fun (BatchIndex :  $\mathbb{N} \rightarrow \text{Type } u_1$ ) (Data :  $\text{Type } u_2$ ) (a :  $\mathbb{N}$ ) =>
  match a with
  | 0 => PUnit.{( $\max u_1 u_2$ ) + 1}
  | round.succ => BatchIndex round  $\rightarrow$  Data
```

AppliedModelingLib.Learning.Prediction.HardGroup

Declaration location: reusable library

Lean declaration body

```
type:
Type  $u_1 \rightarrow \text{Type } u_1$ 

value:
fun (X :  $\text{Type } u_1$ ) => X  $\rightarrow$  Bool
```

AppliedModelingLib.Learning.Prediction.Model

Declaration location: reusable library

Lean declaration body

```
type:
Type  $u_1 \rightarrow \text{Type } u_2 \rightarrow \text{Type } (\max u_1 u_2)$ 

value:
fun (X :  $\text{Type } u_1$ ) (Y :  $\text{Type } u_2$ ) => X  $\rightarrow$  Y
```

AppliedModelingLib.Learning.Prediction.hardGroupIndicator

Declaration location: reusable library

Lean declaration body

```
type:
{X :  $\text{Type } u_1$ }  $\rightarrow$  AppliedModelingLib.Learning.Prediction.HardGroup X  $\rightarrow$  X  $\rightarrow \mathbb{R}$ 

value:
fun {X :  $\text{Type } u_1$ } (group : AppliedModelingLib.Learning.Prediction.HardGroup X) (x : X) =>
  if group x = true then 1 else 0
```

Direct retained dependencies

- [AppliedModelingLib.Learning.Prediction.HardGroup](#)

AppliedModelingLib.Probability.finiteEmpiricalMean

Declaration location: reusable library

Lean declaration body

```
type:
  {Ω : Type u_1} →
  {Hypothesis : Type u_2} → {Index : Type u_3} → [Fintype Index] → (Hypothesis → Index → Ω → ℝ) → Hypothesis → Ω → ℝ

value:
  fun {Ω : Type u_1} {Hypothesis : Type u_2} {Index : Type u_3} [Fintype Index] (observation : Hypothesis → Index → Ω → ℝ)
    (hypothesis : Hypothesis) (outcome : Ω) =>
    (Σ index : Index, observation hypothesis index outcome) / ↑(Fintype.card Index)
```

AppliedModelingLib.Probability.finiteIIDGhostSample

Declaration location: reusable library

Lean declaration body

```
type:
  {X : Type u_1} → (count : ℕ) → (Fin (count + count) → X) → Fin count → X

value:
  fun {X : Type u_1} (count : ℕ) (a : Fin (count + count) → X) (a_1 : Fin count) => a (Fin.natAdd count a_1)
```

AppliedModelingLib.Probability.finiteIIDSampleLaw

Declaration location: reusable library

Lean declaration body

```
type:
  {X : Type u_1} →
  [inst : MeasurableSpace X] → MeasureTheory.Measure X → (count : ℕ) → MeasureTheory.Measure (Fin count → X)

value:
  fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure X) (count : ℕ) =>
  MeasureTheory.Measure.pi fun (x : Fin count) => law
```

AppliedModelingLib.Probability.finiteIIDTrainSample

Declaration location: reusable library

Lean declaration body

```
type:
  {X : Type u_1} → (count : ℕ) → (Fin (count + count) → X) → Fin count → X

value:
  fun {X : Type u_1} (count : ℕ) (a : Fin (count + count) → X) (a_1 : Fin count) => a (Fin.castAdd count a_1)
```

AppliedModelingLib.Probability.finiteIIDScoreSum

Declaration location: reusable library

Lean declaration body

```
type:
  {α : Type u_1} → {ι : Type u_2} → [Fintype ι] → (α → ℝ) → (ι → α) → ℝ

value:
  fun {α : Type u_1} {ι : Type u_2} [Fintype ι] (score : α → ℝ) (sample : ι → α) => Σ i : ι, score (sample i)
```

AppliedModelingLib.Statistics.BinaryClassifier

Declaration location: reusable library

Lean declaration body

```
type:
  Type u_1 → Type u_1

value:
  fun (X : Type u_1) => X → Bool
```

AppliedModelingLib.Statistics.binaryClassificationLoss

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} → AppliedModelingLib.Statistics.BinaryClassifier X → X × Bool → ℝ

value:
fun {X : Type u_1} (classifier : AppliedModelingLib.Statistics.BinaryClassifier X) (datum : X × Bool) =>
  if classifier datum.1 = datum.2 then 0 else 1
```

Direct retained dependencies

- [AppliedModelingLib.Statistics.BinaryClassifier](#)

AppliedModelingLib.Statistics.binaryPopulationError

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} →
  [inst : MeasurableSpace X] → MeasureTheory.Measure (X × Bool) → AppliedModelingLib.Statistics.BinaryClassifier X → ℝ

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
  (classifier : AppliedModelingLib.Statistics.BinaryClassifier X) =>
  ∫ (x : X × Bool), AppliedModelingLib.Statistics.binaryClassificationLoss classifier x ∂law
```

Direct retained dependencies

- [AppliedModelingLib.Statistics.BinaryClassifier](#)
- [AppliedModelingLib.Statistics.binaryClassificationLoss](#)

AppliedModelingLib.Statistics.binarySwappedFirstSample

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} → {Index : Type u_2} → (Index → X × Bool) → (Index → X × Bool) → (Index → Bool) → Index → X × Bool

value:
fun {X : Type u_1} {Index : Type u_2} (firstSample secondSample : Index → X × Bool) (swap : Index → Bool) (a : Index) =>
  if swap a = true then firstSample a else secondSample a
```

AppliedModelingLib.Statistics.binarySwappedSecondSample

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} → {Index : Type u_2} → (Index → X × Bool) → (Index → X × Bool) → (Index → Bool) → Index → X × Bool

value:
fun {X : Type u_1} {Index : Type u_2} (firstSample secondSample : Index → X × Bool) (swap : Index → Bool) (a : Index) =>
  if swap a = true then secondSample a else firstSample a
```

AppliedModelingLib.Statistics.binaryTrace

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} → Set (AppliedModelingLib.Statistics.BinaryClassifier X) → (sample : Finset X) → Finset (Finset ↑sample)

value:
fun {X : Type u_1} (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (sample : Finset X) =>
  {labels : Finset ↑sample | ∃ classifier ∈ concepts, ∀ (point : ↑sample), point ∈ labels ↔ classifier ↑point = true}
```

Direct retained dependencies

- [AppliedModelingLib.Statistics.BinaryClassifier](#)

AppliedModelingLib.Statistics.binaryTraceVCDimension

Declaration location: reusable library

Lean declaration body

```
type:
  {X : Type u_1} → Set (AppliedModelingLib.Statistics.BinaryClassifier X) → Finset X → ℕ
value:
  fun {X : Type u_1} (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (sample : Finset X) =>
    (AppliedModelingLib.Statistics.binaryTrace concepts sample).vcDim
```

Direct retained dependencies

- [AppliedModelingLib.Statistics.BinaryClassifier](#)
- [AppliedModelingLib.Statistics.binaryTrace](#)

AppliedModelingLib.Statistics.VCDimensionAtMost

Declaration location: reusable library

Lean declaration body

```
type:
  {X : Type u_1} → Set (AppliedModelingLib.Statistics.BinaryClassifier X) → ℕ → Prop
value:
  fun {X : Type u_1} (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (d : ℕ) =>
    ∀ (sample : Finset X), AppliedModelingLib.Statistics.binaryTraceVCDimension concepts sample ≤ d
```

Direct retained dependencies

- [AppliedModelingLib.Statistics.BinaryClassifier](#)
- [AppliedModelingLib.Statistics.binaryTraceVCDimension](#)

AppliedModelingLib.Statistics.finiteBinaryEmpiricalError

Declaration location: reusable library

Lean declaration body

```
type:
  {X : Type u_1} →
    {Index : Type u_2} → [Fintype Index] → (Index → X × Bool) → AppliedModelingLib.Statistics.BinaryClassifier X → ℝ
value:
  fun {X : Type u_1} {Index : Type u_2} [Fintype Index] (sample : Index → X × Bool)
    (classifier : AppliedModelingLib.Statistics.BinaryClassifier X) =>
    (∑ index : Index, AppliedModelingLib.Statistics.binaryClassificationLoss classifier (sample index)) /
      ↑(Fintype.card Index)
```

Direct retained dependencies

- [AppliedModelingLib.Statistics.BinaryClassifier](#)
- [AppliedModelingLib.Statistics.binaryClassificationLoss](#)

AppliedModelingLib.Statistics.finiteIIDVCPairSwapBadEvent

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} →
  Set (AppliedModelingLib.Statistics.BinaryClassifier X) →
    (count : ℕ) → ℝ → (Fin (count + count) → X × Bool) × (Fin count → Bool) → Prop

value:
fun {X : Type u_1} (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (count : ℕ) (error : ℝ)
  (a : (Fin (count + count) → X × Bool) × (Fin count → Bool)) =>
  {pair : (Fin (count + count) → X × Bool) × (Fin count → Bool) |
    ∃ classifier ∈ concepts,
      error ≤
        AppliedModelingLib.Statistics.finiteBinaryEmpiricalError
          (AppliedModelingLib.Statistics.binarySwappedFirstSample
            (AppliedModelingLib.Probability.finiteIIDTrainSample count pair.1)
            (AppliedModelingLib.Probability.finiteIIDGhostSample count pair.1) pair.2)
          classifier -
        AppliedModelingLib.Statistics.finiteBinaryEmpiricalError
          (AppliedModelingLib.Statistics.binarySwappedSecondSample
            (AppliedModelingLib.Probability.finiteIIDTrainSample count pair.1)
            (AppliedModelingLib.Probability.finiteIIDGhostSample count pair.1) pair.2)
          classifier}
a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.finiteIIDGhostSample](#)
- [AppliedModelingLib.Probability.finiteIIDTrainSample](#)
- [AppliedModelingLib.Statistics.BinaryClassifier](#)
- [AppliedModelingLib.Statistics.binarySwappedFirstSample](#)
- [AppliedModelingLib.Statistics.binarySwappedSecondSample](#)
- [AppliedModelingLib.Statistics.finiteBinaryEmpiricalError](#)

AppliedModelingLib.Statistics.finiteIIDVCPopulationOverestimateBadEvent

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} →
  [inst : MeasurableSpace X] →
    MeasureTheory.Measure (X × Bool) →
      Set (AppliedModelingLib.Statistics.BinaryClassifier X) → (count : ℕ) → ℝ → (Fin count → X × Bool) → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
  (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (count : ℕ) (error : ℝ)
  (a : Fin count → X × Bool) =>
  {trainingSample : Fin count → X × Bool |
    ∃ classifier ∈ concepts,
      error ≤
        AppliedModelingLib.Statistics.finiteBinaryEmpiricalError trainingSample classifier -
        AppliedModelingLib.Statistics.binaryPopulationError law classifier}
a
```

Direct retained dependencies

- [AppliedModelingLib.Statistics.BinaryClassifier](#)
- [AppliedModelingLib.Statistics.binaryPopulationError](#)
- [AppliedModelingLib.Statistics.finiteBinaryEmpiricalError](#)

AppliedModelingLib.Statistics.finiteIIDVCPopulationUnderestimateBadEvent

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) →
Set (AppliedModelingLib.Statistics.BinaryClassifier X) → (count : ℕ) → ℝ → (Fin count → X × Bool) → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
  (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (count : ℕ) (error : ℝ)
  (a : Fin count → X × Bool) =>
{trainingSample : Fin count → X × Bool |
  ∃ classifier ∈ concepts,
    error ≤
      AppliedModelingLib.Statistics.binaryPopulationError law classifier -
      AppliedModelingLib.Statistics.finiteBinaryEmpiricalError trainingSample classifier}
a
```

Direct retained dependencies

- [AppliedModelingLib.Statistics.BinaryClassifier](#)
- [AppliedModelingLib.Statistics.binaryPopulationError](#)
- [AppliedModelingLib.Statistics.finiteBinaryEmpiricalError](#)

AppliedModelingLib.Statistics.finiteIIDVCSourceConfidenceRadius

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → ℕ → ℝ → ℝ

value:
fun (d count : ℕ) (delta : ℝ) => 2 * √((↑d * (Real.log (↑(count + count) / ↑d) + 1) + Real.log (9 / delta)) / ↑count)
```

AppliedModelingLib.Statistics.finiteIIDVCTwoSampleBadEvent

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} →
Set (AppliedModelingLib.Statistics.BinaryClassifier X) → (count : ℕ) → ℝ → (Fin (count + count) → X × Bool) → Prop

value:
fun {X : Type u_1} (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (count : ℕ) (error : ℝ)
  (a : Fin (count + count) → X × Bool) =>
{sample : Fin (count + count) → X × Bool |
  ∃ classifier ∈ concepts,
    error ≤
      AppliedModelingLib.Statistics.finiteBinaryEmpiricalError
        (AppliedModelingLib.Probability.finiteIIDTrainSample count sample) classifier -
      AppliedModelingLib.Statistics.finiteBinaryEmpiricalError
        (AppliedModelingLib.Probability.finiteIIDGhostSample count sample) classifier}
a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.finiteIIDGhostSample](#)
- [AppliedModelingLib.Probability.finiteIIDTrainSample](#)
- [AppliedModelingLib.Statistics.BinaryClassifier](#)
- [AppliedModelingLib.Statistics.finiteBinaryEmpiricalError](#)

AppliedModelingLib.Statistics.finiteIIDVCTwoSampleProductBadEvent

Declaration location: reusable library

Lean declaration body

```
type:
{X : Type u_1} →
Set (AppliedModelingLib.Statistics.BinaryClassifier X) →
(count : ℕ) → ℝ → (Fin count → X × Bool) × (Fin count → X × Bool) → Prop

value:
fun {X : Type u_1} (concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (count : ℕ) (error : ℝ)
(a : (Fin count → X × Bool) × (Fin count → X × Bool)) =>
{samples : (Fin count → X × Bool) × (Fin count → X × Bool) |
  ∃ classifier ∈ concepts,
    error ≤
      AppliedModelingLib.Statistics.finiteBinaryEmpiricalError samples.1 classifier -
      AppliedModelingLib.Statistics.finiteBinaryEmpiricalError samples.2 classifier}
a
```

Direct retained dependencies

- [AppliedModelingLib.Statistics.BinaryClassifier](#)
- [AppliedModelingLib.Statistics.finiteBinaryEmpiricalError](#)

AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace

Declaration location: reusable library

Lean declaration body

```
type:
(BatchIndex : ℕ → Type u_1) →
(Data : Type u_2) →
[MeasurableSpace Data] →
(round : ℕ) → MeasurableSpace (AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data round)

value:
fun (BatchIndex : ℕ → Type u_1) (Data : Type u_2) [MeasurableSpace Data] (round : ℕ) =>
Nat.casesAuxOn (motive := fun (a : ℕ) =>
round = a → MeasurableSpace (AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data round)) round
(fun (h : round = 0) =>
  AppliedModelingLib.heterogeneousBatchTraceCoordinateFintype._proof_1 round h ▶ id inferInstance)
(fun (n : ℕ) (h : round = n + 1) =>
  AppliedModelingLib.heterogeneousBatchTraceCoordinateFintype._proof_2 round n h ▶ id inferInstance)
(Eq.refl round)
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)

AppliedModelingLib.heterogeneousBatchTraceInfiniteLaw

Declaration location: reusable library

Lean declaration body

```
type:
{Data : Type u_1} →
[inst : MeasurableSpace Data] →
(BatchIndex : ℕ → Type u_2) →
(kernel :
  (iteration : ℕ) →
  ProbabilityTheory.Kernel
    ((i : 1 (Finset.lic iteration)) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data ↑i)
    (AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data (iteration + 1))) →
[∀ (iteration : ℕ), ProbabilityTheory.IsMarkovKernel (kernel iteration)] →
MeasureTheory.Measure ((i : ℕ) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data i)

value:
fun {Data : Type u_1} [MeasurableSpace Data] (BatchIndex : ℕ → Type u_2)
(kernel :
  (iteration : ℕ) →
  ProbabilityTheory.Kernel
    ((i : 1 (Finset.lic iteration)) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data ↑i)
    (AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data (iteration + 1)))
[∀ (iteration : ℕ), ProbabilityTheory.IsMarkovKernel (kernel iteration)] =>
ProbabilityTheory.Kernel.trajMeasure (MeasureTheory.Measure.dirac PUnit.unit) kernel
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace](#)

AppliedModelingLib.heterogeneousBatchTraceOfInfiniteTrace

Declaration location: reusable library

Lean declaration body

```
type:
{BatchIndex : ℕ → Type u_1} →
{Data : Type u_2} →
(iteration : ℕ) →
((i : ℕ) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data i) →
(round : Fin iteration) → BatchIndex ↑round → Data

value:
fun {BatchIndex : ℕ → Type u_1} {Data : Type u_2} (iteration : ℕ)
  (trace : (i : ℕ) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data i) (round : Fin iteration)
  (a : BatchIndex ↑round) =>
  trace (↑round + 1) a
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)

AppliedModelingLib.heterogeneousBatchTraceOfPrefix

Declaration location: reusable library

Lean declaration body

```
type:
{BatchIndex : ℕ → Type u_1} →
{Data : Type u_2} →
(iteration : ℕ) →
((i : 1 (Finset.lic iteration)) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data ↑i) →
(round : Fin iteration) → BatchIndex ↑round → Data

value:
fun {BatchIndex : ℕ → Type u_1} {Data : Type u_2} (iteration : ℕ)
  (historyPrefix :
    (i : 1 (Finset.lic iteration)) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data ↑i)
  (round : Fin iteration) (a : BatchIndex ↑round) =>
  historyPrefix (↑round + 1, AppliedModelingLib.heterogeneousBatchTraceOfPrefix._proof_1 iteration round) a
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)

AppliedModelingLib.heterogeneousBatchTraceBadEventPair

Declaration location: reusable library

Lean declaration body

```
type:
{Data : Type u_1} →
(BatchIndex : ℕ → Type u_2) →
(iteration : ℕ) →
  AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration → (BatchIndex iteration → Data) → Prop) →
(iteration : ℕ) →
  ((i : 1 (Finset.lic iteration)) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data ↑i) ×
  AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data (iteration + 1) →
  Prop

value:
fun {Data : Type u_1} (BatchIndex : ℕ → Type u_2)
  (bad :
    (iteration : ℕ) →
      AppliedModelingLib.HeterogeneousBatchTrace BatchIndex Data iteration → (BatchIndex iteration → Data) → Prop)
  (iteration : ℕ)
  (a :
    ((i : 1 (Finset.lic iteration)) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data ↑i) ×
    AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data (iteration + 1)) =>
  {pair :
    ((i : 1 (Finset.lic iteration)) → AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data ↑i) ×
    AppliedModelingLib.HeterogeneousBatchTraceCoordinate BatchIndex Data (iteration + 1) |
    bad iteration (AppliedModelingLib.heterogeneousBatchTraceOfPrefix iteration pair.1) pair.2}
  a
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)
- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.heterogeneousBatchTraceOfPrefix](#)

GHKR22BiasBounties.AEOptimalAgainstEveryIntegrableModelForRisk

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} → {Y : Type u_2} → [inst : MeasurableSpace X] → MeasureTheory.Measure X → (X → Y → ℝ) → (X → Y) → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] (law : MeasureTheory.Measure X) (risk : X → Y → ℝ)
  (current : X → Y) =>
  ∀ (alternative : X → Y),
    MeasureTheory.Integrable (fun (x : X) => risk x (alternative x)) law →
    (fun (x : X) => risk x (current x)) ≤m[law] fun (x : X) => risk x (alternative x)
```

GHKR22BiasBounties.CertificateDecision

Declaration location: paper-local

Lean declaration body

```
type:
Type

constructor GHKR22BiasBounties.CertificateDecision.rejected:
GHKR22BiasBounties.CertificateDecision

constructor GHKR22BiasBounties.CertificateDecision.accepted:
GHKR22BiasBounties.CertificateDecision
```

GHKR22BiasBounties.MeasurableBoundedLoss

Declaration location: paper-local

Lean declaration body

```
type:
{Y : Type u_1} → [MeasurableSpace Y] → GHKR22BiasBounties.BoundedLoss Y → Prop

value:
fun {Y : Type u_1} [MeasurableSpace Y] (loss : GHKR22BiasBounties.BoundedLoss Y) =>
  Measurable fun (pair : Y × Y) => loss.value pair.1 pair.2
```

Direct retained dependencies

- [AppliedModelingLib.Learning.Prediction.BoundedLoss](#)
- [GHKR22BiasBounties.BoundedLoss](#)

GHKR22BiasBounties.MeasurableGroup

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} → [MeasurableSpace X] → GHKR22BiasBounties.Group X → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (group : GHKR22BiasBounties.Group X) => Measurable group
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)

GHKR22BiasBounties.MeasureTrainingBatchIndex

Declaration location: paper-local

Lean declaration body

```
type:
  ℕ → ℕ → Type

value:
  fun (n_round : ℕ) => Fin n
```

GHKR22BiasBounties.CandidatePair

Declaration location: paper-local

Lean declaration body

```
type:
  Type u_1 → Type u_2 → Type (max u_1 u_2 u_1)

value:
  fun (X : Type u_1) (Y : Type u_2) => GHKR22BiasBounties.Group X × GHKR22BiasBounties.Model X Y
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.CandidateClassContainedIn

Declaration location: paper-local

Lean declaration body

```
type:
  {X : Type u_1} →
  Set (GHKR22BiasBounties.CandidatePair X Bool) →
  Set (GHKR22BiasBounties.Group X) → Set (GHKR22BiasBounties.Model X Bool) → Prop

value:
  fun {X : Type u_1} (C : Set (GHKR22BiasBounties.CandidatePair X Bool)) (G : Set (GHKR22BiasBounties.Group X))
    (H : Set (GHKR22BiasBounties.Model X Bool)) =>
    ∀ candidate ∈ C, candidate.1 ∈ G ∧ candidate.2 ∈ H
```

Direct retained dependencies

- [GHKR22BiasBounties.CandidatePair](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.CertificatePair

Declaration location: paper-local

Lean declaration body

```
field group:
  {X : Type u_1} → GHKR22BiasBounties.CertificatePair X → GHKR22BiasBounties.Group X

field replacement:
  {X : Type u_1} → GHKR22BiasBounties.CertificatePair X → GHKR22BiasBounties.Model X Bool
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.FalsifyState

Declaration location: paper-local

Lean declaration body

```
field current:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.FalsifyState X Y → GHKR22BiasBounties.Model X Y

field accepted:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.FalsifyState X Y → ℕ

field reverseTranscript:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.FalsifyState X Y → List GHKR22BiasBounties.CertificateDecision
```

Direct retained dependencies

- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.MeasurableModel

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} → {Y : Type u_2} → [MeasurableSpace X] → [MeasurableSpace Y] → GHKR22BiasBounties.Model X Y → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (model : GHKR22BiasBounties.Model X Y) =>
  Measurable model
```

Direct retained dependencies

- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.MeasureCandidateClassMeasurable

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} → [MeasurableSpace X] → [MeasurableSpace Y] → Set (GHKR22BiasBounties.CandidatePair X Y) → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y]
  (C : Set (GHKR22BiasBounties.CandidatePair X Y)) =>
  ∀ candidate ∈ C, GHKR22BiasBounties.MeasurableGroup candidate.1 ∧ GHKR22BiasBounties.MeasurableModel candidate.2
```

Direct retained dependencies

- [GHKR22BiasBounties.CandidatePair](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasurableGroup](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.MonotonePublication

Declaration location: paper-local

Lean declaration body

```
field output:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.MonotonePublication X Y → GHKR22BiasBounties.Model X Y

field protectedGroups:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.MonotonePublication X Y → List (GHKR22BiasBounties.Group X)

field pastModels:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.MonotonePublication X Y → List (GHKR22BiasBounties.Model X Y)
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.OptimalOnEveryMeasurableGroupForRisk

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} → {Y : Type u_2} → [inst : MeasurableSpace X] → MeasureTheory.Measure X → (X → Y → ℝ) → (X → Y) → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] (law : MeasureTheory.Measure X) (risk : X → Y → ℝ)
  (current : X → Y) =>
  ∀ (group : Set X),
    MeasurableSet group →
    ∀ (alternative : X → Y),
      MeasureTheory.Integrable (fun (x : X) => risk x (alternative x)) law →
      ∫ (x : X) in group, risk x (current x) ∂law ≤ ∫ (x : X) in group, risk x (alternative x) ∂law
```

GHKR22BiasBounties.PairAdmissible

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
Set (GHKR22BiasBounties.Group X) → Set (GHKR22BiasBounties.Model X Bool) → GHKR22BiasBounties.CertificatePair X → Prop

value:
fun {X : Type u_1} (G : Set (GHKR22BiasBounties.Group X)) (H : Set (GHKR22BiasBounties.Model X Bool))
  (pair : GHKR22BiasBounties.CertificatePair X) =>
  pair.group ∈ G ∧ pair.replacement ∈ H
```

Direct retained dependencies

- [GHKR22BiasBounties.CertificatePair](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.ProposedUpdate

Declaration location: paper-local

Lean declaration body

```
field group:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.ProposedUpdate X Y → GHKR22BiasBounties.Group X

field replacement:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.ProposedUpdate X Y → GHKR22BiasBounties.Model X Y
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy

Declaration location: paper-local

Lean declaration body

```
type:
Type u_1 → Type u_2 → Type (max (max 0 u_2) u_1)

value:
fun (X : Type u_1) (Y : Type u_2) => List GHKR22BiasBounties.CertificateDecision → GHKR22BiasBounties.ProposedUpdate X Y
```

Direct retained dependencies

- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.ProposedUpdate](#)

GHKR22BiasBounties.AdaptiveProposalStrategy

Declaration location: paper-local

Lean declaration body

```
type:
Type u_1 → Type u_2 → Type (max (max 0 u_2) u_1)

value:
fun (X : Type u_1) (Y : Type u_2) => List GHKR22BiasBounties.CertificateDecision → GHKR22BiasBounties.ProposedUpdate X Y
```

Direct retained dependencies

- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.ProposedUpdate](#)

GHKR22BiasBounties.MeasureProposalMeasurable

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} → [MeasurableSpace X] → [MeasurableSpace Y] → GHKR22BiasBounties.ProposedUpdate X Y → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y]
(proposal : GHKR22BiasBounties.ProposedUpdate X Y) =>
GHKR22BiasBounties.MeasurableGroup proposal.group ∧ GHKR22BiasBounties.MeasurableModel proposal.replacement
```

Direct retained dependencies

- [GHKR22BiasBounties.MeasurableGroup](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.ProposedUpdate](#)

GHKR22BiasBounties.MonotoneBountyState

Declaration location: paper-local

Lean declaration body

```
field current:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.MonotoneBountyState X Y → GHKR22BiasBounties.Model X Y

field accepted:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.MonotoneBountyState X Y → ℕ

field externalProcessed:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.MonotoneBountyState X Y → ℕ

field protectedGroups:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.MonotoneBountyState X Y → List (GHKR22BiasBounties.Group X)

field publicModels:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.MonotoneBountyState X Y → List (GHKR22BiasBounties.Model X Y)

field remainingRepairs:
{X : Type u_1} →
{Y : Type u_2} → GHKR22BiasBounties.MonotoneBountyState X Y → List (GHKR22BiasBounties.ProposedUpdate X Y)

field rejectedRepairsReverse:
{X : Type u_1} →
{Y : Type u_2} → GHKR22BiasBounties.MonotoneBountyState X Y → List (GHKR22BiasBounties.ProposedUpdate X Y)

field publicationsReverse:
{X : Type u_1} →
{Y : Type u_2} → GHKR22BiasBounties.MonotoneBountyState X Y → List (GHKR22BiasBounties.MonotonePublication X Y)

field reverseTranscript:
{X : Type u_1} →
{Y : Type u_2} → GHKR22BiasBounties.MonotoneBountyState X Y → List GHKR22BiasBounties.CertificateDecision
```

Direct retained dependencies

- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.MonotonePublication](#)
- [GHKR22BiasBounties.ProposedUpdate](#)

GHKR22BiasBounties.SparseTranscriptCode

Declaration location: paper-local

Lean declaration body

```
type:
  ℕ → ℕ → Type

value:
  fun (U K : ℕ) => Fin K → Fin (U + 1)
```

GHKR22BiasBounties.AdaptiveQueryIndex

Declaration location: paper-local

Lean declaration body

```
type:
  ℕ → ℕ → Type

value:
  fun (U K : ℕ) => Fin U × GHKR22BiasBounties.SparseTranscriptCode U K
```

Direct retained dependencies

- [GHKR22BiasBounties.SparseTranscriptCode](#)

GHKR22BiasBounties.Submission

Declaration location: paper-local

Lean declaration body

```
field current:
  {X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.Submission X Y → GHKR22BiasBounties.Model X Y

field group:
  {X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.Submission X Y → GHKR22BiasBounties.Group X

field replacement:
  {X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.Submission X Y → GHKR22BiasBounties.Model X Y
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.AdaptiveSubmissionStrategy

Declaration location: paper-local

Lean declaration body

```
type:
  Type u_1 → Type u_2 → Type (max (max 0 u_2) u_1)

value:
  fun (X : Type u_1) (Y : Type u_2) => List GHKR22BiasBounties.CertificateDecision → GHKR22BiasBounties.Submission X Y
```


Direct retained dependencies

- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.Submission](#)

GHKR22BiasBounties.MeasureSubmissionMeasurable

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} → {Y : Type u_2} → [MeasurableSpace X] → [MeasurableSpace Y] → GHKR22BiasBounties.Submission X Y → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y]
(submission : GHKR22BiasBounties.Submission X Y) =>
GHKR22BiasBounties.MeasurableModel submission.current ∧
GHKR22BiasBounties.MeasurableGroup submission.group ∧ GHKR22BiasBounties.MeasurableModel submission.replacement
```

Direct retained dependencies

- [GHKR22BiasBounties.MeasurableGroup](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.Submission](#)

GHKR22BiasBounties.TernaryLabel

Declaration location: paper-local

Lean declaration body

```
type:
Type

constructor GHKR22BiasBounties.TernaryLabel.zero:
GHKR22BiasBounties.TernaryLabel

constructor GHKR22BiasBounties.TernaryLabel.one:
GHKR22BiasBounties.TernaryLabel

constructor GHKR22BiasBounties.TernaryLabel.defer:
GHKR22BiasBounties.TernaryLabel
```

GHKR22BiasBounties.TernaryPredictor

Declaration location: paper-local

Lean declaration body

```
type:
Type u_1 → Type u_1

value:
fun (X : Type u_1) => X → GHKR22BiasBounties.TernaryLabel
```

Direct retained dependencies

- [GHKR22BiasBounties.TernaryLabel](#)

GHKR22BiasBounties.TrainByOptOutcome

Declaration location: paper-local

Lean declaration body

```
type:
Type u_1 → Type u_2 → Type (max u_1 u_2)

constructor GHKR22BiasBounties.TrainByOptOutcome.running:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.Model X Y → ℕ → GHKR22BiasBounties.TrainByOptOutcome X Y

constructor GHKR22BiasBounties.TrainByOptOutcome.stopped:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.Model X Y → ℕ → GHKR22BiasBounties.TrainByOptOutcome X Y
```

Direct retained dependencies

- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.TrainByOptOutcome.calls

Declaration location: paper-local

Lean declaration body

```
type:
  {X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.TrainByOptOutcome X Y → ℕ
value:
  fun {X : Type u_1} {Y : Type u_2} (a : GHKR22BiasBounties.TrainByOptOutcome X Y) =>
    match a with
    | GHKR22BiasBounties.TrainByOptOutcome.running model count => count
    | GHKR22BiasBounties.TrainByOptOutcome.stopped model count => count
```

Direct retained dependencies

- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TrainByOptOutcome](#)

GHKR22BiasBounties.TrainByOptOutcome.output

Declaration location: paper-local

Lean declaration body

```
type:
  {X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.TrainByOptOutcome X Y → X → Y
value:
  fun {X : Type u_1} {Y : Type u_2} (a : GHKR22BiasBounties.TrainByOptOutcome X Y) (a_1 : X) =>
    (match a with
    | GHKR22BiasBounties.TrainByOptOutcome.running model calls => model
    | GHKR22BiasBounties.TrainByOptOutcome.stopped model calls => model)
  a_1
```

Direct retained dependencies

- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TrainByOptOutcome](#)

GHKR22BiasBounties.TrainingOracle

Declaration location: paper-local

Lean declaration body

```
type:
  Type u_1 → Type u_2 → ℕ → Type (max u_2 u_1)
value:
  fun (X : Type u_1) (Y : Type u_2) (n : ℕ) =>
    GHKR22BiasBounties.Model X Y → (Fin n → X × Y) → GHKR22BiasBounties.CandidatePair X Y
```

Direct retained dependencies

- [GHKR22BiasBounties.CandidatePair](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.Update

Declaration location: paper-local

Lean declaration body

```
field group:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.Update X Y → GHKR22BiasBounties.Group X

field replacement:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.Update X Y → GHKR22BiasBounties.Model X Y

field mu:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.Update X Y → ℝ

field Delta:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.Update X Y → ℝ
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.VCSourceEventsMeasurable

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) → Set (AppliedModelingLib.Statistics.BinaryClassifier X) → ℕ → ℕ → ℝ → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
(concepts : Set (AppliedModelingLib.Statistics.BinaryClassifier X)) (d count : ℕ) (delta : ℝ) =>
MeasurableSet
(AppliedModelingLib.Statistics.finiteIIDVCPopulationUnderestimateBadEvent law concepts count
(AppliedModelingLib.Statistics.finiteIIDVCSourceConfidenceRadius d count delta)) ∧
MeasurableSet
(AppliedModelingLib.Statistics.finiteIIDVCPopulationOverestimateBadEvent law concepts count
(AppliedModelingLib.Statistics.finiteIIDVCSourceConfidenceRadius d count delta)) ∧
MeasurableSet
(AppliedModelingLib.Statistics.finiteIIDVCTwoSampleBadEvent concepts count
(AppliedModelingLib.Statistics.finiteIIDVCSourceConfidenceRadius d count delta -
AppliedModelingLib.Statistics.finiteIIDVCSourceConfidenceRadius d count delta / 4)) ∧
MeasurableSet
(AppliedModelingLib.Statistics.finiteIIDVCPairSwapBadEvent concepts count
(AppliedModelingLib.Statistics.finiteIIDVCSourceConfidenceRadius d count delta -
AppliedModelingLib.Statistics.finiteIIDVCSourceConfidenceRadius d count delta / 4)) ∧
MeasurableSet
(AppliedModelingLib.Statistics.finiteIIDVCTwoSampleProductBadEvent concepts count
(AppliedModelingLib.Statistics.finiteIIDVCSourceConfidenceRadius d count delta -
AppliedModelingLib.Statistics.finiteIIDVCSourceConfidenceRadius d count delta / 4))
```

Direct retained dependencies

- [AppliedModelingLib.Statistics.BinaryClassifier](#)
- [AppliedModelingLib.Statistics.finiteIIDVCPairSwapBadEvent](#)
- [AppliedModelingLib.Statistics.finiteIIDVCPopulationOverestimateBadEvent](#)
- [AppliedModelingLib.Statistics.finiteIIDVCPopulationUnderestimateBadEvent](#)
- [AppliedModelingLib.Statistics.finiteIIDVCSourceConfidenceRadius](#)
- [AppliedModelingLib.Statistics.finiteIIDVCTwoSampleBadEvent](#)
- [AppliedModelingLib.Statistics.finiteIIDVCTwoSampleProductBadEvent](#)

GHKR22BiasBounties.binaryZeroOneLoss

Declaration location: paper-local

Lean declaration body

```
type:
GHKR22BiasBounties.BoundedLoss Bool

value:
{ value := fun (prediction truth : Bool) => if prediction = truth then 0 else 1,
nonneg := GHKR22BiasBounties.binaryZeroOneLoss._proof_1, le_one := GHKR22BiasBounties.binaryZeroOneLoss._proof_2 }
```

Direct retained dependencies

- [AppliedModelingLib.Learning.Prediction.BoundedLoss](#)
- [GHKR22BiasBounties.BoundedLoss](#)

GHKR22BiasBounties.datumLoss

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.BoundedLoss Y → GHKR22BiasBounties.Model X Y → X × Y → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} (loss : GHKR22BiasBounties.BoundedLoss Y) (f : GHKR22BiasBounties.Model X Y)
  (datum : X × Y) =>
  loss.value (f datum.1) datum.2
```

Direct retained dependencies

- [AppliedModelingLib.Learning.Prediction.BoundedLoss](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.disagreementLabel

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} → GHKR22BiasBounties.Model X Bool → X × Bool → Bool

value:
fun {X : Type u_1} (replacement : GHKR22BiasBounties.Model X Bool) (datum : X × Bool) =>
  decide (replacement datum.1 = datum.2)
```

Direct retained dependencies

- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.initialFalsifyState

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.FalsifyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} (f : GHKR22BiasBounties.Model X Y) =>
  { current := f, accepted := 0, reverseTranscript := [] }
```

Direct retained dependencies

- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.FalsifyState](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.initialMonotoneBountyState

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.MonotoneBountyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} (initial : GHKR22BiasBounties.Model X Y) =>
{ current := initial, accepted := 0, externalProcessed := 0, protectedGroups := [], publicModels := [initial],
  remainingRepairs := [], rejectedRepairsReverse := [], publicationsReverse := [], reverseTranscript := [] }
```

Direct retained dependencies

- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.MonotoneBountyState](#)
- [GHKR22BiasBounties.MonotonePublication](#)
- [GHKR22BiasBounties.ProposedUpdate](#)

GHKR22BiasBounties.ListUpdateClass

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
GHKR22BiasBounties.Model X Bool →
Set (GHKR22BiasBounties.Group X) →
Set (GHKR22BiasBounties.Model X Bool) → AppliedModelingLib.Statistics.BinaryClassifier X → Prop

value:
fun {X : Type u_1} (current : GHKR22BiasBounties.Model X Bool) (G : Set (GHKR22BiasBounties.Group X))
(H : Set (GHKR22BiasBounties.Model X Bool)) (a : AppliedModelingLib.Statistics.BinaryClassifier X) =>
{ updated : AppliedModelingLib.Statistics.BinaryClassifier X |
  ∃ g ∈ G, ∃ h ∈ H, updated = GHKR22BiasBounties.listUpdate current g h }
a
```

Direct retained dependencies

- [AppliedModelingLib.Statistics.BinaryClassifier](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.listUpdate](#)

GHKR22BiasBounties.falsifyStepFromDecision

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
GHKR22BiasBounties.FalsifyState X Y →
GHKR22BiasBounties.ProposedUpdate X Y →
GHKR22BiasBounties.CertificateDecision → GHKR22BiasBounties.FalsifyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} (state : GHKR22BiasBounties.FalsifyState X Y)
(proposal : GHKR22BiasBounties.ProposedUpdate X Y) (decision : GHKR22BiasBounties.CertificateDecision) =>
match decision with
| GHKR22BiasBounties.CertificateDecision.rejected =>
{ current := state.current, accepted := state.accepted, reverseTranscript := decision :: state.reverseTranscript }
| GHKR22BiasBounties.CertificateDecision.accepted =>
{ current := GHKR22BiasBounties.listUpdate state.current proposal.group proposal.replacement,
  accepted := state.accepted + 1, reverseTranscript := decision :: state.reverseTranscript }
```

Direct retained dependencies

- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.FalsifyState](#)
- [GHKR22BiasBounties.ProposedUpdate](#)
- [GHKR22BiasBounties.listUpdate](#)

GHKR22BiasBounties.falsifyReplayAux

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
  GHKR22BiasBounties.AdaptiveProposalStrategy X Y →
  GHKR22BiasBounties.FalsifyState X Y →
  List GHKR22BiasBounties.CertificateDecision → GHKR22BiasBounties.FalsifyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} (strategy : GHKR22BiasBounties.AdaptiveProposalStrategy X Y)
(a : GHKR22BiasBounties.FalsifyState X Y) (a_1 : List GHKR22BiasBounties.CertificateDecision) =>
List.brecOn (motive := fun (x : List GHKR22BiasBounties.CertificateDecision) =>
  GHKR22BiasBounties.FalsifyState X Y → GHKR22BiasBounties.FalsifyState X Y) a_1
(GHKR22BiasBounties.falsifyReplayAux._f strategy) a
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveProposalStrategy](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.FalsifyState](#)
- [GHKR22BiasBounties.ProposedUpdate](#)
- [GHKR22BiasBounties.falsifyStepFromDecision](#)

GHKR22BiasBounties.falsifyStateFromTranscript

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
  GHKR22BiasBounties.Model X Y →
  GHKR22BiasBounties.AdaptiveProposalStrategy X Y →
  List GHKR22BiasBounties.CertificateDecision → GHKR22BiasBounties.FalsifyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} (initial : GHKR22BiasBounties.Model X Y)
(strategy : GHKR22BiasBounties.AdaptiveProposalStrategy X Y)
(transcript : List GHKR22BiasBounties.CertificateDecision) =>
  GHKR22BiasBounties.falsifyReplayAux strategy (GHKR22BiasBounties.initialFalsifyState initial) transcript
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveProposalStrategy](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.FalsifyState](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.falsifyReplayAux](#)
- [GHKR22BiasBounties.initialFalsifyState](#)

GHKR22BiasBounties.listUpdateVCTotal

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N}$ 

value:
fun (dG dH :  $\mathbb{N}$ ) => dG + 1 + (dH + 1)
```

GHKR22BiasBounties.measureCoordinateAscentFuel

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{N}$ 

value:
fun (epsilon :  $\mathbb{R}$ ) => [2 / epsilon]+ - 1
```

GHKR22BiasBounties.measureDisagreementLossIntegrand

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
GHKR22BiasBounties.Model X Bool → GHKR22BiasBounties.Model X Bool → GHKR22BiasBounties.Group X → X × Bool →  $\mathbb{R}$ 

value:
fun {X : Type u_1} (current replacement : GHKR22BiasBounties.Model X Bool) (g : GHKR22BiasBounties.Group X)
  (datum : X × Bool) =>
  if current datum.1 = replacement datum.1 then 0
  else GHKR22BiasBounties.binaryZeroOneLoss.value (g datum.1) (GHKR22BiasBounties.disagreementLabel replacement datum)
```

Direct retained dependencies

- [AppliedModelingLib.Learning.Prediction.BoundedLoss](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.binaryZeroOneLoss](#)
- [GHKR22BiasBounties.disagreementLabel](#)

GHKR22BiasBounties.measureDisagreementGroupLoss

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) →
GHKR22BiasBounties.Model X Bool → GHKR22BiasBounties.Model X Bool → GHKR22BiasBounties.Group X →  $\mathbb{R}$ 

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
  (current replacement : GHKR22BiasBounties.Model X Bool) (g : GHKR22BiasBounties.Group X) =>
  ∫ (datum : X × Bool), GHKR22BiasBounties.measureDisagreementLossIntegrand current replacement g datum dlaw
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.measureDisagreementLossIntegrand](#)

GHKR22BiasBounties.MeasureDisagreementERM

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) →
GHKR22BiasBounties.Model X Bool →
GHKR22BiasBounties.Model X Bool → Set (GHKR22BiasBounties.Group X) → GHKR22BiasBounties.Group X → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
(current replacement : GHKR22BiasBounties.Model X Bool) (G : Set (GHKR22BiasBounties.Group X))
(gStar : GHKR22BiasBounties.Group X) =>
gStar ∈ G ∧
∀ g ∈ G,
GHKR22BiasBounties.measureDisagreementGroupLoss law current replacement gStar ≤
GHKR22BiasBounties.measureDisagreementGroupLoss law current replacement g
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.measureDisagreementGroupLoss](#)

GHKR22BiasBounties.MeasureGroupRestrictedERM

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) →
GHKR22BiasBounties.Group X → Set (GHKR22BiasBounties.Model X Bool) → GHKR22BiasBounties.Model X Bool → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool)) (g : GHKR22BiasBounties.Group X)
(H : Set (GHKR22BiasBounties.Model X Bool)) (hStar : GHKR22BiasBounties.Model X Bool) =>
hStar ∈ H ∧
∀ h ∈ H,
GHKR22BiasBounties.measureGroupLossNumerator law GHKR22BiasBounties.binaryZeroOneLoss hStar g ≤
GHKR22BiasBounties.measureGroupLossNumerator law GHKR22BiasBounties.binaryZeroOneLoss h g
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.binaryZeroOneLoss](#)
- [GHKR22BiasBounties.measureGroupLossNumerator](#)

GHKR22BiasBounties.MeasureApproxGroupwiseMonotoneOnHistory

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
ℝ →
GHKR22BiasBounties.Model X Y →
List (GHKR22BiasBounties.Group X) → List (GHKR22BiasBounties.Model X Y) → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
(loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) (current : GHKR22BiasBounties.Model X Y)
(groups : List (GHKR22BiasBounties.Group X)) (pastModels : List (GHKR22BiasBounties.Model X Y)) =>
∀ group ∈ groups,
∀ past ∈ pastModels,
0 < GHKR22BiasBounties.measureGroupMass law group →
GHKR22BiasBounties.measureGroupLoss law loss current group ≤
GHKR22BiasBounties.measureGroupLoss law loss past group +
epsilon / GHKR22BiasBounties.measureGroupMass law group
```


Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.measureGroupLoss](#)
- [GHKR22BiasBounties.measureGroupMass](#)

GHKR22BiasBounties.MeasureCheckerDecisionGuaranteed

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
ℝ → GHKR22BiasBounties.Submission X Y → GHKR22BiasBounties.CertificateDecision → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) (submission : GHKR22BiasBounties.Submission X Y)
  (decision : GHKR22BiasBounties.CertificateDecision) =>
  (decision = GHKR22BiasBounties.CertificateDecision.rejected →
    ∀ (mu Delta : ℝ),
      epsilon ≤ mu * Delta →
        ¬GHKR22BiasBounties.MeasureCertificateOfSuboptimality law loss submission.current submission.group
        submission.replacement mu Delta) ∧
  (decision = GHKR22BiasBounties.CertificateDecision.accepted →
    ∃ (mu : ℝ) (Delta : ℝ),
      GHKR22BiasBounties.MeasureCertificateOfSuboptimality law loss submission.current submission.group
        submission.replacement mu Delta ∧
        epsilon / 2 ≤ mu * Delta)
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.MeasureCertificateOfSuboptimality](#)
- [GHKR22BiasBounties.Submission](#)

GHKR22BiasBounties.MeasureMonotonePublicationsSound

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y → ℝ → GHKR22BiasBounties.MonotoneBountyState X Y → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) (state : GHKR22BiasBounties.MonotoneBountyState X Y) =>
  ∀ publication ∈ state.publicationsReverse,
    GHKR22BiasBounties.MeasureApproxGroupwiseMonotoneOnHistory law loss epsilon publication.output
      publication.protectedGroups publication.pastModels
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.MeasureApproxGroupwiseMonotoneOnHistory](#)
- [GHKR22BiasBounties.MonotoneBountyState](#)
- [GHKR22BiasBounties.MonotonePublication](#)

GHKR22BiasBounties.MeasureValidUpdateSequence

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
ℝ → GHKR22BiasBounties.Model X Y → List (GHKR22BiasBounties.Update X Y) → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
(loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) (a : GHKR22BiasBounties.Model X Y)
(a_1 : List (GHKR22BiasBounties.Update X Y)) =>
List.brecOn (motive := fun (x : List (GHKR22BiasBounties.Update X Y)) => GHKR22BiasBounties.Model X Y → Prop) a_1
(GHKR22BiasBounties.MeasureValidUpdateSequence._f law loss epsilon) a
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.MeasurableGroup](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.MeasureCertificateOfSuboptimality](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.Update](#)
- [GHKR22BiasBounties.listUpdate](#)

GHKR22BiasBounties.measureCertificateImprovementScore

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.Group X → GHKR22BiasBounties.Model X Y → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
(loss : GHKR22BiasBounties.BoundedLoss Y) (current : GHKR22BiasBounties.Model X Y)
(group : GHKR22BiasBounties.Group X) (replacement : GHKR22BiasBounties.Model X Y) =>
GHKR22BiasBounties.measureGroupMass law group *
(GHKR22BiasBounties.measureGroupLoss law loss current group -
GHKR22BiasBounties.measureGroupLoss law loss replacement group)
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.measureGroupLoss](#)
- [GHKR22BiasBounties.measureGroupMass](#)

GHKR22BiasBounties.MeasureBinaryCertificateObjective

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
  [inst : MeasurableSpace X] →
    MeasureTheory.Measure (X × Bool) →
      GHKR22BiasBounties.Model X Bool → GHKR22BiasBounties.Group X → GHKR22BiasBounties.Model X Bool → ℝ

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
  (current : GHKR22BiasBounties.Model X Bool) (g : GHKR22BiasBounties.Group X)
  (h : GHKR22BiasBounties.Model X Bool) =>
  GHKR22BiasBounties.measureCertificateImprovementScore law GHKR22BiasBounties.binaryZeroOneLoss current g h
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.binaryZeroOneLoss](#)
- [GHKR22BiasBounties.measureCertificateImprovementScore](#)

GHKR22BiasBounties.MeasureCoordinateBestResponses

Declaration location: paper-local

Lean declaration body

```
field bestGroup:
{X : Type u_1} →
  [inst : MeasurableSpace X] →
    {law : MeasureTheory.Measure (X × Bool)} →
      {current : GHKR22BiasBounties.Model X Bool} →
        {G : Set (GHKR22BiasBounties.Group X)} →
          {H : Set (GHKR22BiasBounties.Model X Bool)} →
            GHKR22BiasBounties.MeasureCoordinateBestResponses law current G H →
              GHKR22BiasBounties.Model X Bool → GHKR22BiasBounties.Group X

field bestModel:
{X : Type u_1} →
  [inst : MeasurableSpace X] →
    {law : MeasureTheory.Measure (X × Bool)} →
      {current : GHKR22BiasBounties.Model X Bool} →
        {G : Set (GHKR22BiasBounties.Group X)} →
          {H : Set (GHKR22BiasBounties.Model X Bool)} →
            GHKR22BiasBounties.MeasureCoordinateBestResponses law current G H →
              GHKR22BiasBounties.Group X → GHKR22BiasBounties.Model X Bool

field bestGroup_mem:
∀ {X : Type u_1} [inst : MeasurableSpace X] {law : MeasureTheory.Measure (X × Bool)}
  {current : GHKR22BiasBounties.Model X Bool} {G : Set (GHKR22BiasBounties.Group X)}
  {H : Set (GHKR22BiasBounties.Model X Bool)} (self : GHKR22BiasBounties.MeasureCoordinateBestResponses law current G H)
  (h : GHKR22BiasBounties.Model X Bool), self.bestGroup h ∈ G

field bestModel_mem:
∀ {X : Type u_1} [inst : MeasurableSpace X] {law : MeasureTheory.Measure (X × Bool)}
  {current : GHKR22BiasBounties.Model X Bool} {G : Set (GHKR22BiasBounties.Group X)}
  {H : Set (GHKR22BiasBounties.Model X Bool)} (self : GHKR22BiasBounties.MeasureCoordinateBestResponses law current G H)
  (g : GHKR22BiasBounties.Group X), self.bestModel g ∈ H

field bestGroup_max:
∀ {X : Type u_1} [inst : MeasurableSpace X] {law : MeasureTheory.Measure (X × Bool)}
  {current : GHKR22BiasBounties.Model X Bool} {G : Set (GHKR22BiasBounties.Group X)}
  {H : Set (GHKR22BiasBounties.Model X Bool)} (self : GHKR22BiasBounties.MeasureCoordinateBestResponses law current G H)
  (h : GHKR22BiasBounties.Model X Bool),
  ∀ g ∈ G,
    GHKR22BiasBounties.MeasureBinaryCertificateObjective law current g h ≤
      GHKR22BiasBounties.MeasureBinaryCertificateObjective law current (self.bestGroup h) h

field bestModel_max:
∀ {X : Type u_1} [inst : MeasurableSpace X] {law : MeasureTheory.Measure (X × Bool)}
  {current : GHKR22BiasBounties.Model X Bool} {G : Set (GHKR22BiasBounties.Group X)}
  {H : Set (GHKR22BiasBounties.Model X Bool)} (self : GHKR22BiasBounties.MeasureCoordinateBestResponses law current G H)
  (g : GHKR22BiasBounties.Group X),
  ∀ h ∈ H,
    GHKR22BiasBounties.MeasureBinaryCertificateObjective law current g h ≤
      GHKR22BiasBounties.MeasureBinaryCertificateObjective law current g (self.bestModel h)
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasureBinaryCertificateObjective](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.MeasureDerivedCertificateMaximizer

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) →
GHKR22BiasBounties.Model X Bool →
Set (GHKR22BiasBounties.TernaryPredictor X) → GHKR22BiasBounties.TernaryPredictor X → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
(current : GHKR22BiasBounties.Model X Bool) (K : Set (GHKR22BiasBounties.TernaryPredictor X))
(pStar : GHKR22BiasBounties.TernaryPredictor X) =>
pStar ∈ K ∧
∀ p ∈ K,
GHKR22BiasBounties.measureCertificateImprovementScore law GHKR22BiasBounties.binaryZeroOneLoss current
(GHKR22BiasBounties.derivedGroup p) (GHKR22BiasBounties.derivedModel p) ≤
GHKR22BiasBounties.measureCertificateImprovementScore law GHKR22BiasBounties.binaryZeroOneLoss current
(GHKR22BiasBounties.derivedGroup pStar) (GHKR22BiasBounties.derivedModel pStar)
```

Direct retained dependencies

- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TernaryPredictor](#)
- [GHKR22BiasBounties.binaryZeroOneLoss](#)
- [GHKR22BiasBounties.derivedGroup](#)
- [GHKR22BiasBounties.derivedModel](#)
- [GHKR22BiasBounties.measureCertificateImprovementScore](#)

GHKR22BiasBounties.MeasureEpsilonCoordinatewiseLocal

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) →
GHKR22BiasBounties.Model X Bool →
Set (GHKR22BiasBounties.Group X) →
Set (GHKR22BiasBounties.Model X Bool) → ℝ → GHKR22BiasBounties.CertificatePair X → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
(current : GHKR22BiasBounties.Model X Bool) (G : Set (GHKR22BiasBounties.Group X))
(H : Set (GHKR22BiasBounties.Model X Bool)) (epsilon : ℝ) (pair : GHKR22BiasBounties.CertificatePair X) =>
GHKR22BiasBounties.PairAdmissible G H pair ∧
(∀ h ∈ H,
GHKR22BiasBounties.MeasureBinaryCertificateObjective law current pair.group h ≤
GHKR22BiasBounties.MeasureBinaryCertificateObjective law current pair.group pair.replacement + epsilon) ∧
∀ g ∈ G,
GHKR22BiasBounties.MeasureBinaryCertificateObjective law current g pair.replacement ≤
GHKR22BiasBounties.MeasureBinaryCertificateObjective law current pair.group pair.replacement + epsilon
```

Direct retained dependencies

- [GHKR22BiasBounties.CertificatePair](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasureBinaryCertificateObjective](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.PairAdmissible](#)

GHKR22BiasBounties.MeasureImprovingCoordinateSequence

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) →
GHKR22BiasBounties.Model X Bool →
ℝ → GHKR22BiasBounties.CertificatePair X → List (GHKR22BiasBounties.CertificatePair X) → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
(current : GHKR22BiasBounties.Model X Bool) (epsilon : ℝ) (a : GHKR22BiasBounties.CertificatePair X)
(a_1 : List (GHKR22BiasBounties.CertificatePair X)) =>
List.brecOn (motive := fun (x : List (GHKR22BiasBounties.CertificatePair X)) =>
GHKR22BiasBounties.CertificatePair X → Prop) a_1
(GHKR22BiasBounties.MeasureImprovingCoordinateSequence._f law current epsilon) a
```

Direct retained dependencies

- [GHKR22BiasBounties.CertificatePair](#)
- [GHKR22BiasBounties.MeasureBinaryCertificateObjective](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.MeasureFalsifyDecisionGuarantee

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
ℝ →
GHKR22BiasBounties.FalsifyState X Y →
GHKR22BiasBounties.ProposedUpdate X Y → GHKR22BiasBounties.CertificateDecision → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
(loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) (state : GHKR22BiasBounties.FalsifyState X Y)
(proposal : GHKR22BiasBounties.ProposedUpdate X Y) (decision : GHKR22BiasBounties.CertificateDecision) =>
have next : GHKR22BiasBounties.FalsifyState X Y := GHKR22BiasBounties.falsifyStepFromDecision state proposal decision;
(decision = GHKR22BiasBounties.CertificateDecision.rejected →
  ∀ (mu Delta : ℝ),
    epsilon ≤ mu * Delta →
      ¬GHKR22BiasBounties.MeasureCertificateOfSuboptimality law loss state.current proposal.group
        proposal.replacement mu Delta) ∧
(decision = GHKR22BiasBounties.CertificateDecision.accepted →
  ∃ (mu : ℝ) (Delta : ℝ),
    GHKR22BiasBounties.MeasureCertificateOfSuboptimality law loss state.current proposal.group proposal.replacement
      mu Delta ∧
    epsilon / 2 ≤ mu * Delta ∧
    GHKR22BiasBounties.measureGroupLoss law loss next.current proposal.group =
      GHKR22BiasBounties.measureGroupLoss law loss proposal.replacement proposal.group ∧
    GHKR22BiasBounties.measureModelLoss law loss next.current ≤
      GHKR22BiasBounties.measureModelLoss law loss state.current - epsilon / 2)
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.FalsifyState](#)
- [GHKR22BiasBounties.MeasureCertificateOfSuboptimality](#)
- [GHKR22BiasBounties.ProposedUpdate](#)
- [GHKR22BiasBounties.falsifyStepFromDecision](#)
- [GHKR22BiasBounties.measureGroupLoss](#)
- [GHKR22BiasBounties.measureModelLoss](#)

GHKR22BiasBounties.measureTrainingBatchKernel

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
(law : MeasureTheory.Measure (X × Bool)) →
[MeasureTheory.IsProbabilityMeasure law] →
(n iteration : ℕ) →
ProbabilityTheory.Kernel
((i : 1 (Finset.lic iteration)) →
AppliedModelingLib.HeterogeneousBatchTraceCoordinate (GHKR22BiasBounties.MeasureTrainingBatchIndex n)
(X × Bool) ↑ i)
(AppliedModelingLib.HeterogeneousBatchTraceCoordinate (GHKR22BiasBounties.MeasureTrainingBatchIndex n)
(X × Bool) (iteration + 1))

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool)) [MeasureTheory.IsProbabilityMeasure law]
(n iteration : ℕ) =>
ProbabilityTheory.Kernel.const
((i : 1 (Finset.lic iteration)) →
AppliedModelingLib.HeterogeneousBatchTraceCoordinate (GHKR22BiasBounties.MeasureTrainingBatchIndex n) (X × Bool)
↑ i)
(AppliedModelingLib.Probability.finiteIIDSampleLaw law n)
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.Probability.finiteIIDSampleLaw](#)
- [AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace](#)
- [GHKR22BiasBounties.MeasureTrainingBatchIndex](#)

GHKR22BiasBounties.measureTrainingInfiniteTraceLaw

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
(law : MeasureTheory.Measure (X × Bool)) →
[MeasureTheory.IsProbabilityMeasure law] →
(n : ℕ) →
MeasureTheory.Measure
((i : ℕ) →
AppliedModelingLib.HeterogeneousBatchTraceCoordinate (GHKR22BiasBounties.MeasureTrainingBatchIndex n)
(X × Bool) i)

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool)) [MeasureTheory.IsProbabilityMeasure law]
(n : ℕ) =>
AppliedModelingLib.heterogeneousBatchTraceInfiniteLaw (GHKR22BiasBounties.MeasureTrainingBatchIndex n)
(GHKR22BiasBounties.measureTrainingBatchKernel law n)
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace](#)
- [AppliedModelingLib.heterogeneousBatchTraceInfiniteLaw](#)
- [GHKR22BiasBounties.MeasureTrainingBatchIndex](#)
- [GHKR22BiasBounties.measureTrainingBatchKernel](#)

GHKR22BiasBounties.monotoneBountyQueryBudget

Declaration location: paper-local

Lean declaration body

```
type:
ℝ → ℕ → ℕ

value:
fun (epsilon : ℝ) (U : ℕ) => U + GHKR22BiasBounties.checkerAcceptanceBudget epsilon ^ 3
```

Direct retained dependencies

- [GHKR22BiasBounties.checkerAcceptanceBudget](#)

GHKR22BiasBounties.numberAccepted

Declaration location: paper-local

Lean declaration body

type:
List GHKR22BiasBounties.CertificateDecision → ℕ

value:
fun (a : List GHKR22BiasBounties.CertificateDecision) => List.count GHKR22BiasBounties.CertificateDecision.accepted a

Direct retained dependencies

- [GHKR22BiasBounties.CertificateDecision](#)

GHKR22BiasBounties.paperInterfaceTernaryLabelMeasurableSpace

Declaration location: paper-local

Lean declaration body

type:
MeasurableSpace GHKR22BiasBounties.TernaryLabel

value:
T

Direct retained dependencies

- [GHKR22BiasBounties.TernaryLabel](#)

GHKR22BiasBounties.proposalSubmission

Declaration location: paper-local

Lean declaration body

type:
{X : Type u_1} →
{Y : Type u_2} →
GHKR22BiasBounties.FalsifyState X Y → GHKR22BiasBounties.ProposedUpdate X Y → GHKR22BiasBounties.Submission X Y

value:
fun {X : Type u_1} {Y : Type u_2} (state : GHKR22BiasBounties.FalsifyState X Y)
(proposal : GHKR22BiasBounties.ProposedUpdate X Y) =>
{ current := state.current, group := proposal.group, replacement := proposal.replacement }

Direct retained dependencies

- [GHKR22BiasBounties.FalsifyState](#)
- [GHKR22BiasBounties.ProposedUpdate](#)
- [GHKR22BiasBounties.Submission](#)

GHKR22BiasBounties.falsifySubmissionStrategy

Declaration location: paper-local

Lean declaration body

type:
{X : Type u_1} →
{Y : Type u_2} →
GHKR22BiasBounties.Model X Y →
GHKR22BiasBounties.AdaptiveProposalStrategy X Y →
List GHKR22BiasBounties.CertificateDecision → GHKR22BiasBounties.Submission X Y

value:
fun {X : Type u_1} {Y : Type u_2} (initial : GHKR22BiasBounties.Model X Y)
(strategy : GHKR22BiasBounties.AdaptiveProposalStrategy X Y) (a : List GHKR22BiasBounties.CertificateDecision) =>
GHKR22BiasBounties.proposalSubmission (GHKR22BiasBounties.falsifyStateFromTranscript initial strategy a) (strategy a)

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveProposalStrategy](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.falsifyStateFromTranscript](#)
- [GHKR22BiasBounties.proposalSubmission](#)

GHKR22BiasBounties.publishMonotoneBountyState

Declaration location: paper-local

Lean declaration body

```
type:
  {X : Type u_1} →
  {Y : Type u_2} → GHKR22BiasBounties.MonotoneBountyState X Y → GHKR22BiasBounties.MonotoneBountyState X Y

value:
  fun {X : Type u_1} {Y : Type u_2} (state : GHKR22BiasBounties.MonotoneBountyState X Y) =>
    have publication : GHKR22BiasBounties.MonotonePublication X Y :=
      { output := state.current, protectedGroups := state.protectedGroups, pastModels := state.publicModels };
    { current := state.current, accepted := state.accepted, externalProcessed := state.externalProcessed,
      protectedGroups := state.protectedGroups, publicModels := state.current :: state.publicModels,
      remainingRepairs := [], rejectedRepairsReverse := [],
      publicationsReverse := publication :: state.publicationsReverse, reverseTranscript := state.reverseTranscript }
```

Direct retained dependencies

- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.MonotoneBountyState](#)
- [GHKR22BiasBounties.MonotonePublication](#)
- [GHKR22BiasBounties.ProposedUpdate](#)

GHKR22BiasBounties.repairCandidates

Declaration location: paper-local

Lean declaration body

```
type:
  {X : Type u_1} →
  {Y : Type u_2} →
  List (GHKR22BiasBounties.Group X) →
  List (GHKR22BiasBounties.Model X Y) → List (GHKR22BiasBounties.ProposedUpdate X Y)

value:
  fun {X : Type u_1} {Y : Type u_2} (groups : List (GHKR22BiasBounties.Group X))
    (pastModels : List (GHKR22BiasBounties.Model X Y)) =>
    List.flatMap
      (fun (g : GHKR22BiasBounties.Group X) =>
        List.map (fun (f : GHKR22BiasBounties.Model X Y) => { group := g, replacement := f }) pastModels)
      groups
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.ProposedUpdate](#)

GHKR22BiasBounties.repairNoopSubmission

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.Submission X Y

value:
fun {X : Type u_1} {Y : Type u_2} (current : GHKR22BiasBounties.Model X Y) =>
{ current := current, group := fun (x : X) => false, replacement := current }
```

Direct retained dependencies

- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.Submission](#)

GHKR22BiasBounties.monotoneBountySubmission

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
ℕ →
GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y →
GHKR22BiasBounties.MonotoneBountyState X Y → GHKR22BiasBounties.Submission X Y

value:
fun {X : Type u_1} {Y : Type u_2} (U : ℕ) (strategy : GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y)
(state : GHKR22BiasBounties.MonotoneBountyState X Y) =>
match state.remainingRepairs with
| candidate :: tail => { current := state.current, group := candidate.group, replacement := candidate.replacement }
| [] =>
if state.externalProcessed < U then
have proposal : GHKR22BiasBounties.ProposedUpdate X Y := strategy state.reverseTranscript.reverse;
{ current := state.current, group := proposal.group, replacement := proposal.replacement }
else GHKR22BiasBounties.repairNoopSubmission state.current
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.MonotoneBountyState](#)
- [GHKR22BiasBounties.ProposedUpdate](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.repairNoopSubmission](#)

GHKR22BiasBounties.restartMonotoneRepairScan

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} → GHKR22BiasBounties.MonotoneBountyState X Y → GHKR22BiasBounties.MonotoneBountyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} (state : GHKR22BiasBounties.MonotoneBountyState X Y) =>
have candidates : List (GHKR22BiasBounties.ProposedUpdate X Y) :=
GHKR22BiasBounties.repairCandidates state.protectedGroups state.publicModels;
match candidates with
| [] => GHKR22BiasBounties.publishMonotoneBountyState state
| head :: tail =>
{ current := state.current, accepted := state.accepted, externalProcessed := state.externalProcessed,
protectedGroups := state.protectedGroups, publicModels := state.publicModels, remainingRepairs := candidates,
rejectedRepairsReverse := [], publicationsReverse := state.publicationsReverse,
reverseTranscript := state.reverseTranscript }
```

Direct retained dependencies

- [GHKR22BiasBounties.MonotoneBountyState](#)
- [GHKR22BiasBounties.ProposedUpdate](#)
- [GHKR22BiasBounties.publishMonotoneBountyState](#)
- [GHKR22BiasBounties.repairCandidates](#)

GHKR22BiasBounties.monotoneBountyStepCore

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
  N →
  GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y →
  GHKR22BiasBounties.MonotoneBountyState X Y →
  GHKR22BiasBounties.CertificateDecision → GHKR22BiasBounties.MonotoneBountyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} (U : N) (strategy : GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y)
  (state : GHKR22BiasBounties.MonotoneBountyState X Y) (decision : GHKR22BiasBounties.CertificateDecision) =>
  match state.remainingRepairs with
  | candidate :: rest =>
    match decision with
    | GHKR22BiasBounties.CertificateDecision.rejected =>
      have advanced : GHKR22BiasBounties.MonotoneBountyState X Y :=
        { current := state.current, accepted := state.accepted, externalProcessed := state.externalProcessed,
          protectedGroups := state.protectedGroups, publicModels := state.publicModels, remainingRepairs := rest,
          rejectedRepairsReverse := candidate :: state.rejectedRepairsReverse,
          publicationsReverse := state.publicationsReverse, reverseTranscript := state.reverseTranscript };
      match rest with
      | [] => GHKR22BiasBounties.publishMonotoneBountyState advanced
      | head :: tail => advanced
    | GHKR22BiasBounties.CertificateDecision.accepted =>
      GHKR22BiasBounties.restartMonotoneRepairScan
      { current := GHKR22BiasBounties.listUpdate state.current candidate.group candidate.replacement,
        accepted := state.accepted + 1, externalProcessed := state.externalProcessed,
        protectedGroups := state.protectedGroups, publicModels := state.publicModels,
        remainingRepairs := state.remainingRepairs, rejectedRepairsReverse := state.rejectedRepairsReverse,
        publicationsReverse := state.publicationsReverse, reverseTranscript := state.reverseTranscript }
  | [] =>
    if state.externalProcessed < U then
      have proposal : GHKR22BiasBounties.ProposedUpdate X Y := strategy state.reverseTranscript.reverse;
      match decision with
      | GHKR22BiasBounties.CertificateDecision.rejected =>
        { current := state.current, accepted := state.accepted, externalProcessed := state.externalProcessed + 1,
          protectedGroups := state.protectedGroups, publicModels := state.publicModels,
          remainingRepairs := state.remainingRepairs, rejectedRepairsReverse := state.rejectedRepairsReverse,
          publicationsReverse := state.publicationsReverse, reverseTranscript := state.reverseTranscript }
      | GHKR22BiasBounties.CertificateDecision.accepted =>
        GHKR22BiasBounties.restartMonotoneRepairScan
        { current := GHKR22BiasBounties.listUpdate state.current proposal.group proposal.replacement,
          accepted := state.accepted + 1, externalProcessed := state.externalProcessed + 1,
          protectedGroups := proposal.group :: state.protectedGroups, publicModels := state.publicModels,
          remainingRepairs := state.remainingRepairs, rejectedRepairsReverse := state.rejectedRepairsReverse,
          publicationsReverse := state.publicationsReverse, reverseTranscript := state.reverseTranscript }
    else
      match decision with
      | GHKR22BiasBounties.CertificateDecision.rejected => state
      | GHKR22BiasBounties.CertificateDecision.accepted =>
        { current := state.current, accepted := state.accepted + 1, externalProcessed := state.externalProcessed,
          protectedGroups := state.protectedGroups, publicModels := state.publicModels,
          remainingRepairs := state.remainingRepairs, rejectedRepairsReverse := state.rejectedRepairsReverse,
          publicationsReverse := state.publicationsReverse, reverseTranscript := state.reverseTranscript }
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MonotoneBountyState](#)
- [GHKR22BiasBounties.ProposedUpdate](#)
- [GHKR22BiasBounties.listUpdate](#)
- [GHKR22BiasBounties.publishMonotoneBountyState](#)
- [GHKR22BiasBounties.restartMonotoneRepairScan](#)

GHKR22BiasBounties.monotoneBountyStepFromDecision

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
ℕ →
  GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y →
  GHKR22BiasBounties.MonotoneBountyState X Y →
  GHKR22BiasBounties.CertificateDecision → GHKR22BiasBounties.MonotoneBountyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} (U : ℕ) (strategy : GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y)
  (state : GHKR22BiasBounties.MonotoneBountyState X Y) (decision : GHKR22BiasBounties.CertificateDecision) =>
  have __src : GHKR22BiasBounties.MonotoneBountyState X Y :=
    GHKR22BiasBounties.monotoneBountyStepCore U strategy state decision;
  { current := __src.current, accepted := __src.accepted, externalProcessed := __src.externalProcessed,
    protectedGroups := __src.protectedGroups, publicModels := __src.publicModels,
    remainingRepairs := __src.remainingRepairs, rejectedRepairsReverse := __src.rejectedRepairsReverse,
    publicationsReverse := __src.publicationsReverse, reverseTranscript := decision :: state.reverseTranscript }
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.MonotoneBountyState](#)
- [GHKR22BiasBounties.monotoneBountyStepCore](#)

GHKR22BiasBounties.monotoneBountyReplayAux

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
ℕ →
  GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y →
  GHKR22BiasBounties.MonotoneBountyState X Y →
  List GHKR22BiasBounties.CertificateDecision → GHKR22BiasBounties.MonotoneBountyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} (U : ℕ) (strategy : GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y)
  (a : GHKR22BiasBounties.MonotoneBountyState X Y) (a_1 : List GHKR22BiasBounties.CertificateDecision) =>
  List.brecOn (motive := fun (x : List GHKR22BiasBounties.CertificateDecision) =>
    GHKR22BiasBounties.MonotoneBountyState X Y → GHKR22BiasBounties.MonotoneBountyState X Y) a_1
  (GHKR22BiasBounties.monotoneBountyReplayAux_f U strategy) a
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.MonotoneBountyState](#)
- [GHKR22BiasBounties.monotoneBountyStepFromDecision](#)

GHKR22BiasBounties.monotoneBountyStateFromTranscript

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
  GHKR22BiasBounties.Model X Y →
  ℕ →
    GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y →
    List GHKR22BiasBounties.CertificateDecision → GHKR22BiasBounties.MonotoneBountyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} (initial : GHKR22BiasBounties.Model X Y) (U : ℕ)
  (strategy : GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y)
  (transcript : List GHKR22BiasBounties.CertificateDecision) =>
  GHKR22BiasBounties.monotoneBountyReplayAux U strategy (GHKR22BiasBounties.initialMonotoneBountyState initial)
  transcript
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.MonotoneBountyState](#)
- [GHKR22BiasBounties.initialMonotoneBountyState](#)
- [GHKR22BiasBounties.monotoneBountyReplayAux](#)

GHKR22BiasBounties.monotoneBountySubmissionStrategy

Declaration location: paper-local

Lean declaration body

```
type:
  {X : Type u_1} →
  {Y : Type u_2} →
  GHKR22BiasBounties.Model X Y →
  ℕ →
  GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y →
  List GHKR22BiasBounties.CertificateDecision → GHKR22BiasBounties.Submission X Y

value:
  fun {X : Type u_1} {Y : Type u_2} (initial : GHKR22BiasBounties.Model X Y) (U : ℕ)
    (strategy : GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y)
    (a : List GHKR22BiasBounties.CertificateDecision) =>
    GHKR22BiasBounties.monotoneBountySubmission U strategy
    (GHKR22BiasBounties.monotoneBountyStateFromTranscript initial U strategy a)
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.monotoneBountyStateFromTranscript](#)
- [GHKR22BiasBounties.monotoneBountySubmission](#)

GHKR22BiasBounties.sourceListUpdateVCBound

Declaration location: paper-local

Lean declaration body

```
type:
  ℕ → ℕ → ℝ

value:
  fun (dG dH : ℕ) =>
    2 * ↑(GHKR22BiasBounties.listUpdateVCTotal dG dH) *
    Real.logb 2 (Real.exp 1 * ↑(GHKR22BiasBounties.listUpdateVCTotal dG dH))
```

Direct retained dependencies

- [GHKR22BiasBounties.listUpdateVCTotal](#)

GHKR22BiasBounties.listUpdateVCDimensionBound

Declaration location: paper-local

Lean declaration body

```
type:
  ℕ → ℕ → ℕ

value:
  fun (dG dH : ℕ) => [GHKR22BiasBounties.sourceListUpdateVCBound dG dH] +
```

Direct retained dependencies

- [GHKR22BiasBounties.sourceListUpdateVCBound](#)

GHKR22BiasBounties.certificateVCConfidenceRadius

Declaration location: paper-local

Lean declaration body

```
type:
   $\mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{N} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 

value:
  fun (dG dH n :  $\mathbb{N}$ ) (delta :  $\mathbb{R}$ ) =>
    AppliedModelingLib.Statistics.finiteIIDVCSourcesConfidenceRadius (GHKR22BiasBounties.listUpdateVCDimensionBound dG dH)
      n (delta / 2) +
    AppliedModelingLib.Statistics.finiteIIDVCSourcesConfidenceRadius 1 n (delta / 2)
```

Direct retained dependencies

- [AppliedModelingLib.Statistics.finiteIIDVCSourcesConfidenceRadius](#)
- [GHKR22BiasBounties.listUpdateVCDimensionBound](#)

GHKR22BiasBounties.submissionScore

Declaration location: paper-local

Lean declaration body

```
type:
  {X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.BoundedLoss Y → GHKR22BiasBounties.Submission X Y →  $X \times Y \rightarrow \mathbb{R}$ 

value:
  fun {X : Type u_1} {Y : Type u_2} (loss : GHKR22BiasBounties.BoundedLoss Y)
    (submission : GHKR22BiasBounties.Submission X Y) (datum :  $X \times Y$ ) =>
    GHKR22BiasBounties.groupIndicator submission.group datum.1 *
    (GHKR22BiasBounties.datumLoss loss submission.current datum -
    GHKR22BiasBounties.datumLoss loss submission.replacement datum)
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.datumLoss](#)
- [GHKR22BiasBounties.groupIndicator](#)

GHKR22BiasBounties.empiricalSubmissionScore

Declaration location: paper-local

Lean declaration body

```
type:
  {X : Type u_1} →
  {Y : Type u_2} → {n :  $\mathbb{N}$ } → GHKR22BiasBounties.BoundedLoss Y → (Fin n →  $X \times Y$ ) → GHKR22BiasBounties.Submission X Y →  $\mathbb{R}$ 

value:
  fun {X : Type u_1} {Y : Type u_2} {n :  $\mathbb{N}$ } (loss : GHKR22BiasBounties.BoundedLoss Y) (sample : Fin n →  $X \times Y$ )
    (submission : GHKR22BiasBounties.Submission X Y) =>
    AppliedModelingLib.Probability.finiteIdScoreSum (GHKR22BiasBounties.submissionScore loss submission) sample / ↑n
```

Direct retained dependencies

- [AppliedModelingLib.Probability.finiteIdScoreSum](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.submissionScore](#)

GHKR22BiasBounties.MeasureCertificateVCDeviationEvent

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) →
GHKR22BiasBounties.Model X Bool →
Set (GHKR22BiasBounties.Group X) →
Set (GHKR22BiasBounties.Model X Bool) → ℝ → {n : ℕ} → (Fin n → X × Bool) → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
(current : GHKR22BiasBounties.Model X Bool) (G : Set (GHKR22BiasBounties.Group X))
(H : Set (GHKR22BiasBounties.Model X Bool)) (tolerance : ℝ) {n : ℕ} (sample : Fin n → X × Bool) =>
  ∃ g ∈ G,
  ∃ h ∈ H,
  tolerance <
  |GHKR22BiasBounties.empiricalSubmissionScore GHKR22BiasBounties.binaryZeroOneLoss sample
   { current := current, group := g, replacement := h } -
  GHKR22BiasBounties.measureCertificateImprovementScore law GHKR22BiasBounties.binaryZeroOneLoss current g h|
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.binaryZeroOneLoss](#)
- [GHKR22BiasBounties.empiricalSubmissionScore](#)
- [GHKR22BiasBounties.measureCertificateImprovementScore](#)

GHKR22BiasBounties.MeasureCertificateVCDeviationFailure

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) →
GHKR22BiasBounties.Model X Bool →
Set (GHKR22BiasBounties.Group X) → Set (GHKR22BiasBounties.Model X Bool) → ℝ → ℕ → ℝ

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
(current : GHKR22BiasBounties.Model X Bool) (G : Set (GHKR22BiasBounties.Group X))
(H : Set (GHKR22BiasBounties.Model X Bool)) (tolerance : ℝ) (n : ℕ) =>
  (AppliedModelingLib.Probability.finiteIIDSampleLaw law n).real
  {sample : Fin n → X × Bool | GHKR22BiasBounties.MeasureCertificateVCDeviationEvent law current G H tolerance sample}
```

Direct retained dependencies

- [AppliedModelingLib.Probability.finiteIIDSampleLaw](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasureCertificateVCDeviationEvent](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.MeasureAdaptiveFalsifyRunAuxGuaranteed

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
ℝ →
{n : ℕ} →
(Fin n → X × Y) →
GHKR22BiasBounties.Model X Y →
GHKR22BiasBounties.AdaptiveProposalStrategy X Y →
ℕ → List GHKR22BiasBounties.CertificateDecision → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) {n : ℕ} (sample : Fin n → X × Y)
  (initial : GHKR22BiasBounties.Model X Y) (strategy : GHKR22BiasBounties.AdaptiveProposalStrategy X Y) (a : ℕ)
  (a_1 : List GHKR22BiasBounties.CertificateDecision) =>
  Nat.brecOn (motive := fun (x : ℕ) => List GHKR22BiasBounties.CertificateDecision → Prop) a
  (GHKR22BiasBounties.MeasureAdaptiveFalsifyRunAuxGuaranteed._f law loss epsilon sample initial strategy) a_1
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.FalsifyState](#)
- [GHKR22BiasBounties.MeasureFalsifyDecisionGuarantee](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.ProposedUpdate](#)
- [GHKR22BiasBounties.certificateCheckerDecision](#)
- [GHKR22BiasBounties.falsifyStateFromTranscript](#)
- [GHKR22BiasBounties.numberAccepted](#)
- [GHKR22BiasBounties.proposalSubmission](#)

GHKR22BiasBounties.MeasureAdaptiveFalsifyRunGuaranteed

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
ℝ →
{n : ℕ} →
(Fin n → X × Y) →
ℕ → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.AdaptiveProposalStrategy X Y → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) {n : ℕ} (sample : Fin n → X × Y) (U : ℕ)
  (initial : GHKR22BiasBounties.Model X Y) (strategy : GHKR22BiasBounties.AdaptiveProposalStrategy X Y) =>
  GHKR22BiasBounties.MeasureAdaptiveFalsifyRunAuxGuaranteed law loss epsilon sample initial strategy U []
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.MeasureAdaptiveFalsifyRunAuxGuaranteed](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.MeasureCheckerRunAuxGuaranteed

Declaration location: paper-local

Lean declaration body

```
type:
  {X : Type u_1} →
  {Y : Type u_2} →
  [inst : MeasurableSpace X] →
  [inst_1 : MeasurableSpace Y] →
  MeasureTheory.Measure (X × Y) →
  GHKR22BiasBounties.BoundedLoss Y →
  ℝ →
  {n : ℕ} →
  (Fin n → X × Y) →
  GHKR22BiasBounties.AdaptiveSubmissionStrategy X Y →
  ℕ → List GHKR22BiasBounties.CertificateDecision → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) {n : ℕ} (sample : Fin n → X × Y)
  (strategy : GHKR22BiasBounties.AdaptiveSubmissionStrategy X Y) (a : ℕ)
  (a_1 : List GHKR22BiasBounties.CertificateDecision) =>
  Nat.brecOn (motive := fun (x : ℕ) => List GHKR22BiasBounties.CertificateDecision → Prop) a
  (GHKR22BiasBounties.MeasureCheckerRunAuxGuaranteed._f law loss epsilon sample strategy) a_1
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveSubmissionStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.MeasureCheckerDecisionGuaranteed](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.certificateCheckerDecision](#)
- [GHKR22BiasBounties.numberAccepted](#)

GHKR22BiasBounties.MeasureMonotoneBountyRunGuaranteed

Declaration location: paper-local

Lean declaration body

```
type:
  {X : Type u_1} →
  {Y : Type u_2} →
  [inst : MeasurableSpace X] →
  [inst_1 : MeasurableSpace Y] →
  MeasureTheory.Measure (X × Y) →
  GHKR22BiasBounties.BoundedLoss Y →
  ℝ →
  {n : ℕ} →
  (Fin n → X × Y) →
  ℕ → ℕ → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) {n : ℕ} (sample : Fin n → X × Y) (Q U : ℕ)
  (initial : GHKR22BiasBounties.Model X Y) (strategy : GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y) =>
  GHKR22BiasBounties.MeasureCheckerRunAuxGuaranteed law loss epsilon sample
  (GHKR22BiasBounties.monotoneBountySubmissionStrategy initial U strategy) Q []
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.MeasureCheckerRunAuxGuaranteed](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.monotoneBountySubmissionStrategy](#)

GHKR22BiasBounties.certificateCheckerRunAux

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{n : ℕ} →
ℝ →
  GHKR22BiasBounties.BoundedLoss Y →
  (Fin n → X × Y) →
  GHKR22BiasBounties.AdaptiveSubmissionStrategy X Y →
  ℕ → List GHKR22BiasBounties.CertificateDecision → List GHKR22BiasBounties.CertificateDecision

value:
fun {X : Type u_1} {Y : Type u_2} {n : ℕ} (epsilon : ℝ) (loss : GHKR22BiasBounties.BoundedLoss Y)
  (sample : Fin n → X × Y) (strategy : GHKR22BiasBounties.AdaptiveSubmissionStrategy X Y) (a : ℕ)
  (a_1 : List GHKR22BiasBounties.CertificateDecision) =>
  Nat.brecOn (motive := fun (x : ℕ) =>
    List GHKR22BiasBounties.CertificateDecision → List GHKR22BiasBounties.CertificateDecision) a
  (GHKR22BiasBounties.certificateCheckerRunAux._f epsilon loss sample strategy) a_1
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveSubmissionStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.certificateCheckerDecision](#)
- [GHKR22BiasBounties.numberAccepted](#)

GHKR22BiasBounties.adaptiveFalsifyTranscript

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{n : ℕ} →
ℝ →
  GHKR22BiasBounties.BoundedLoss Y →
  (Fin n → X × Y) →
  ℕ →
  GHKR22BiasBounties.Model X Y →
  GHKR22BiasBounties.AdaptiveProposalStrategy X Y → List GHKR22BiasBounties.CertificateDecision

value:
fun {X : Type u_1} {Y : Type u_2} {n : ℕ} (epsilon : ℝ) (loss : GHKR22BiasBounties.BoundedLoss Y)
  (sample : Fin n → X × Y) (U : ℕ) (initial : GHKR22BiasBounties.Model X Y)
  (strategy : GHKR22BiasBounties.AdaptiveProposalStrategy X Y) =>
  GHKR22BiasBounties.certificateCheckerRun epsilon loss sample U
  (GHKR22BiasBounties.falsifySubmissionStrategy initial strategy)
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.certificateCheckerRun](#)
- [GHKR22BiasBounties.falsifySubmissionStrategy](#)

GHKR22BiasBounties.MeasureAdaptiveFalsifyRunConclusion

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
ℝ →
{n : ℕ} →
(Fin n → X × Y) →
ℕ → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.AdaptiveProposalStrategy X Y → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) {n : ℕ} (sample : Fin n → X × Y) (U : ℕ)
  (initial : GHKR22BiasBounties.Model X Y) (strategy : GHKR22BiasBounties.AdaptiveProposalStrategy X Y) =>
  GHKR22BiasBounties.MeasureAdaptiveFalsifyRunGuaranteed law loss epsilon sample U initial strategy ∧
  (GHKR22BiasBounties.adaptiveFalsifyTranscript epsilon loss sample U initial strategy).length = U ∧
  ↑(GHKR22BiasBounties.adaptiveFalsifyAndUpdate epsilon loss sample U initial strategy).accepted ≤ 2 / epsilon
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.FalsifyState](#)
- [GHKR22BiasBounties.MeasureAdaptiveFalsifyRunGuaranteed](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.adaptiveFalsifyAndUpdate](#)
- [GHKR22BiasBounties.adaptiveFalsifyTranscript](#)

GHKR22BiasBounties.empiricalCandidateObjective

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{n : ℕ} →
GHKR22BiasBounties.BoundedLoss Y →
(Fin n → X × Y) → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.CandidatePair X Y → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} {n : ℕ} (loss : GHKR22BiasBounties.BoundedLoss Y) (sample : Fin n → X × Y)
  (current : GHKR22BiasBounties.Model X Y) (candidate : GHKR22BiasBounties.CandidatePair X Y) =>
  GHKR22BiasBounties.empiricalSubmissionScore loss sample
  { current := current, group := candidate.1, replacement := candidate.2 }
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CandidatePair](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.empiricalSubmissionScore](#)

GHKR22BiasBounties.EmpiricalCandidateMaximizer

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{n : ℕ} →
  GHKR22BiasBounties.BoundedLoss Y →
  (Fin n → X × Y) →
    GHKR22BiasBounties.Model X Y →
      Set (GHKR22BiasBounties.CandidatePair X Y) → GHKR22BiasBounties.CandidatePair X Y → Prop

value:
fun {X : Type u_1} {Y : Type u_2} {n : ℕ} (loss : GHKR22BiasBounties.BoundedLoss Y) (sample : Fin n → X × Y)
  (current : GHKR22BiasBounties.Model X Y) (C : Set (GHKR22BiasBounties.CandidatePair X Y))
  (chosen : GHKR22BiasBounties.CandidatePair X Y) =>
  chosen ∈ C ∧
  ∀ candidate ∈ C,
    GHKR22BiasBounties.empiricalCandidateObjective loss sample current candidate ≤
    GHKR22BiasBounties.empiricalCandidateObjective loss sample current chosen
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CandidatePair](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.empiricalCandidateObjective](#)

GHKR22BiasBounties.ExactTrainingOracle

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{n : ℕ} →
  GHKR22BiasBounties.BoundedLoss Y →
  Set (GHKR22BiasBounties.CandidatePair X Y) → GHKR22BiasBounties.TrainingOracle X Y n → Prop

value:
fun {X : Type u_1} {Y : Type u_2} {n : ℕ} (loss : GHKR22BiasBounties.BoundedLoss Y)
  (C : Set (GHKR22BiasBounties.CandidatePair X Y)) (oracle : GHKR22BiasBounties.TrainingOracle X Y n) =>
  ∀ (current : GHKR22BiasBounties.Model X Y) (sample : Fin n → X × Y),
    GHKR22BiasBounties.EmpiricalCandidateMaximizer loss sample current C (oracle current sample)
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CandidatePair](#)
- [GHKR22BiasBounties.EmpiricalCandidateMaximizer](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TrainingOracle](#)

GHKR22BiasBounties.measureAdaptiveFalsifyAndUpdateFailure

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
  MeasureTheory.Measure (X × Y) →
  GHKR22BiasBounties.BoundedLoss Y →
  ℝ → ℕ → ℕ → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.AdaptiveProposalStrategy X Y → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) (n U : ℕ) (initial : GHKR22BiasBounties.Model X Y)
  (strategy : GHKR22BiasBounties.AdaptiveProposalStrategy X Y) =>
  (AppliedModelingLib.Probability.finitelIDSampleLaw law n).real
  {sample : Fin n → X × Y |
    ¬GHKR22BiasBounties.MeasureAdaptiveFalsifyRunConclusion law loss epsilon sample U initial strategy}
```

Direct retained dependencies

- [AppliedModelingLib.Probability.finiteIIDSampleLaw](#)
- [GHKR22BiasBounties.AdaptiveProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.MeasureAdaptiveFalsifyRunConclusion](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.measureCertificateCheckerRunGuaranteeFailure

Declaration location: paper-local

Lean declaration body

```
type:
  {X : Type u_1} →
  {Y : Type u_2} →
  [inst : MeasurableSpace X] →
  [inst_1 : MeasurableSpace Y] →
  MeasureTheory.Measure (X × Y) →
  GHKR22BiasBounties.BoundedLoss Y → ℝ → ℕ → ℕ → GHKR22BiasBounties.AdaptiveSubmissionStrategy X Y → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) (n U : ℕ)
  (strategy : GHKR22BiasBounties.AdaptiveSubmissionStrategy X Y) =>
  (AppliedModelingLib.Probability.finiteIIDSampleLaw law n).real
  {sample : Fin n → X × Y | ¬GHKR22BiasBounties.MeasureCheckerRunAuxGuaranteed law loss epsilon sample strategy U []}
```

Direct retained dependencies

- [AppliedModelingLib.Probability.finiteIIDSampleLaw](#)
- [GHKR22BiasBounties.AdaptiveSubmissionStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.MeasureCheckerRunAuxGuaranteed](#)

GHKR22BiasBounties.monotoneBountyTranscript

Declaration location: paper-local

Lean declaration body

```
type:
  {X : Type u_1} →
  {Y : Type u_2} →
  {n : ℕ} →
  ℝ →
  GHKR22BiasBounties.BoundedLoss Y →
  (Fin n → X × Y) →
  ℕ →
  GHKR22BiasBounties.Model X Y →
  GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y → List GHKR22BiasBounties.CertificateDecision

value:
fun {X : Type u_1} {Y : Type u_2} {n : ℕ} (epsilon : ℝ) (loss : GHKR22BiasBounties.BoundedLoss Y)
  (sample : Fin n → X × Y) (U : ℕ) (initial : GHKR22BiasBounties.Model X Y)
  (strategy : GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y) =>
  GHKR22BiasBounties.certificateCheckerRun epsilon loss sample (GHKR22BiasBounties.monotoneBountyQueryBudget epsilon U)
  (GHKR22BiasBounties.monotoneBountySubmissionStrategy initial U strategy)
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.certificateCheckerRun](#)
- [GHKR22BiasBounties.monotoneBountyQueryBudget](#)
- [GHKR22BiasBounties.monotoneBountySubmissionStrategy](#)

GHKR22BiasBounties.MeasureMonotoneBountyConclusion

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
ℝ →
{n : ℕ} →
(Fin n → X × Y) →
ℕ → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) {n : ℕ} (sample : Fin n → X × Y) (U : ℕ)
  (initial : GHKR22BiasBounties.Model X Y) (strategy : GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y) =>
have Q : ℕ := GHKR22BiasBounties.monotoneBountyQueryBudget epsilon U;
have transcript : List GHKR22BiasBounties.CertificateDecision :=
  GHKR22BiasBounties.monotoneBountyTranscript epsilon loss sample U initial strategy;
have output : GHKR22BiasBounties.MonotoneBountyState X Y :=
  GHKR22BiasBounties.monotoneBountyOutput epsilon loss sample U initial strategy;
GHKR22BiasBounties.MeasureMonotoneBountyRunGuaranteed law loss epsilon sample Q U initial strategy ∧
transcript.length = Q ∧
output.externalProcessed = U ∧
output.remainingRepairs = [] ∧
output.accepted ≤ GHKR22BiasBounties.checkerAcceptanceBudget epsilon ∧
GHKR22BiasBounties.MeasureMonotonePublicationsSound law loss epsilon output
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.MeasureMonotoneBountyRunGuaranteed](#)
- [GHKR22BiasBounties.MeasureMonotonePublicationsSound](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.MonotoneBountyState](#)
- [GHKR22BiasBounties.ProposedUpdate](#)
- [GHKR22BiasBounties.checkerAcceptanceBudget](#)
- [GHKR22BiasBounties.monotoneBountyOutput](#)
- [GHKR22BiasBounties.monotoneBountyQueryBudget](#)
- [GHKR22BiasBounties.monotoneBountyTranscript](#)

GHKR22BiasBounties.measureMonotoneBountyFailure

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
ℝ → ℕ → ℕ → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ) (n U : ℕ) (initial : GHKR22BiasBounties.Model X Y)
  (strategy : GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y) =>
(AppliedModelingLib.Probability.finitelldSampleLaw law n).real
{sample : Fin n → X × Y |
  →GHKR22BiasBounties.MeasureMonotoneBountyConclusion law loss epsilon sample U initial strategy}
```

Direct retained dependencies

- [AppliedModelingLib.Probability.finiteIIDSampleLaw](#)
- [GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.MeasureMonotoneBountyConclusion](#)
- [GHKR22BiasBounties.Model](#)

GHKR22BiasBounties.normalizedSubmissionScore

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} → {Y : Type u_2} → GHKR22BiasBounties.BoundedLoss Y → GHKR22BiasBounties.Submission X Y → X × Y → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} (loss : GHKR22BiasBounties.BoundedLoss Y)
  (submission : GHKR22BiasBounties.Submission X Y) (datum : X × Y) =>
  (GHKR22BiasBounties.submissionScore loss submission datum + 1) / 2
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.submissionScore](#)

GHKR22BiasBounties.measureAdaptiveCheckerDeviationEvent

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{Theta : Type u_3} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
(Theta → GHKR22BiasBounties.Submission X Y) → ℝ → {n : ℕ} → (Fin n → X × Y) → Prop

value:
fun {X : Type u_1} {Y : Type u_2} {Theta : Type u_3} [MeasurableSpace X] [MeasurableSpace Y]
  (law : MeasureTheory.Measure (X × Y)) (loss : GHKR22BiasBounties.BoundedLoss Y)
  (family : Theta → GHKR22BiasBounties.Submission X Y) (epsilon : ℝ) {n : ℕ} (sample : Fin n → X × Y) =>
  ∃ (parameter : Theta),
  epsilon / 8 <
  |AppliedModelingLib.Probability.finiteEmpiricalMean
    (fun (query : Theta) (index : Fin n) (outcome : Fin n → X × Y) =>
      GHKR22BiasBounties.normalizedSubmissionScore loss (family query) (outcome index))
    parameter sample -
  | (datum : X × Y), GHKR22BiasBounties.normalizedSubmissionScore loss (family parameter) datum @law|
```

Direct retained dependencies

- [AppliedModelingLib.Probability.finiteEmpiricalMean](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.normalizedSubmissionScore](#)

GHKR22BiasBounties.measureAdaptiveCheckerDeviationFailure

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{Theta : Type u_3} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y → (Theta → GHKR22BiasBounties.Submission X Y) → ℝ → ℕ → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} {Theta : Type u_3} [MeasurableSpace X] [MeasurableSpace Y]
(law : MeasureTheory.Measure (X × Y)) (loss : GHKR22BiasBounties.BoundedLoss Y)
(family : Theta → GHKR22BiasBounties.Submission X Y) (epsilon : ℝ) (n : ℕ) =>
(AppliedModelingLib.Probability.finiteIIDSampleLaw law n).real
{sample : Fin n → X × Y | GHKR22BiasBounties.measureAdaptiveCheckerDeviationEvent law loss family epsilon sample}
```

Direct retained dependencies

- [AppliedModelingLib.Probability.finiteIIDSampleLaw](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.measureAdaptiveCheckerDeviationEvent](#)

GHKR22BiasBounties.ternaryBinaryPrediction

Declaration location: paper-local

Lean declaration body

```
type:
GHKR22BiasBounties.TernaryLabel → Bool

value:
fun (a : GHKR22BiasBounties.TernaryLabel) =>
  match a with
  | GHKR22BiasBounties.TernaryLabel.zero => false
  | GHKR22BiasBounties.TernaryLabel.one => true
  | GHKR22BiasBounties.TernaryLabel.defer => false
```

Direct retained dependencies

- [GHKR22BiasBounties.TernaryLabel](#)

GHKR22BiasBounties.measureExpectedTernaryCost

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) → GHKR22BiasBounties.Model X Bool → GHKR22BiasBounties.TernaryPredictor X → ℝ

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
(current : GHKR22BiasBounties.Model X Bool) (p : GHKR22BiasBounties.TernaryPredictor X) =>
∫ (datum : X × Bool), GHKR22BiasBounties.inducedCost current datum (p datum.1) ∂law
```

Direct retained dependencies

- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TernaryPredictor](#)
- [GHKR22BiasBounties.inducedCost](#)

GHKR22BiasBounties.trainByOptRoundBudget

Declaration location: paper-local

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{N}$ 

value:
fun (epsilon :  $\mathbb{R}$ ) => [2 / epsilon]_+
```

GHKR22BiasBounties.MeasureTrainByOptOutputFailureProbability

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
(law : MeasureTheory.Measure (X × Bool)) →
[MeasureTheory.IsProbabilityMeasure law] →
Set (GHKR22BiasBounties.CandidatePair X Bool) →
 $\mathbb{R} \rightarrow \{n : \mathbb{N}\} \rightarrow \text{GHKR22BiasBounties.TrainingOracle } X \text{ Bool } n \rightarrow \text{GHKR22BiasBounties.Model } X \text{ Bool} \rightarrow \mathbb{N} \rightarrow \mathbb{R}$ 

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool)) [MeasureTheory.IsProbabilityMeasure law]
(C : Set (GHKR22BiasBounties.CandidatePair X Bool)) (epsilon :  $\mathbb{R}$ ) {n :  $\mathbb{N}$ }
(oracle : GHKR22BiasBounties.TrainingOracle X Bool n) (initial : GHKR22BiasBounties.Model X Bool) (rounds :  $\mathbb{N}$ ) =>
(GHKR22BiasBounties.measureTrainingInfiniteTraceLaw law n).real
{trace :
(i :  $\mathbb{N}$ ) →
AppliedModelingLib.HeterogeneousBatchTraceCoordinate (GHKR22BiasBounties.MeasureTrainingBatchIndex n) (X × Bool)
}
→ GHKR22BiasBounties.MeasureApproxBayesOptimal law GHKR22BiasBounties.binaryZeroOneLoss C epsilon
(GHKR22BiasBounties.trainByOptRun GHKR22BiasBounties.binaryZeroOneLoss epsilon oracle initial rounds
(AppliedModelingLib.heterogeneousBatchTraceOfInfiniteTrace rounds trace)).output}
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace](#)
- [AppliedModelingLib.heterogeneousBatchTraceOfInfiniteTrace](#)
- [GHKR22BiasBounties.CandidatePair](#)
- [GHKR22BiasBounties.MeasureApproxBayesOptimal](#)
- [GHKR22BiasBounties.MeasureTrainingBatchIndex](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TrainByOptOutcome.output](#)
- [GHKR22BiasBounties.TrainingOracle](#)
- [GHKR22BiasBounties.binaryZeroOneLoss](#)
- [GHKR22BiasBounties.measureTrainingInfiniteTraceLaw](#)
- [GHKR22BiasBounties.trainByOptRun](#)

GHKR22BiasBounties.trainByOptCurrentOfHistory

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{n :  $\mathbb{N}$ } →
GHKR22BiasBounties.BoundedLoss Y →
 $\mathbb{R} \rightarrow$ 
GHKR22BiasBounties.TrainingOracle X Y n →
GHKR22BiasBounties.Model X Y → (rounds :  $\mathbb{N}$ ) → (Fin rounds → Fin n → X × Y) → X → Y

value:
fun {X : Type u_1} {Y : Type u_2} {n :  $\mathbb{N}$ } (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon :  $\mathbb{R}$ )
(oracle : GHKR22BiasBounties.TrainingOracle X Y n) (initial : GHKR22BiasBounties.Model X Y) (rounds :  $\mathbb{N}$ )
(history : Fin rounds → Fin n → X × Y) (a : X) =>
(GHKR22BiasBounties.trainByOptRun loss epsilon oracle initial rounds history).output a
```


Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TrainByOptOutcome.output](#)
- [GHKR22BiasBounties.TrainingOracle](#)
- [GHKR22BiasBounties.trainByOptRun](#)

GHKR22BiasBounties.MeasureTrainByOptVCBadEvent

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) →
Set (GHKR22BiasBounties.Group X) →
Set (GHKR22BiasBounties.Model X Bool) →
ℝ →
ℝ →
{n : ℕ} →
GHKR22BiasBounties.TrainingOracle X Bool n →
GHKR22BiasBounties.Model X Bool →
(iteration : ℕ) →
AppliedModelingLib.HeterogeneousBatchTrace (GHKR22BiasBounties.MeasureTrainingBatchIndex n)
(X × Bool) iteration →
(GHKR22BiasBounties.MeasureTrainingBatchIndex n iteration → X × Bool) → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool)) (G : Set (GHKR22BiasBounties.Group X))
(H : Set (GHKR22BiasBounties.Model X Bool)) (epsilon eta : ℝ) {n : ℕ}
(oracle : GHKR22BiasBounties.TrainingOracle X Bool n) (initial : GHKR22BiasBounties.Model X Bool) (iteration : ℕ)
(history :
AppliedModelingLib.HeterogeneousBatchTrace (GHKR22BiasBounties.MeasureTrainingBatchIndex n) (X × Bool) iteration)
(sample : GHKR22BiasBounties.MeasureTrainingBatchIndex n iteration → X × Bool) =>
GHKR22BiasBounties.MeasureCertificateVCDeviationEvent law
(GHKR22BiasBounties.trainByOptCurrentOfHistory GHKR22BiasBounties.binaryZeroOneLoss epsilon oracle initial iteration
history)
G H eta sample
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTrace](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasureCertificateVCDeviationEvent](#)
- [GHKR22BiasBounties.MeasureTrainingBatchIndex](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TrainingOracle](#)
- [GHKR22BiasBounties.binaryZeroOneLoss](#)
- [GHKR22BiasBounties.trainByOptCurrentOfHistory](#)

GHKR22BiasBounties.MeasureTrainByOptVCBadEventPairMeasurable

Declaration location: paper-local

Lean declaration body

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) →
Set (GHKR22BiasBounties.Group X) →
Set (GHKR22BiasBounties.Model X Bool) →
ℝ → ℝ → {n : ℕ} → GHKR22BiasBounties.TrainingOracle X Bool n → GHKR22BiasBounties.Model X Bool → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool)) (G : Set (GHKR22BiasBounties.Group X))
(H : Set (GHKR22BiasBounties.Model X Bool)) (epsilon eta : ℝ) {n : ℕ}
(oracle : GHKR22BiasBounties.TrainingOracle X Bool n) (initial : GHKR22BiasBounties.Model X Bool) =>
∀ (iteration : ℕ),
MeasurableSet
(AppliedModelingLib.heterogeneousBatchTraceBadEventPair (GHKR22BiasBounties.MeasureTrainingBatchIndex n)
(GHKR22BiasBounties.MeasureTrainByOptVCBadEvent law G H epsilon eta oracle initial) iteration)
```

Direct retained dependencies

- [AppliedModelingLib.HeterogeneousBatchTraceCoordinate](#)
- [AppliedModelingLib.heterogeneousBatchTraceBadEventPair](#)
- [AppliedModelingLib.heterogeneousBatchTraceCoordinateMeasurableSpace](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasureTrainByOptVCBadEvent](#)
- [GHKR22BiasBounties.MeasureTrainingBatchIndex](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TrainingOracle](#)

Material library prerequisites

AppliedModelingLib.Learning.Prediction.BoundedLoss

Source locator: cited publication, lines 310-311

Verbatim paper-source connection

\section{Preliminaries}

We consider a supervised learning problem defined over \emph{labelled examples} $(x, y) \in \mathcal{X} \times \mathcal{Y}$. This can represent (for example) a binary classification problem if $\mathcal{Y} = \{0, 1\}$, a discrete multiclass classification problem if $\mathcal{Y} = \{1, \dots, k\}$, or a regression problem if $\mathcal{Y} = \mathbb{R}$. We write $\mathcal{D}$ to denote a joint probability distribution over labelled examples: $\mathcal{D} \in \Delta(\mathcal{X} \times \mathcal{Y})$. We will write $\mathcal{D} \sim \mathcal{D}^n$ to denote a dataset consisting of n labelled examples sampled i.i.d. from $\mathcal{D}$. Our goal is to learn some \emph{model} represented as a function $f: \mathcal{X} \rightarrow \mathcal{Y}$ which aims to predict the label of an example from its features, and we will evaluate the performance of our model with a loss function $\ell: \mathcal{Y} \times \mathcal{Y} \rightarrow [0, 1]$, where $\ell(\hat{y}, y)$ represents the “cost” of mistakenly labelling an example that has true label y with the prediction $\hat{y}$. We will be interested in the performance of models f not just \emph{overall} on the underlying distribution, but also on particular subgroups of interest. A subgroup corresponds to an arbitrary subset of the feature space $\mathcal{X}$, which we will model using an indicator function:

Lean declaration type (metadata)

field value:

$\{Y : \text{Type } u_1\} \rightarrow \text{AppliedModelingLib.Learning.Prediction.BoundedLoss } Y \rightarrow Y \rightarrow Y \rightarrow \mathbb{R}$

field nonneg:

$\forall \{Y : \text{Type } u_1\} \text{ (self : AppliedModelingLib.Learning.Prediction.BoundedLoss } Y) \text{ (prediction truth : } Y),$
 $0 \leq \text{self.value prediction truth}$

field le_one:

$\forall \{Y : \text{Type } u_1\} \text{ (self : AppliedModelingLib.Learning.Prediction.BoundedLoss } Y) \text{ (prediction truth : } Y),$
 $\text{self.value prediction truth} \leq 1$

Exact Lean library declaration

-- A loss function normalized to the unit interval. -/

```
structure BoundedLoss (Y : Type*) where
  value : Y → Y → ℝ
  nonneg : ∀ prediction truth, 0 ≤ value prediction truth
  le_one : ∀ prediction truth, value prediction truth ≤ 1
```

Recorded source-to-library screening Verdict: matches

Reason: The reusable structure is exactly the paper’s primitive loss ℓ from two labels to a real value in $[0, 1]$: the value field gives the loss, and the two proof fields enforce pointwise nonnegativity and the unit upper bound.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-specific semantic prerequisites

GHKR22BiasBounties.AEBayesOptimalForRisk

Paper-local declaration:

GHKR22BiasBounties.AEBayesOptimalForRisk

Source locator: cited publication, lines 333-339

Verbatim paper-source connection

```
\begin{definition}
A \emph{Bayes Optimal} model  $f^*: \mathcal{X} \rightarrow \mathcal{Y}$  with respect to a loss function  $\ell$  and a distribution  $\mathcal{D}$  satisfies:
\ifdraft

$$f^*(x) = \arg\min_{y \in \mathcal{Y}} \mathbb{E}_{(x', y') \sim \mathcal{D}} [\ell(y, y') \mid x' = x]$$

\else 
$$f^*(x) = \arg\min_{y \in \mathcal{Y}} \mathbb{E}_{(x', y') \sim \mathcal{D}} [\ell(y, y') \mid x' = x]$$
, \fi
where  $f^*(x)$  can be defined arbitrarily for any  $x$  that is not in the support of  $\mathcal{D}$ .
\end{definition}
```

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For an arbitrary population law, Bayes optimality means almost-everywhere minimization of conditional risk under a measurable regular conditional label $\hookrightarrow$ kernel; off the governing feature law the model is irrelevant.

Archival source anchor: cited publication, lines 333-339

Lean-expanded paper semantic target

```
type:
{X : Type u_1} → {Y : Type u_2} → [inst : MeasurableSpace X] → MeasureTheory.Measure X → (X → Y → ℝ) → (X → Y) → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] (law : MeasureTheory.Measure X) (risk : X → Y → ℝ)
  (current : X → Y) =>
  ∀m (x : X) ∂law, ∀ (prediction : Y), risk x (current x) ≤ risk x prediction
```

Recorded source-to-declaration screening Verdict: matches_approved_corrected_target

Reason: Compared Definition 3 (main.tex:333-339) with the bound observation-4-pointwise-null-set correction. The actual predicate requires, on one feature-law almost-everywhere set, that current risk is no larger than the risk of every prediction. It imposes no restriction off that set and is the minimization predicate itself. The separately displayed kernelConditionalRisk supplies the conditional-loss integral for the source instantiation. This matches the expressly corrected almost-everywhere target, without claiming equivalence to the archival pointwise wording.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.BoundedLoss

Paper-local declaration:

GHKR22BiasBounties.BoundedLoss

Source locator: cited publication, lines 310-311

Verbatim paper-source connection

\section{Preliminaries}
We consider a supervised learning problem defined over \emph{labelled examples} $(x,y) \in \mathcal{X} \times \mathcal{Y}$. This can represent (for example) a binary
↪ classification problem if $\mathcal{Y} = \{0,1\}$, a discrete multiclass classification problem if $\mathcal{Y} = \{1,\ldots,k\}$, or a regression problem if $\mathcal{Y} =$
↪ $\mathbb{R}$. We write $\mathcal{D}$ to denote a joint probability distribution over labelled examples: $\mathcal{D} \in \Delta(\mathcal{X} \times \mathcal{Y})$. We will write $\mathcal{D} \sim$
↪ $\mathcal{D}^n$ to denote a dataset consisting of n labelled examples sampled i.i.d. from $\mathcal{D}$. Our goal is to learn some \emph{model} represented as a
↪ function $f: \mathcal{X} \rightarrow \mathcal{Y}$ which aims to predict the label of an example from its features, and we will evaluate the performance of our model
↪ with a loss function $\ell: \mathcal{Y} \times \mathcal{Y} \rightarrow [0,1]$, where $\ell(\hat{y}, y)$ represents the ``cost'' of mistakenly labelling an example that has
↪ true label y with the prediction $f(x) = \hat{y}$. We will be interested in the performance of models f not just \emph{overall} on the underlying
↪ distribution, but also on particular subgroups of interest. A subgroup corresponds to an arbitrary subset of the feature space $\mathcal{X}$, which we will
↪ model using an indicator function:

Lean-expanded paper semantic target

type:
Type u_1 → Type u_1

value:
fun (Y : Type u_1) => AppliedModelingLib.Learning.Prediction.BoundedLoss Y

Direct retained dependencies

- [AppliedModelingLib.Learning.Prediction.BoundedLoss](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The structure represents the paper’s loss function into $[0,1]$ by a real-valued loss with pointwise lower bound zero and upper bound one.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

GHKR22BiasBounties.Group

Source locator: cited publication, lines 310-311

Verbatim paper-source connection

`\section{Preliminaries}`
We consider a supervised learning problem defined over `\emph{labelled examples}` $(x,y) \in \mathcal{X} \times \mathcal{Y}$. This can represent (for example) a binary
→ classification problem if $\mathcal{Y} = \{0,1\}$, a discrete multiclass classification problem if $\mathcal{Y} = \{1,\ldots,k\}$, or a regression problem if $\mathcal{Y} =$
→ $\mathbb{R}$. We write $\mathcal{D}$ to denote a joint probability distribution over labelled examples: $\mathcal{D} \in \Delta(\mathcal{X} \times \mathcal{Y})$. We will write $\mathcal{D} \sim$
→ $\mathcal{D}^n$ to denote a dataset consisting of n labelled examples sampled i.i.d. from $\mathcal{D}$. Our goal is to learn some `\emph{model}` represented as a
→ function $f: \mathcal{X} \rightarrow \mathcal{Y}$ which aims to predict the label of an example from its features, and we will evaluate the performance of our model
→ with a loss function $\ell: \mathcal{Y} \times \mathcal{Y} \rightarrow [0,1]$, where $\ell(\hat{y}, y)$ represents the “cost” of mistakenly labelling an example that has
→ true label y with the prediction $\hat{y}$. We will be interested in the performance of models f not just `\emph{overall}` on the underlying
→ distribution, but also on particular subgroups of interest. A subgroup corresponds to an arbitrary subset of the feature space $\mathcal{X}$, which we will
→ model using an indicator function:

Lean-expanded paper semantic target

type:
Type u_1 → Type u_1

value:
fun (X : Type u_1) => AppliedModelingLib.Learning.Prediction.HardGroup X

Direct retained dependencies

- [AppliedModelingLib.Learning.Prediction.HardGroup](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The supervised-learning setup introduces subgroups as subsets of the feature space represented by indicators; the Boolean predicate X to Bool is exactly that representation, with Definition 1 supplying the subsequent notation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.Model

Paper-local declaration:

GHKR22BiasBounties.Model

Source locator: cited publication, lines 310-311

Verbatim paper-source connection

`\section{Preliminaries}`
We consider a supervised learning problem defined over `\emph{labelled examples}` $(x,y) \in \mathcal{X} \times \mathcal{Y}$. This can represent (for example) a binary
→ classification problem if $\mathcal{Y} = \{0,1\}$, a discrete multiclass classification problem if $\mathcal{Y} = \{1,\dots,k\}$, or a regression problem if $\mathcal{Y} =$
→ $\mathbb{R}$. We write $\mathcal{D}$ to denote a joint probability distribution over labelled examples: $\mathcal{D} \in \Delta(\mathcal{X} \times \mathcal{Y})$. We will write $D \sim$
→ $\mathcal{D}^n$ to denote a dataset consisting of n labelled examples sampled i.i.d. from $\mathcal{D}$. Our goal is to learn some `\emph{model}` represented as a
→ function $f: \mathcal{X} \rightarrow \mathcal{Y}$ which aims to predict the label of an example from its features, and we will evaluate the performance of our model
→ with a loss function $\ell: \mathcal{Y} \times \mathcal{Y} \rightarrow [0,1]$, where $\ell(\hat{y}, y)$ represents the “cost” of mistakenly labelling an example that has
→ true label y with the prediction $\hat{y}$. We will be interested in the performance of models f not just `\emph{overall}` on the underlying
→ distribution, but also on particular subgroups of interest. A subgroup corresponds to an arbitrary subset of the feature space $\mathcal{X}$, which we will
→ model using an indicator function:

Lean-expanded paper semantic target

type:
Type u_1 → Type u_2 → Type (max u_1 u_2)

value:
fun (X : Type u_1) (Y : Type u_2) => AppliedModelingLib.Learning.Prediction.Model X Y

Direct retained dependencies

- [AppliedModelingLib.Learning.Prediction.Model](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The abbreviation X to Y is exactly the paper’s deterministic prediction model.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.MeasureApproxBayesOptimal

Paper-local declaration:

GHKR22BiasBounties.MeasureApproxBayesOptimal

Source locator: cited publication, lines 354-361

Verbatim paper-source connection

```
\begin{definition}
A model  $f: \mathcal{X} \rightarrow \mathcal{Y}$  is  $(\epsilon, \mathcal{C})$ -Bayes optimal with respect to a collection of (group, model) pairs  $\mathcal{C}$  if for each  $(g, h) \in \mathcal{C}$ , the
 $\hookrightarrow$  performance of  $f$  on  $g$  is within  $\epsilon$  of the performance of  $h$  on  $g$ . In other words, for every  $(g, h) \in \mathcal{C}$ :
\ifdraft
 $L(\mathcal{C}, f, g) \leq L(\mathcal{C}, h, g) + \frac{\epsilon}{\mu_g(\mathcal{C})}$ .
\else  $L(\mathcal{C}, f, g) \leq L(\mathcal{C}, h, g) + \frac{\epsilon}{\mu_g(\mathcal{C})}$ .
\fi
When  $\mathcal{C}$  is a product set  $\mathcal{C} = \mathcal{G} \times \mathcal{H}$ , then we call  $f$ 
`` $(\epsilon, \mathcal{G}, \mathcal{H})$ -Bayes Optimal" and the condition is equivalent to requiring that for every  $g \in \mathcal{G}$ ,  $f$  has performance on  $g$  that is within
 $\hookrightarrow$   $\epsilon$  of the best model  $h \in \mathcal{H}$  on  $g$ . When  $\mathcal{G}$  and  $\mathcal{H}$  represent the set of all groups and models respectively, we call  $f$ 
 $\hookrightarrow$   $\epsilon$ -Bayes optimal.
\end{definition}
```

Settled source reading The mass-weighted group-loss numerator is the total primitive; conditional group loss is totalized to zero at mass zero, and every divided source display is recovered on positive-mass groups.

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
Set (GHKR22BiasBounties.Group X × GHKR22BiasBounties.Model X Y) → ℝ → GHKR22BiasBounties.Model X Y → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
(loss : GHKR22BiasBounties.BoundedLoss Y)
(certificates : Set (GHKR22BiasBounties.Group X × GHKR22BiasBounties.Model X Y)) (epsilon : ℝ)
(current : GHKR22BiasBounties.Model X Y) =>
∀ pair ∈ certificates, GHKR22BiasBounties.measureCertificateImprovementScore law loss current pair.1 pair.2 ≤ epsilon
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.measureCertificateImprovementScore](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Multiplying the conditional-loss comparison by group mass yields exactly the paper's epsilon-weighted certificate score bound for every listed group/model pair, including the totalized null-group case.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.groupIndicator

Paper-local declaration:

GHKR22BiasBounties.groupIndicator

Source locator: cited publication, lines 312-318

Verbatim paper-source connection

$\begin{array}{l} \text{\textit{Subgroups}} \\ \text{A \textit{subgroup} of the feature space } \mathcal{X} \text{ will be represented as an indicator function } g:\mathcal{X} \rightarrow \{0,1\}. \text{ We say that } x \in \mathcal{X} \text{ is in group } \\ \hookrightarrow g \text{ if } g(x) = 1 \text{ and } x \text{ is \textit{not} in group } g \text{ otherwise. Given a group } g, \text{ we write } \mu_g(\mathcal{D}) \text{ to denote its measure under the } \\ \hookrightarrow \text{probability distribution } \mathcal{D}: \\ \text{\textit{ifdraft}} \\ \mu_g(\mathcal{D}) = \Pr_{\mathcal{D}}[g(x) = 1]. \\ \text{else } \mu_g(\mathcal{D}) = \Pr_{\mathcal{D}}[g(x) = 1]. \end{array}$
We write $\mu_g(\mathcal{D})$ to denote the corresponding empirical measure under $\mathcal{D}$, which results from viewing $\mathcal{D}$ as the uniform distribution over its
 $\hookrightarrow$ elements.
 $\end{array}$

Lean-expanded paper semantic target

type:
 $\{X : \text{Type } u_1\} \rightarrow \text{GHKR22BiasBounties.Group } X \rightarrow X \rightarrow \mathbb{R}$
value:
 $\text{fun } \{X : \text{Type } u_1\} (g : \text{GHKR22BiasBounties.Group } X) (x : X) =>$
 $\text{AppliedModelingLib.Learning.Prediction.hardGroupIndicator } g \ x$

Direct retained dependencies

- [AppliedModelingLib.Learning.Prediction.hardGroupIndicator](#)
- [GHKR22BiasBounties.Group](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The real value is one exactly on Boolean group membership and zero otherwise, the numeric indicator used in the source mass and loss formulas.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.measureGroupLossNumerator

Paper-local declaration:

GHKR22BiasBounties.measureGroupLossNumerator

Source locator: cited publication, lines 321-330

Verbatim paper-source connection

```
\begin{definition}[Model Loss]
Given a model  $f: \mathcal{X} \rightarrow \mathcal{Y}$ 
We write  $L(\mathcal{D}, f)$  to denote the average loss of  $f$  on distribution  $\mathcal{D}$ :
\ifdraft
 $L(\mathcal{D}, f) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(f(x), y)]$ 
\else  $L(\mathcal{D}, f) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(f(x), y)]$ . \fi
We write  $L(\mathcal{D}, f, g)$  to denote the loss on  $f$  conditional on membership in  $g$ :
 $L(\mathcal{D}, f, g) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(f(x), y) \mid g(x) = 1]$ .
Given a dataset  $D$ , we write  $L(D, f)$  and  $L(D, f, g)$  to denote the corresponding empirical losses on  $D$ , which result from viewing  $D$  as the uniform
 $\hookrightarrow$  distribution over its elements.
\end{definition}
```

Settled source reading The mass-weighted group-loss numerator is the total primitive; conditional group loss is totalized to zero at mass zero, and every divided source display is recovered on positive-mass groups.

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.Group X → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (model : GHKR22BiasBounties.Model X Y)
  (group : GHKR22BiasBounties.Group X) =>
  ∫ (datum : X × Y),
    GHKR22BiasBounties.groupIndicator group datum.1 * GHKR22BiasBounties.datumLoss loss model datum ∂law
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.datumLoss](#)
- [GHKR22BiasBounties.groupIndicator](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The integral is exactly group mass times conditional group loss, written without division so the source certificate objective remains meaningful on arbitrary laws.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.measureGroupMass

Paper-local declaration:

GHKR22BiasBounties.measureGroupMass

Source locator: cited publication, lines 312-318

Verbatim paper-source connection

```
\begin{definition}[Subgroups]
A \emph{subgroup} of the feature space  $\mathcal{C}X$  will be represented as an indicator function  $g:\mathcal{C}X \rightarrow \{0,1\}$ . We say that  $x \in \mathcal{C}X$  is in group
 $\hookrightarrow$   $g$  if  $g(x) = 1$  and  $x$  is \emph{not} in group  $g$  otherwise. Given a group  $g$ , we write  $\mu_g(\mathcal{C}D)$  to denote its measure under the
 $\hookrightarrow$  probability distribution  $\mathcal{C}D$ :
\ifdraft
 $\mu_g(\mathcal{C}D) = \Pr_{\mathcal{C}D}[g(x) = 1]$ .
\else  $\mu_g(\mathcal{C}D) = \Pr_{\mathcal{C}D}[g(x) = 1]$ . \fi
We write  $\mu_g(D)$  to denote the corresponding empirical measure under  $D$ , which results from viewing  $D$  as the uniform distribution over its
 $\hookrightarrow$  elements.
\end{definition}
```

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] → MeasureTheory.Measure (X × Y) → GHKR22BiasBounties.Group X → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (group : GHKR22BiasBounties.Group X) =>
  ∫ (datum : X × Y), GHKR22BiasBounties.groupIndicator group datum.1 ∂law
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.groupIndicator](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Integrating the Boolean subgroup indicator gives exactly $\Pr[g(x)=1]$, the source's Definition 1 quantity $\mu_g(D)$; this declaration is the direct measure-theoretic realization of that definition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.measureGroupLoss

Paper-local declaration:

GHKR22BiasBounties.measureGroupLoss

Source locator: cited publication, lines 321-330

Verbatim paper-source connection

```
\begin{definition}[Model Loss]
Given a model  $f: \mathcal{X} \rightarrow \mathcal{Y}$ 
We write  $L(\mathcal{D}, f)$  to denote the average loss of  $f$  on distribution  $\mathcal{D}$ :
\ifdraft
 $L(\mathcal{D}, f) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(f(x), y)]$ 
\else  $L(\mathcal{D}, f) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(f(x), y)]$ . \fi
We write  $L(\mathcal{D}, f, g)$  to denote the loss on  $f$  conditional on membership in  $g$ :
 $L(\mathcal{D}, f, g) = \mathbb{E}_{(x,y) \sim \mathcal{D}}[\ell(f(x), y) \mid g(x) = 1]$ .
Given a dataset  $D$ , we write  $L(D, f)$  and  $L(D, f, g)$  to denote the corresponding empirical losses on  $D$ , which result from viewing  $D$  as the uniform
 $\hookrightarrow$  distribution over its elements.
\end{definition}
```

Settled source reading The mass-weighted group-loss numerator is the total primitive; conditional group loss is totalized to zero at mass zero, and every divided source display is recovered on positive-mass groups.

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.Group X → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
(loss : GHKR22BiasBounties.BoundedLoss Y) (model : GHKR22BiasBounties.Model X Y)
(group : GHKR22BiasBounties.Group X) =>
GHKR22BiasBounties.measureGroupLossNumerator law loss model group / GHKR22BiasBounties.measureGroupMass law group
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.measureGroupLossNumerator](#)
- [GHKR22BiasBounties.measureGroupMass](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The quotient is the paper's conditional group loss, with the denominator equal to group mass and the null-group value totalized to zero.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.MeasureCertificateOfSuboptimality

Paper-local declaration:

GHKR22BiasBounties.MeasureCertificateOfSuboptimality

Source locator: cited publication, lines 377-384

Verbatim paper-source connection

```
\begin{definition}
A group indicator function  $g:\mathcal{X}\rightarrow\{0,1\}$  together with a model  $h:\mathcal{X}\rightarrow\mathcal{Y}$  form a  $(\mu, \Delta)$ -certificate of sub-optimality for
 $\hookrightarrow$  a model  $f$  under distribution  $\mathcal{D}$  if:
\begin{enumerate}
\item Group  $g$  has probability mass at least  $\mu$  under  $\mathcal{D}$ :  $\mu_g(\mathcal{D}) \geq \mu$ , and
\item  $h$  improves on the performance of  $f$  on group  $g$  by at least  $\Delta$ :  $L(\mathcal{D}, f, g) \geq L(\mathcal{D}, h, g) + \Delta$ 
\end{enumerate}
\end{definition}
\ifdraft We say that  $(g, h)$  form a certificate of sub-optimality for  $f$  if they form a  $(\mu, \Delta)$ -certificate of optimality for  $f$  for any constants
 $\hookrightarrow \mu, \Delta > 0$ . \fi
\end{definition}
```

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) →
GHKR22BiasBounties.BoundedLoss Y →
GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.Group X → GHKR22BiasBounties.Model X Y → ℝ → ℝ → Prop

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
(loss : GHKR22BiasBounties.BoundedLoss Y) (current : GHKR22BiasBounties.Model X Y)
(group : GHKR22BiasBounties.Group X) (replacement : GHKR22BiasBounties.Model X Y) (mu Delta : ℝ) =>
0 < mu ∧
0 < Delta ∧
mu ≤ GHKR22BiasBounties.measureGroupMass law group ∧
GHKR22BiasBounties.measureGroupLoss law loss replacement group + Delta ≤
GHKR22BiasBounties.measureGroupLoss law loss current group
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.measureGroupLoss](#)
- [GHKR22BiasBounties.measureGroupMass](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The predicate records positive μ and Δ , group mass at least μ , and replacement conditional loss improving current loss by at least Δ , exactly Definition 7.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.MeasureCostSensitiveMinimizer

Paper-local declaration:

GHKR22BiasBounties.MeasureCostSensitiveMinimizer

Source locator: cited publication, lines 868-874

Verbatim paper-source connection

```
\begin{definition}
A cost-sensitive classification problem is defined by a model class  $\mathcal{K}$  consisting of functions  $p:\mathcal{X}\rightarrow \mathcal{Y}$ , where  $\mathcal{Y}$  is some finite label set,
 $\hookrightarrow$  a distribution  $\mathcal{D}$  over  $\mathcal{X}\times \mathcal{Y}$ , and a set of real valued costs  $c_{(x,y)}$  for each pair  $(x, y)$  in the support of  $\mathcal{D}$  and each
 $\hookrightarrow$  label  $\hat{y}$  in  $\mathcal{Y}$ . A solution  $p^* \in \mathcal{K}$  to the cost-sensitive classification problem  $(\mathcal{D}, \mathcal{K}, \{c_{(x,y)}\})$  is given by:
\ifdraft

$$p^* \in \arg\min_{p \in \mathcal{K}} \mathbb{E}_{(x,y) \sim \mathcal{D}} [c_{(x,y)}(p(x))]$$

\else 
$$p^* \in \arg\min_{p \in \mathcal{K}} \mathbb{E}_{(x,y) \sim \mathcal{D}} [c_{(x,y)}(p(x))]$$
 \fi
i.e. the model that minimizes the expected costs for the labels it assigns to points drawn from  $\mathcal{D}$ .
\end{definition}
```

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
MeasureTheory.Measure (X × Bool) →
GHKR22BiasBounties.Model X Bool →
Set (GHKR22BiasBounties.TernaryPredictor X) → GHKR22BiasBounties.TernaryPredictor X → Prop

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
(current : GHKR22BiasBounties.Model X Bool) (K : Set (GHKR22BiasBounties.TernaryPredictor X))
(pStar : GHKR22BiasBounties.TernaryPredictor X) =>
pStar ∈ K ∧
∀ p ∈ K,
GHKR22BiasBounties.measureExpectedTernaryCost law current pStar ≤
GHKR22BiasBounties.measureExpectedTernaryCost law current p
```

Direct retained dependencies

- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TernaryPredictor](#)
- [GHKR22BiasBounties.measureExpectedTernaryCost](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The predicate says pStar belongs to K and minimizes the population expected pointwise cost over every p in K, exactly Definition 18.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.adaptiveFalsifyAndUpdate

Paper-local declaration:

GHKR22BiasBounties.adaptiveFalsifyAndUpdate

Source locator: cited publication, lines 604-631

Verbatim paper-source connection

```
\begin{algorithm}
\ifdraft
\else
\small
\fi
\KwInInput{An initial model  $f_0$ , a holdout dataset  $D$ , Target  $\epsilon$ , and a stream of submissions  $(g_1, h_1), (g_2, h_2), \dots$  of length at most  $U$ }
Let  $t \leftarrow 0$ 
Initialize an instance of CertificateChecker( $\epsilon, D, U, \dots$ )
\While{CertificateChecker has not halted}{
  Consider the next submission  $(g_i, h_i)$ 
  Feed the triple  $(f_t, g_i, h_i)$  to CertificateChecker and receive  $\pi_i \in \{\text{bot}, \text{top}\}$ 
  \If{ $\pi_i = \text{bot}$ }{
    \KwOut{Submission  $(g_i, h_i)$  is rejected.}
  }
  \Else{
    Let  $t \leftarrow t+1$  and let  $f_t = \text{ListUpdate}(f_{t-1}, (g_i, h_i))$ 
    \KwOut{Submission  $(g_i, h_i)$  is accepted. The new model is  $f_t$ .}
  }
}
\caption{FalsifyAndUpdate( $\epsilon, D, (f_1, g_1, h_1), \dots$ ): An algorithm that maintains a sequence of models  $f_1, \dots, f_T$  and accepts submissions
 $\hookrightarrow$  of proposed certificates of sub-optimality  $(g_i, h_i)$  that attempt to falsify the assertion that the current model  $f_t$  is not  $\epsilon$ -Bayes optimal.
 $\hookrightarrow$  It either accepts or rejects each submission: if it accepts a submission then it also updates its current model  $f_t$  and outputs a new model
 $\hookrightarrow f_{t+1}$ .}
\label{alg:FalsifyAndUpdate}
\end{algorithm}
```

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{n : ℕ} →
ℝ →
  GHKR22BiasBounties.BoundedLoss Y →
  (Fin n → X × Y) →
  ℕ →
    GHKR22BiasBounties.Model X Y →
    GHKR22BiasBounties.AdaptiveProposalStrategy X Y → GHKR22BiasBounties.FalsifyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} {n : ℕ} (epsilon : ℝ) (loss : GHKR22BiasBounties.BoundedLoss Y)
  (sample : Fin n → X × Y) (U : ℕ) (initial : GHKR22BiasBounties.Model X Y)
  (strategy : GHKR22BiasBounties.AdaptiveProposalStrategy X Y) =>
  GHKR22BiasBounties.falsifyStateFromTranscript initial strategy
  (GHKR22BiasBounties.adaptiveFalsifyTranscript epsilon loss sample U initial strategy)
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.FalsifyState](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.adaptiveFalsifyTranscript](#)
- [GHKR22BiasBounties.falsifyStateFromTranscript](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The function runs the transcript-adaptive checker strategy and reconstructs the current model and accepted updates, exactly Algorithm 3's finite-horizon behavior.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.certificateCheckerDecision

Paper-local declaration:

GHKR22BiasBounties.certificateCheckerDecision

Source locator: cited publication, lines 524-547

Verbatim paper-source connection

```
\begin{algorithm}
\ifdraft
\else
\small
\fi
\KwInput{Holdout dataset  $D$ , Target  $\epsilon$ , and a stream of submissions  $(f_1, g_1, h_1), (f_2, g_2, h_2), \dots$  of length at most  $U$ }
 $\mathbf{NumberAccepted} \leftarrow 0$ 

\While{ $\mathbf{NumberAccepted} \leq 2\epsilon$  and  $i \leq U$ }{
  Consider the next submission  $(f_i, g_i, h_i)$ 

  Compute  $\mu_i \leftarrow \mu_D(g_i)$ ,  $\Delta_i \leftarrow L(D, f_i, g_i) - L(D, h_i, g_i)$ 

  \If{ $\mu_i \cdot \Delta_i < \frac{3\epsilon}{4}$ } {
    \KwOut{ $\pi_i = \text{bot}$  (Submission Rejected)}
  }
  \Else{
    \KwOut{ $\pi_i = \text{top}$  (Submission Accepted)}
     $\mathbf{NumberAccepted} \leftarrow \mathbf{NumberAccepted} + 1$ .
  }
}
\caption{CertificateChecker( $\epsilon, D, U, (f_1, g_1, h_1), \dots$ ): An algorithm that takes as input a stream of submissions  $(f_i, g_i, h_i)$  and checks if  $(g_i, h_i)$  is a certificate of sub-optimality for  $f_i$  with sufficiently high parameter values.}
\label{alg:checker}
\end{algorithm}
```

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{n : ℕ} →
ℝ →
GHKR22BiasBounties.BoundedLoss Y →
(Fin n → X × Y) → GHKR22BiasBounties.Submission X Y → GHKR22BiasBounties.CertificateDecision

value:
fun {X : Type u_1} {Y : Type u_2} {n : ℕ} (epsilon : ℝ) (loss : GHKR22BiasBounties.BoundedLoss Y)
  (sample : Fin n → X × Y) (submission : GHKR22BiasBounties.Submission X Y) =>
  if GHKR22BiasBounties.empiricalSubmissionScore loss sample submission < 3 * epsilon / 4 then
    GHKR22BiasBounties.CertificateDecision.rejected
  else GHKR22BiasBounties.CertificateDecision.accepted
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.empiricalSubmissionScore](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The definition rejects precisely when the empirical mass-weighted loss improvement is below $3\epsilon / 4$ and accepts otherwise, exactly Algorithm 2.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.certificateCheckerRun

Paper-local declaration:

GHKR22BiasBounties.certificateCheckerRun

Source locator: cited publication, lines 524-547

Verbatim paper-source connection

```
\begin{algorithm}
\ifdraft
\else
\small
\fi
\KwInput{Holdout dataset  $D$ , Target  $\epsilon$ , and a stream of submissions  $(f_{1,g_1,h_1}, f_{2,g_2,h_2}, \dots)$  of length at most  $U$ }
 $\mathbf{NumberAccepted} \leftarrow 0$ 

\While{ $\mathbf{NumberAccepted} \leq 2\epsilon$  and  $i \leq U$ }{
  Consider the next submission  $(f_{i,g_i,h_i})$ 

  Compute  $\mu_i \leftarrow \mu_D(g_i)$ ,  $\Delta_i \leftarrow L(D, f_{i,g_i}) - L(D, h_{i,g_i})$ 

  \If{ $\mu_i \cdot \Delta_i < \frac{3\epsilon}{4}$ } {
    \KwOut{ $\pi_i = \text{bot}$  (Submission Rejected)}
  }
  \Else{
    \KwOut{ $\pi_i = \text{top}$  (Submission Accepted)}
     $\mathbf{NumberAccepted} \leftarrow \mathbf{NumberAccepted} + 1$ .
  }
}
\caption{CertificateChecker( $\epsilon, D, U, (f_{1,g_1,h_1}, \dots)$ ): An algorithm that takes as input a stream of submissions  $(f_{i,g_i,h_i})$  and checks if
 $\hookrightarrow (g_{i,h_i})$  is a certificate of sub-optimality for  $f_i$  with sufficiently high parameter values.}
\label{alg:checker}
\end{algorithm}
```

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{n : ℕ} →
ℝ →
GHKR22BiasBounties.BoundedLoss Y →
(Fin n → X × Y) →
ℕ → GHKR22BiasBounties.AdaptiveSubmissionStrategy X Y → List GHKR22BiasBounties.CertificateDecision

value:
fun {X : Type u_1} {Y : Type u_2} {n : ℕ} (epsilon : ℝ) (loss : GHKR22BiasBounties.BoundedLoss Y)
(sample : Fin n → X × Y) (U : ℕ) (strategy : GHKR22BiasBounties.AdaptiveSubmissionStrategy X Y) =>
GHKR22BiasBounties.certificateCheckerRunAux epsilon loss sample strategy U []
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveSubmissionStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.certificateCheckerRunAux](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The function exposes Algorithm 2's public decision stream for at most U queried submissions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.checkerAcceptanceBudget

Paper-local declaration:

GHKR22BiasBounties.checkerAcceptanceBudget

Source locator: cited publication, lines 524-547

Verbatim paper-source connection

```
\begin{algorithm}
\ifdraft
\else
\small
\fi
\KwInInput{Holdout dataset  $D$ , Target  $\epsilon$ , and a stream of submissions  $(f_1,g_1,h_1),(f_2,g_2,h_2),\ldots$  of length at most  $U$ }
 $\mathbf{NumberAccepted} \leftarrow 0$ 

\While{ $\mathbf{NumberAccepted} \leq 2/\epsilon$  and  $i \leq U$ }{
  Consider the next submission  $(f_i,g_i,h_i)$ 

  Compute  $\mu_i \leftarrow \mu_D(g_i)$ ,  $\Delta_i \leftarrow L(D,f_i,g_i) - L(D,h_i,g_i)$ 

  \If{ $\mu_i \cdot \Delta_i < \frac{3\epsilon}{4}$ } {
    \KwOut{ $\pi_i = \text{bot}$  (Submission Rejected)}
  }
  \Else{
    \KwOut{ $\pi_i = \text{top}$  (Submission Accepted)}
     $\mathbf{NumberAccepted} \leftarrow \mathbf{NumberAccepted} + 1$ .
  }
}
\caption{CertificateChecker( $\epsilon,D,U,(f_1,g_1,h_1),\ldots$ ): An algorithm that takes as input a stream of submissions  $(f_i,g_i,h_i)$  and checks if
 $\leftrightarrow$   $(g_i,h_i)$  is a certificate of sub-optimality for  $f_i$  with sufficiently high parameter values.}
\label{alg:checker}
\end{algorithm}
```

Lean-expanded paper semantic target

```
type:
ℝ → ℕ

value:
fun (epsilon : ℝ) => ⌊2 / epsilon⌋+
```

Recorded source-to-declaration screening Verdict: matches
Reason: floor(2/epsilon) is exactly the largest integer accepted count for which the printed non-strict loop-entry guard remains true; it does not erase the separately documented possible final acceptance.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

GHKR22BiasBounties.derivedGroup

Paper-local declaration:

GHKR22BiasBounties.derivedGroup

Source locator: cited publication, lines 847-865

Verbatim paper-source connection

```
\begin{definition}
Given a ternary valued function  $p: \mathcal{X} \rightarrow \{0,1,?\}$ , the  $p$ -derived group  $g_p$  and model  $h_p$  are defined as:
\ifdraft
 $g_p(x) = \begin{cases} 1 & \text{if } p(x) \in \{0,1\} \\ 0 & \text{if } p(x) = ? \end{cases}$ 
 $h_p(x) = \begin{cases} p(x) & \text{if } p(x) \in \{0,1\} \\ 0 & \text{if } p(x) = ? \end{cases}$ 
\else  $g_p(x) = \begin{cases} 1 & \text{if } p(x) \in \{0,1\} \\ 0 & \text{if } p(x) = ? \end{cases}$ 
 $h_p(x) = \begin{cases} p(x) & \text{if } p(x) \in \{0,1\} \\ 0 & \text{if } p(x) = ? \end{cases}$ 
\fi
In other words, interpreting  $p(x) = ?$  as the decision for  $p$  to ``defer'' on  $x$ ,  $g_p$  defines exactly the group of examples that  $p$  does \emph{not}
 $\hookrightarrow$  defer on, and  $h_p$  is the model that makes the same prediction as  $p$  on every example that  $p$  does not defer on. Given a class of ternary
 $\hookrightarrow$  functions  $K$ , let the  $K$ -derived certificates  $\mathcal{C}_K$  denote the set of pairs  $(g_p, h_p)$  that can be so derived from some  $p \in K$ :  $\mathcal{C}_K =$ 
 $\hookrightarrow \{(g_p, h_p) : p \in K\}$ . Similarly let  $\mathcal{G}_K = \{g_p : p \in K\}$  and  $\mathcal{H}_K = \{h_p : p \in K\}$  denote the class of group indicator functions and
 $\hookrightarrow$  models derived from  $K$  respectively.
\end{definition}
```

Lean-expanded paper semantic target

```
type:
{X : Type u_1} → GHKR22BiasBounties.TernaryPredictor X → X → Bool

value:
fun {X : Type u_1} (p : GHKR22BiasBounties.TernaryPredictor X) (a : X) => p a != GHKR22BiasBounties.TernaryLabel.defer
```

Direct retained dependencies

- [GHKR22BiasBounties.TernaryLabel](#)
- [GHKR22BiasBounties.TernaryPredictor](#)

Recorded source-to-declaration screening **Verdict:** matches

Reason: The Boolean group is true exactly where the ternary predictor does not defer, matching g_p in Definition 17.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.derivedModel

Paper-local declaration:

GHKR22BiasBounties.derivedModel

Source locator: cited publication, lines 847-865

Verbatim paper-source connection

```
\begin{definition}
Given a ternary valued function  $p: \mathcal{X} \rightarrow \{0,1,?\}$ , the  $p$ -derived group  $g_p$  and model  $h_p$  are defined as:
\ifdraft
 $g_p(x) = \begin{cases} 1 & \text{if } p(x) \in \{0,1\} \\ 0 & \text{if } p(x) = ? \end{cases}$ 
 $h_p(x) = \begin{cases} p(x) & \text{if } p(x) \in \{0,1\} \\ 0 & \text{if } p(x) = ? \end{cases}$ 
\else  $g_p(x) = \begin{cases} 1 & \text{if } p(x) \in \{0,1\} \\ 0 & \text{if } p(x) = ? \end{cases}$ 
 $h_p(x) = \begin{cases} p(x) & \text{if } p(x) \in \{0,1\} \\ 0 & \text{if } p(x) = ? \end{cases}$ 
\fi
In other words, interpreting  $p(x) = ?$  as the decision for  $p$  to “defer” on  $x$ ,  $g_p$  defines exactly the group of examples that  $p$  does \emph{not}
 $\hookrightarrow$  defer on, and  $h_p$  is the model that makes the same prediction as  $p$  on every example that  $p$  does not defer on. Given a class of ternary
 $\hookrightarrow$  functions  $K$ , let the  $K$ -derived certificates  $\mathcal{C}_K$  denote the set of pairs  $(g_p, h_p)$  that can be so derived from some  $p \in K$ :  $\mathcal{C}_K =$ 
 $\hookrightarrow \{(g_p, h_p) : p \in K\}$ . Similarly let  $\mathcal{C}_G = \{g_p : p \in K\}$  and  $\mathcal{C}_H = \{h_p : p \in K\}$  denote the class of group indicator functions and
 $\hookrightarrow$  models derived from  $K$  respectively.
\end{definition}
```

Lean-expanded paper semantic target

```
type:
{X : Type u_1} → GHKR22BiasBounties.TernaryPredictor X → X → Bool

value:
fun {X : Type u_1} (p : GHKR22BiasBounties.TernaryPredictor X) (a : X) =>
  match p a with
  | GHKR22BiasBounties.TernaryLabel.zero => false
  | GHKR22BiasBounties.TernaryLabel.one => true
  | GHKR22BiasBounties.TernaryLabel.defer => false
```

Direct retained dependencies

- [GHKR22BiasBounties.TernaryLabel](#)
- [GHKR22BiasBounties.TernaryPredictor](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The model copies zero and one predictions and assigns the source’s arbitrary zero value on deferral, exactly h_p in Definition 17.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.listUpdate

Paper-local declaration:

GHKR22BiasBounties.listUpdate

Source locator: cited publication, lines 408-424

Verbatim paper-source connection

```
\begin{algorithm}[H]
\ifdraft
\else
\small
\fi
\KwInput{A model  $f_t$ , and a certificate of suboptimality  $(g_{t+1}, h_{t+1})$ }
Define  $f_{t+1}$  as follows:
\[\ f_{t+1}(x) = \begin{cases} f_t(x) & \text{if } g_{t+1}(x) = 0 \\ h_{t+1}(x) & \text{if } g_{t+1}(x) = 1 \end{cases}
\]
\KwOutput{ $f_{t+1}$ }
\caption{ListUpdate( $f_t, (g_{t+1}, h_{t+1})$ ): An Update Algorithm Constructing a Decision List}
\label{alg:basic}
\end{algorithm}
```

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} → GHKR22BiasBounties.Model X Y → GHKR22BiasBounties.Group X → GHKR22BiasBounties.Model X Y → X → Y

value:
fun {X : Type u_1} {Y : Type u_2} (f : GHKR22BiasBounties.Model X Y) (g : GHKR22BiasBounties.Group X)
(h : GHKR22BiasBounties.Model X Y) (a : X) =>
if g a = true then h a else f a
```

Direct retained dependencies

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The function uses h where g is true and f where g is false, exactly Algorithm 1's displayed decision-list update.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.falsifyAndUpdateStep

Paper-local declaration:

GHKR22BiasBounties.falsifyAndUpdateStep

Source locator: cited publication, lines 604-631

Verbatim paper-source connection

```
\begin{algorithm}
\ifdraft
\else
\small
\fi
\KwInput{An initial model  $f_0$ , a holdout dataset  $D$ , Target  $\epsilon$ , and a stream of submissions  $(g_1, h_1), (g_2, h_2), \dots$  of length at most  $U$ }
Let  $t \leftarrow 0$ 

Initialize an instance of CertificateChecker( $\epsilon, D, U, \dots$ )

\While{CertificateChecker has not halted}{
  Consider the next submission  $(g_i, h_i)$ 

  Feed the triple  $(f_t, g_i, h_i)$  to CertificateChecker and receive  $\pi_i$  in  $\{\text{bot}, \text{top}\}$ 

  \If{ $\pi_i = \text{bot}$ }{
    \KwOut{Submission  $(g_i, h_i)$  is rejected.}
  }
  \Else{
    Let  $t \leftarrow t+1$  and let  $f_t = \text{ListUpdate}(f_{t-1}, (g_i, h_i))$ 
    \KwOut{Submission  $(g_i, h_i)$  is accepted. The new model is  $f_t$ .}
  }
}

\caption{FalsifyAndUpdate( $\epsilon, D, (f_1, g_1, h_1), \dots$ ): An algorithm that maintains a sequence of models  $f_1, \dots, f_T$  and accepts submissions
 $\hookrightarrow$  of proposed certificates of sub-optimality  $(g_i, h_i)$  that attempt to falsify the assertion that the current model  $f_t$  is not  $\epsilon$ -Bayes optimal.
 $\hookrightarrow$  It either accepts or rejects each submission: if it accepts a submission then it also updates its current model  $f_t$  and outputs a new model
 $\hookrightarrow f_{t+1}$ .}
\label{alg:FalsifyAndUpdate}
\end{algorithm}
```

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{n : ℕ} →
ℝ →
  GHKR22BiasBounties.BoundedLoss Y →
  (Fin n → X × Y) →
  GHKR22BiasBounties.FalsifyState X Y →
  GHKR22BiasBounties.ProposedUpdate X Y → GHKR22BiasBounties.FalsifyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} {n : ℕ} (epsilon : ℝ) (loss : GHKR22BiasBounties.BoundedLoss Y)
  (sample : Fin n → X × Y) (state : GHKR22BiasBounties.FalsifyState X Y)
  (proposal : GHKR22BiasBounties.ProposedUpdate X Y) =>
have decision : GHKR22BiasBounties.CertificateDecision :=
  GHKR22BiasBounties.certificateCheckerDecision epsilon loss sample
  (GHKR22BiasBounties.proposalSubmission state proposal);
match decision with
| GHKR22BiasBounties.CertificateDecision.rejected =>
{ current := state.current, accepted := state.accepted, reverseTranscript := decision :: state.reverseTranscript }
| GHKR22BiasBounties.CertificateDecision.accepted =>
{ current := GHKR22BiasBounties.listUpdate state.current proposal.group proposal.replacement,
  accepted := state.accepted + 1, reverseTranscript := decision :: state.reverseTranscript }
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.FalsifyState](#)
- [GHKR22BiasBounties.ProposedUpdate](#)
- [GHKR22BiasBounties.certificateCheckerDecision](#)
- [GHKR22BiasBounties.listUpdate](#)
- [GHKR22BiasBounties.proposalSubmission](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The transition leaves the model unchanged after rejection and applies ListUpdate while incrementing the accepted count after acceptance, exactly Algorithm 3.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.inducedCost

Paper-local declaration:

GHKR22BiasBounties.inducedCost

Source locator: cited publication, lines 878-889

Verbatim paper-source connection

```
\begin{definition}
Given a binary model  $f: \mathcal{X} \rightarrow \mathcal{Y}$ , the induced costs of  $f$  are defined as follows:
\ifdraft

$$c^f_{(x,y)}(\hat{y}) = \begin{cases} 0 & \text{if } \hat{y} = ? \\ 1 & \text{if } f(x) = y \neq \hat{y} \\ -1 & \text{if } \hat{y} = y \neq f(x) \\ 0 & \text{otherwise.} \end{cases}$$

\end{cases}
\else  $c^f_{(x,y)}(\hat{y}) = 0$  if  $\hat{y} = ?$  or if  $\hat{y} = f(x)$ ,  $c^f_{(x,y)}(\hat{y}) = 1$  if  $f(x) = y \neq \hat{y}$ , and  $c^f_{(x,y)}(\hat{y}) = -1$  if
 $\hat{y} = y \neq f(x)$ .
\fi
Intuitively, it costs nothing to defer a decision to the existing model  $f$  --- or equivalently, to make the same decision as  $f$ . On the other hand, making
the wrong decision on an example  $x$  costs  $1$  when  $f$  would have made the right decision, and making the right decision ``earns"  $1$ 
when  $f$  would not have.
\end{definition}
```

Lean-expanded paper semantic target

```
type:
{X : Type u_1} → GHKR22BiasBounties.Model X Bool → X × Bool → GHKR22BiasBounties.TernaryLabel → ℝ

value:
fun {X : Type u_1} (current : GHKR22BiasBounties.Model X Bool) (datum : X × Bool)
  (prediction : GHKR22BiasBounties.TernaryLabel) =>
  match prediction with
  | GHKR22BiasBounties.TernaryLabel.defer => 0
  | label =>
    GHKR22BiasBounties.binaryZeroOneLoss.value (GHKR22BiasBounties.ternaryBinaryPrediction label) datum.2 -
    GHKR22BiasBounties.binaryZeroOneLoss.value (current datum.1) datum.2
```

Direct retained dependencies

- [AppliedModelingLib.Learning.Prediction.BoundedLoss](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TernaryLabel](#)
- [GHKR22BiasBounties.binaryZeroOneLoss](#)
- [GHKR22BiasBounties.ternaryBinaryPrediction](#)

Recorded source-to-declaration screening **Verdict:** matches

Reason: Deferral costs zero and a binary prediction costs its zero-one loss minus the current model's loss, which expands to exactly Definition 19's cases.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.kernelConditionalRisk

Paper-local declaration:

GHKR22BiasBounties.kernelConditionalRisk

Source locator: cited publication, lines 333-339

Verbatim paper-source connection

```
\begin{definition}
A \emph{Bayes Optimal} model  $f^*: \mathcal{X} \rightarrow \mathcal{Y}$  with respect to a loss function  $\ell$  and a distribution  $\mathcal{D}$  satisfies:
\ifdraft

$$f^*(x) \in \arg\min_{y \in \mathcal{Y}} \mathbb{E}_{(x', y') \sim \mathcal{D}} [\ell(y, y') \mid x' = x]$$

\else 
$$f^*(x) \in \arg\min_{y \in \mathcal{Y}} \mathbb{E}_{(x', y') \sim \mathcal{D}} [\ell(y, y') \mid x' = x]$$
, \fi
where  $f^*(x)$  can be defined arbitrarily for any  $x$  that is not in the support of  $\mathcal{D}$ .
\end{definition}
```

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For an arbitrary population law, Bayes optimality means almost-everywhere minimization of conditional risk under a measurable regular conditional label $\hookrightarrow$ kernel; off the governing feature law the model is irrelevant.

Archival source anchor: cited publication, lines 333-339

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] → ProbabilityTheory.Kernel X Y → GHKR22BiasBounties.BoundedLoss Y → X → Y → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (labels : ProbabilityTheory.Kernel X Y)
  (loss : GHKR22BiasBounties.BoundedLoss Y) (x : X) (prediction : Y) =>
  ∫ (truth : Y), loss.value prediction truth ∂labels x
```

Direct retained dependencies

- [AppliedModelingLib.Learning.Prediction.BoundedLoss](#)
- [GHKR22BiasBounties.BoundedLoss](#)

Recorded source-to-declaration screening Verdict: matches_approved_corrected target

Reason: Compared Definition 3 and its bound almost-everywhere correction with the actual integral of loss.value prediction truth against labels x. Prediction and realized truth occupy the source loss arguments in the correct order. The displayed source_observation4Spec uses a probability feature law, a Markov label kernel, measurable bounded loss and measurable current model, so the generic kernel integral specializes to the source conditional risk. This is the risk component of the corrected Bayes target; it adds no off-law pointwise optimality claim.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.measureCoordinateAscentStep

Paper-local declaration:

GHKR22BiasBounties.measureCoordinateAscentStep

Source locator: cited publication, lines 991-1005

Verbatim paper-source connection

```
\begin{algorithm}
\KwIn{A dataset  $\mathcal{D}$ , a model  $f_{t-1}$ , group and model classes  $\mathcal{C}_G$  and  $\mathcal{C}_H$ , and an error parameter  $\epsilon$ .}
Let  $(g^*, h^*) \in \mathcal{C}_G \times \mathcal{C}_H$  be an arbitrary initial certificate. \
CurrentValue  $\leftarrow \mu_{\mathcal{D}}(g^*) \cdot (L(\mathcal{D}, f_{t-1}, g^*) - L(\mathcal{D}, h^*, g^*))$  \
Let:
 $\$h^* = \arg\min_{h \in \mathcal{C}_H} L(\mathcal{D}, \{g^*\}, h)$  \ \ \ \  $g^* = \arg\min_{g \in \mathcal{C}_G} L(\mathcal{D}, \{h^*\}, g)$  \
\While{ $\mu_{\mathcal{D}}(\{g^*\}) \cdot (L(\mathcal{D}, f_{t-1}, g^*) - L(\mathcal{D}, h^*, g^*)) \geq \text{CurrentValue} + \epsilon$ } {
CurrentValue  $\leftarrow \mu_{\mathcal{D}}(\{g^*\}) \cdot (L(\mathcal{D}, f_{t-1}, g^*) - L(\mathcal{D}, h^*, g^*))$  \
Let:
 $\$h^* = \arg\min_{h \in \mathcal{C}_H} L(\mathcal{D}, \{g^*\}, h)$  \ \ \ \  $g^* = \arg\min_{g \in \mathcal{C}_G} L(\mathcal{D}, \{h^*\}, g)$  \
}
\KwOut{Certificate  $(g_t, h_t) = (g^*, h^*)$ }
\caption{AltMinCertificateFinder( $\mathcal{D}, f_{t-1}, \mathcal{C}_G, \mathcal{C}_H, \epsilon$ ): An algorithm for finding  $\epsilon$ -locally optimal certificates of sub-optimality  $(g_t, h_t) \in \mathcal{C}_G \times \mathcal{C}_H$  for model  $f_{t-1}$ .}
\label{alg:altmin}
\end{algorithm}
```

Formalized review target The archival source above is not asserted equivalent to this formalized target.

At each state, test the exact model best-response gap and the exact group best-response gap separately, take one coordinate move only if its gap $\hookrightarrow$ exceeds epsilon, and halt only when neither gap does.

Archival source anchor: cited publication, lines 991-1005

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
(law : MeasureTheory.Measure (X × Bool)) →
(current : GHKR22BiasBounties.Model X Bool) →
ℝ →
{G : Set (GHKR22BiasBounties.Group X)} →
{H : Set (GHKR22BiasBounties.Model X Bool)} →
GHKR22BiasBounties.MeasureCoordinateBestResponses law current G H →
GHKR22BiasBounties.CertificatePair X → Option (GHKR22BiasBounties.CertificatePair X)

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
(current : GHKR22BiasBounties.Model X Bool) (epsilon : ℝ) {G : Set (GHKR22BiasBounties.Group X)}
{H : Set (GHKR22BiasBounties.Model X Bool)}
(oracle : GHKR22BiasBounties.MeasureCoordinateBestResponses law current G H)
(pair : GHKR22BiasBounties.CertificatePair X) =>
have modelResponse : GHKR22BiasBounties.CertificatePair X :=
{ group := pair.group, replacement := oracle.bestModel pair.group };
if
GHKR22BiasBounties.MeasureBinaryCertificateObjective law current pair.group pair.replacement + epsilon <
GHKR22BiasBounties.MeasureBinaryCertificateObjective law current modelResponse.group
modelResponse.replacement then
some modelResponse
else
have groupResponse : GHKR22BiasBounties.CertificatePair X :=
{ group := oracle.bestGroup pair.replacement, replacement := pair.replacement };
if
GHKR22BiasBounties.MeasureBinaryCertificateObjective law current pair.group pair.replacement + epsilon <
GHKR22BiasBounties.MeasureBinaryCertificateObjective law current groupResponse.group
groupResponse.replacement then
some groupResponse
else none
```

Direct retained dependencies

- [GHKR22BiasBounties.CertificatePair](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasureBinaryCertificateObjective](#)
- [GHKR22BiasBounties.MeasureCoordinateBestResponses](#)
- [GHKR22BiasBounties.Model](#)

Recorded source-to-declaration screening Verdict: matches_approved_corrected_target

Reason: Compared Algorithm 6 with its bound two-gap correction. Exact bestModel and bestGroup maximize the displayed mass-weighted binary error improvement over their classes. The step first changes only the model if its objective gain strictly exceeds epsilon; otherwise it tests a group-only response at the original pair, changing only that coordinate if its gain exceeds epsilon. It returns none precisely when both gaps are at most epsilon. The binary zero-one specialization is the source Section 4 optimization setting, and the displayed positive-initialization theorem context preserves the corrected certificate scope.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

GHKR22BiasBounties.measureCoordinateAscentLoop

Paper-local declaration:

GHKR22BiasBounties.measureCoordinateAscentLoop

Source locator: cited publication, lines 991-1005

Verbatim paper-source connection

```
\begin{algorithm}
\KwIn{A dataset  $\mathcal{D}$ , a model  $f_{t-1}$ , group and model classes  $\mathcal{C}_G$  and  $\mathcal{C}_H$ , and an error parameter  $\epsilon$ .}
Let  $(g^*, h^*) \in \mathcal{C}_G \times \mathcal{C}_H$  be an arbitrary initial certificate.  $\ll$ 
CurrentValue  $\leftarrow \mu_{\mathcal{D}}(g^*) \cdot (L(\mathcal{D}, f_{t-1}, g^*) - L(\mathcal{D}, h^*, g^*))$   $\ll$ 
Let:

$$g^* = \arg\min_{g \in \mathcal{C}_G} L(\mathcal{D}, g^*, h) \quad \ll \quad g^* = \arg\min_{g \in \mathcal{C}_G} L(\mathcal{D}, h^*, g)$$

While  $\mu_{\mathcal{D}}(g^*) \cdot (L(\mathcal{D}, f_{t-1}, g^*) - L(\mathcal{D}, h^*, g^*)) \geq \text{CurrentValue} + \epsilon$  {
CurrentValue  $\leftarrow \mu_{\mathcal{D}}(g^*) \cdot (L(\mathcal{D}, f_{t-1}, g^*) - L(\mathcal{D}, h^*, g^*))$   $\ll$ 
Let:

$$h^* = \arg\min_{h \in \mathcal{C}_H} L(\mathcal{D}, g^*, h) \quad \ll \quad g^* = \arg\min_{g \in \mathcal{C}_G} L(\mathcal{D}, h^*, g)$$

}
\KwOut{Certificate  $(g_t, h_t) = (g^*, h^*)$ }
\caption{AltMinCertificateFinder( $\mathcal{D}, f_{t-1}, \mathcal{C}_G, \mathcal{C}_H, \epsilon$ ): An algorithm for finding  $\epsilon$ -locally optimal certificates of sub-optimality  $(g_t, h_t) \in \mathcal{C}_G \times \mathcal{C}_H$  for model  $f_{t-1}$ .}
\label{alg:altmin}
\end{algorithm}
```

Formalized review target The archival source above is not asserted equivalent to this formalized target.

At each state, test the exact model best-response gap and the exact group best-response gap separately, take one coordinate move only if its gap exceeds epsilon, and halt only when neither gap does.

Archival source anchor: cited publication, lines 991-1005

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
[inst : MeasurableSpace X] →
(law : MeasureTheory.Measure (X × Bool)) →
(current : GHKR22BiasBounties.Model X Bool) →
ℝ →
{G : Set (GHKR22BiasBounties.Group X)} →
{H : Set (GHKR22BiasBounties.Model X Bool)} →
GHKR22BiasBounties.MeasureCoordinateBestResponses law current G H →
ℕ → GHKR22BiasBounties.CertificatePair X → Option (GHKR22BiasBounties.CertificatePair X)

value:
fun {X : Type u_1} [MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
(current : GHKR22BiasBounties.Model X Bool) (epsilon : ℝ) {G : Set (GHKR22BiasBounties.Group X)}
{H : Set (GHKR22BiasBounties.Model X Bool)}
(oracle : GHKR22BiasBounties.MeasureCoordinateBestResponses law current G H) (a : ℕ)
(a_1 : GHKR22BiasBounties.CertificatePair X) =>
Nat.brecOn (motive := fun (x : ℕ) =>
GHKR22BiasBounties.CertificatePair X → Option (GHKR22BiasBounties.CertificatePair X)) a
(GHKR22BiasBounties.measureCoordinateAscentLoop_f law current epsilon oracle) a_1
```

Direct retained dependencies

- [GHKR22BiasBounties.CertificatePair](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasureCoordinateBestResponses](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.measureCoordinateAscentStep](#)

Recorded source-to-declaration screening Verdict: matches_approved_corrected_target

Reason: Compared Algorithm 6 (main.tex:991-1005) under algorithm-6-two-gap-and-positive-initialization with the full recursive runner and best-response/objective displays. Each iteration invokes the same-state two-gap step; it returns a pair only when that step has neither improving coordinate. Exhausted fuel returns none if a move remains. The displayed Theorem 23 use supplies a positive initial certificate, admissibility, positive epsilon and the canonical rounded fuel, and concludes a returned locally optimal certificate. Thus finite fuel does not masquerade as successful halting before the corrected tests pass; empirical source data is a finite-law instance of this measure runner.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.measureModelLoss

Paper-local declaration:

GHKR22BiasBounties.measureModelLoss

Source locator: cited publication, lines 321-330

Verbatim paper-source connection

```
\begin{definition}[Model Loss]
Given a model  $f: \mathcal{X} \rightarrow \mathcal{Y}$ 
We write  $L(\mathcal{C}D, f)$  to denote the average loss of  $f$  on distribution  $\mathcal{C}D$ :
\ifdraft

$$L(\mathcal{C}D, f) = \mathbb{E}_{(x,y) \sim \mathcal{C}D} [\ell(f(x), y)]$$

\else 
$$L(\mathcal{C}D, f) = \mathbb{E}_{(x,y) \sim \mathcal{C}D} [\ell(f(x), y)]$$
. \fi
We write  $L(\mathcal{C}D, f, g)$  to denote the loss on  $f$  conditional on membership in  $g$ :

$$L(\mathcal{C}D, f, g) = \mathbb{E}_{(x,y) \sim \mathcal{C}D} [\ell(f(x), y) \mid g(x) = 1]$$

Given a dataset  $D$ , we write  $L(D, f)$  and  $L(D, f, g)$  to denote the corresponding empirical losses on  $D$ , which result from viewing  $D$  as the uniform
 $\hookrightarrow$  distribution over its elements.
\end{definition}
```

Settled source reading The mass-weighted group-loss numerator is the total primitive; conditional group loss is totalized to zero at mass zero, and every divided source display is recovered on positive-mass groups.

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
[inst : MeasurableSpace X] →
[inst_1 : MeasurableSpace Y] →
MeasureTheory.Measure (X × Y) → GHKR22BiasBounties.BoundedLoss Y → GHKR22BiasBounties.Model X Y → ℝ

value:
fun {X : Type u_1} {Y : Type u_2} [MeasurableSpace X] [MeasurableSpace Y] (law : MeasureTheory.Measure (X × Y))
  (loss : GHKR22BiasBounties.BoundedLoss Y) (model : GHKR22BiasBounties.Model X Y) =>
  ∫ (datum : X × Y), GHKR22BiasBounties.datumLoss loss model datum dlaw
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.datumLoss](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The integral of pointwise prediction loss under the joint law is exactly the source population model loss $L(D, f)$.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.monotoneBountyOutput

Paper-local declaration:

GHKR22BiasBounties.monotoneBountyOutput

Source locator: cited publication, lines 671-713

Verbatim paper-source connection

```
\begin{algorithm}
\ifdraftelse
\small
\fi
\KwIn{A model  $f_0$ , a holdout dataset  $D$ , Target  $\epsilon$ , and a stream of submissions  $(g_1, h_1), (g_2, h_2), \dots$  of length at most  $U$ }
Let  $t \leftarrow 0$  and Initialize an instance of CertificateChecker( $\epsilon, D, U + \frac{8}{\epsilon^3}$ ,  $\dots$ )

\While{CertificateChecker has not halted}{
  Consider the next submission  $(g_i, h_i)$ 

  Feed the triple  $(f_t, g_i, h_i)$  to CertificateChecker and receive  $\pi_i \in \{\text{bot}, \text{top}\}$ 

  \If{ $\pi_i = \text{bot}$ }{
    \KwOut{Submission  $(g_i, h_i)$  is rejected.}
  }
  \Else{
    MonotoneProgress  $\leftarrow$  FALSE
     $(g_U, h_U) \leftarrow (g_i, h_i)$ 
     $t' \leftarrow t$  and  $f_{t'} \leftarrow f_t$ 
    \While{MonotoneProgress  $\neq$  FALSE and CertificateChecker has not halted}{
      Let  $t' \leftarrow t' + 1$  and let  $f_{t'} = \text{ListUpdate}(f_{t-1}, (g_U, h_U))$ 
      MonotoneProgress  $\leftarrow$  TRUE
    }

    \For{each pair  $\ell < t, g_j$  in  $\text{CG}(f_t)$ }{
      Feed the triple  $(f_t, g_j, f_{\ell})$  to CertificateChecker and receive  $\pi_U \in \{\text{bot}, \text{top}\}$ 
      \If{ $\pi_U = \text{top}$  (Submission accepted)} {
        MonotoneProgress  $\leftarrow$  FALSE
         $(g_U, h_U) \leftarrow (g_j, f_{\ell})$ 
      }
    }
  }

  Let  $t \leftarrow t + 1$  and let  $f_t = f_{t'}$ 
}

\KwOut{Submission  $(g_i, h_i)$  is accepted. The new model is  $f_t$ .}
}

\caption{MonotoneFalsifyAndUpdate( $\epsilon, D, (f_1, g_1, h_1), \dots$ ): A version of FalsifyAndUpdate that guarantees approximate group-wise error  $\hookrightarrow$  monotonicity.}
\label{alg:MonFalsifyAndUpdate}
\end{algorithm}
```

Formalized review target The archival source above is not asserted equivalent to this formalized target.

External submissions and all historical repairs use one shared transcript-driven checker; after every accepted external or repair update, the $\hookrightarrow$ protected-history scan restarts before a model is published.

Archival source anchor: cited publication, lines 671-713

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{n : ℕ} →
ℝ →
GHKR22BiasBounties.BoundedLoss Y →
(Fin n → X × Y) →
ℕ →
GHKR22BiasBounties.Model X Y →
GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y → GHKR22BiasBounties.MonotoneBountyState X Y

value:
fun {X : Type u_1} {Y : Type u_2} {n : ℕ} (epsilon : ℝ) (loss : GHKR22BiasBounties.BoundedLoss Y)
(sample : Fin n → X × Y) (U : ℕ) (initial : GHKR22BiasBounties.Model X Y)
(strategy : GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y) =>
GHKR22BiasBounties.monotoneBountyStateFromTranscript initial U strategy
(GHKR22BiasBounties.monotoneBountyTranscript epsilon loss sample U initial strategy)
```

Direct retained dependencies

- [GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Model](#)

- [GHKR22BiasBounties.MonotoneBountyState](#)
- [GHKR22BiasBounties.monotoneBountyStateFromTranscript](#)
- [GHKR22BiasBounties.monotoneBountyTranscript](#)

Recorded source-to-declaration screening Verdict: matches approved corrected target

Reason: Compared Algorithm 4 (main.tex:671-713) under algorithm-4-shared-state-and-restart with the full checker, transcript, replay, submission and repair-state displays. One shared adaptive checker transcript drives external and historical-repair proposals. Every accepted proposal applies ListUpdate to the current model and restarts the full protected-group/past-model grid; a publication occurs after the repair scan closes. Rejected repairs advance that scan, and completion uses zero-score no-ops. The source-use display supplies positive epsilon and the shared query horizon. These are the bound corrected shared-state/restart semantics, not the stale-state archival pseudocode.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

GHKR22BiasBounties.trainByOptRun

Paper-local declaration:

GHKR22BiasBounties.trainByOptRun

Source locator: cited publication, lines 786-811

Verbatim paper-source connection

```
\begin{algorithm}
\ifdraft
\else
\small
\fi
\KwInput{A dataset  $D$ , a set of group/model pairs  $\mathcal{C}$ , and accuracy parameter  $\epsilon$ .}
Let  $f_0: \mathcal{C} \rightarrow \{0,1\}$  be an arbitrary initial model and  $T = 2/\epsilon$ 
Randomly divide  $D$  into  $T$  equally sized datasets:  $D_1, \dots, D_T$ .

\For{ $t = 1$  to  $T$ }{
  Let
   $(g_t, h_t) = \arg\max_{(g,h) \in \mathcal{C}} (\mu_D(g) \cdot (L(D_{f_{t-1}}, g) - L(D_{h_t}, g)))$ 

  \If{ $\mu_D(g_t) \cdot (L(D_{f_{t-1}}, g_t) - L(D_{h_t}, g_t)) \leq \frac{3\epsilon}{4}$ }{
    \KwOut{Model  $f_{t-1}$ }
  }
  \Else{
    Let  $f_t = \text{ListUpdate}(f_{t-1}, (g_t, h_t))$ 
  }
}
\KwOut{Model  $f_T$ }

\caption{TrainByOpt( $D, \mathcal{C}, \epsilon$ ): An algorithm for training an  $(\epsilon, \mathcal{C})$ -Bayes optimal model given the ability to optimize over certificates}
 $\hookrightarrow$   $(g, h) \in \mathcal{C}$ .
\label{alg:TrainByOpt}
\end{algorithm}
```

Formalized review target The archival source above is not asserted equivalent to this formalized target.

TrainByOpt optimizes and tests on the fresh block D_t at round t and runs for $\text{ceil}(2/\epsilon)$ rounds, applying ListUpdate exactly when the empirical
 $\hookrightarrow$ objective exceeds $3\epsilon / 4$.

Archival source anchor: cited publication, lines 786-811

Lean-expanded paper semantic target

```
type:
{X : Type u_1} →
{Y : Type u_2} →
{n : ℕ} →
  GHKR22BiasBounties.BoundedLoss Y →
  ℝ →
  GHKR22BiasBounties.TrainingOracle X Y n →
  GHKR22BiasBounties.Model X Y →
  (rounds : ℕ) → (Fin rounds → Fin n → X × Y) → GHKR22BiasBounties.TrainByOptOutcome X Y

value:
fun {X : Type u_1} {Y : Type u_2} {n : ℕ} (loss : GHKR22BiasBounties.BoundedLoss Y) (epsilon : ℝ)
  (oracle : GHKR22BiasBounties.TrainingOracle X Y n) (initial : GHKR22BiasBounties.Model X Y) (rounds : ℕ)
  (a : Fin rounds → Fin n → X × Y) =>
  Nat.brecOn (motive := fun (x : ℕ) => (Fin x → Fin n → X × Y) → GHKR22BiasBounties.TrainByOptOutcome X Y) rounds
  (GHKR22BiasBounties.trainByOptRun._f loss epsilon oracle initial) a
```

Direct retained dependencies

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CandidatePair](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TrainByOptOutcome](#)
- [GHKR22BiasBounties.TrainingOracle](#)
- [GHKR22BiasBounties.empiricalCandidateObjective](#)
- [GHKR22BiasBounties.listUpdate](#)

Recorded source-to-declaration screening **Verdict:** matches_approved_corrected_target

Reason: Compared Algorithm 5 (main.tex:786-811) with algorithm-5-fresh-block-and-integer-horizon. The recursive runner uses successive fresh trace blocks; its oracle receives the current model and that same block, and its empirical test uses that block. It stops at objective at most $3 \epsilon / 4$, otherwise performs ListUpdate; after stopping it ignores later blocks. The displayed Theorem 16 instantiates the generic horizon with `trainByOptRoundBudget (ceil(2/epsilon))`, requires an exact empirical optimizer, and evaluates fresh-block sampling. Thus the general runner contains the corrected source execution with its exact threshold and integer horizon.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 342-348

Verbatim source input

```
\begin{observation}
\label{obs:BO}
Fixing a loss function  $\ell$  and a distribution  $\mathcal{C}$ ,  $f^*$  is a Bayes optimal model if and only if for every group  $g$  and every alternative model  $h$ :
\ifdraft
 $L(\mathcal{C}, f^*, g) \leq L(\mathcal{C}, h, g)$ 
\else  $L(\mathcal{C}, f^*, g) \leq L(\mathcal{C}, h, g)$ . \fi
\end{observation}
```

Formalized review target The archival source above is not asserted equivalent to this formalized target.

Under a measurable conditional-label kernel and an arbitrary label space, a model is almost everywhere no worse than every integrable alternative model

- ↪ if and only if no such alternative improves its integrated conditional risk on any measurable subgroup; for countable labels this agrees with
- ↪ common-null-set actionwise Bayes optimality.

Archival source anchor: cited publication, lines 342-348

Expanded PaperInterface specification

```
∀ {X : Type u_1} {Y : Type u_2} [inst : MeasurableSpace X] [inst_1 : MeasurableSpace Y]
(featureLaw : MeasureTheory.Measure X) [MeasureTheory.IsProbabilityMeasure featureLaw]
(labels : ProbabilityTheory.Kernel X Y) [ProbabilityTheory.IsMarkovKernel labels]
(loss : GHKR22BiasBounties.BoundedLoss Y),
GHKR22BiasBounties.MeasurableBoundedLoss loss →
  ∀ (current : GHKR22BiasBounties.Model X Y),
    GHKR22BiasBounties.MeasurableModel current →
      (GHKR22BiasBounties.AEOptimalAgainstEveryIntegrableModelForRisk featureLaw
        (GHKR22BiasBounties.kernelConditionalRisk labels loss) current ↔
        GHKR22BiasBounties.OptimalOnEveryMeasurableGroupForRisk featureLaw
        (GHKR22BiasBounties.kernelConditionalRisk labels loss) current)
```

Retained governing declarations

- [GHKR22BiasBounties.AEOptimalAgainstEveryIntegrableModelForRisk](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.MeasurableBoundedLoss](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.OptimalOnEveryMeasurableGroupForRisk](#)
- [GHKR22BiasBounties.kernelConditionalRisk](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: MeasurableSpace X
4. parameter: MeasurableSpace Y
5. parameter: MeasureTheory.Measure X
6. assumption: MeasureTheory.IsProbabilityMeasure featureLaw
7. parameter: ProbabilityTheory.Kernel X Y
8. assumption: ProbabilityTheory.IsMarkovKernel labels
9. parameter: GHKR22BiasBounties.BoundedLoss Y
10. assumption: GHKR22BiasBounties.MeasurableBoundedLoss loss
11. parameter: GHKR22BiasBounties.Model X Y
12. assumption: GHKR22BiasBounties.MeasurableModel current
13. conclusion: GHKR22BiasBounties.AEOptimalAgainstEveryIntegrableModelForRisk featureLaw
(GHKR22BiasBounties.kernelConditionalRisk labels loss) current ↔
GHKR22BiasBounties.OptimalOnEveryMeasurableGroupForRisk featureLaw
(GHKR22BiasBounties.kernelConditionalRisk labels loss) current

Lean proof endpoint (built separately)

GHKR22BiasBounties.source_observation4

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: The transparent target states that almost-everywhere conditional-risk optimality against every integrable alternative is equivalent to optimality on every measurable subgroup under the conditional-label kernel. This is exactly the approved arbitrary-distribution correction; for countable labels it recovers a common-null-set actionwise Bayes interpretation without asserting the false literal pointwise claim at null features.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 389-392

Verbatim source input

```
\begin{restatable}{theorem}{BOQ}
\label{thm:BOQ}
Fix any  $\epsilon > 0$ , and any collection of (group,model) pairs  $(g,h)$ . There exists a  $(\mu,\Delta)$ -certificate of sub-optimality  $(g,h)$  in  $\mathbb{C}$  for  $f$  if
 $\Leftrightarrow$  and only if  $f$  is not  $(\epsilon,\mathbb{C})$ -Bayes optimal for  $\epsilon < \mu\Delta$ .
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {X : Type u_1} {Y : Type u_2} [inst : MeasurableSpace X] [inst_1 : MeasurableSpace Y]
(law : MeasureTheory.Measure (X × Y)) [MeasureTheory.IsProbabilityMeasure law]
(loss : GHKR22BiasBounties.BoundedLoss Y),
GHKR22BiasBounties.MeasurableBoundedLoss loss →
  ∀ (certificates : Set (GHKR22BiasBounties.Group X × GHKR22BiasBounties.Model X Y)),
  (∀ pair ∈ certificates, GHKR22BiasBounties.MeasurableGroup pair.1 ∧ GHKR22BiasBounties.MeasurableModel pair.2) →
    ∀ (epsilon : ℝ),
    0 ≤ epsilon →
      ∀ (current : GHKR22BiasBounties.Model X Y),
      GHKR22BiasBounties.MeasurableModel current →
        ((∃ (group : GHKR22BiasBounties.Group X) (replacement : GHKR22BiasBounties.Model X Y) (mu : ℝ) (Delta :
          ℝ),
          (group, replacement) ∈ certificates ∧
          GHKR22BiasBounties.MeasureCertificateOfSuboptimality law loss current group replacement mu Delta ∧
          epsilon < mu * Delta) ↔
          ¬GHKR22BiasBounties.MeasureApproxBayesOptimal law loss certificates epsilon current)
```

Retained governing declarations

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasurableBoundedLoss](#)
- [GHKR22BiasBounties.MeasurableGroup](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.MeasureApproxBayesOptimal](#)
- [GHKR22BiasBounties.MeasureCertificateOfSuboptimality](#)
- [GHKR22BiasBounties.Model](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: MeasurableSpace X
4. parameter: MeasurableSpace Y
5. parameter: MeasureTheory.Measure (X × Y)
6. assumption: MeasureTheory.IsProbabilityMeasure law
7. parameter: GHKR22BiasBounties.BoundedLoss Y
8. assumption: GHKR22BiasBounties.MeasurableBoundedLoss loss
9. parameter: Set (GHKR22BiasBounties.Group X × GHKR22BiasBounties.Model X Y)
10. assumption: ∀ pair ∈ certificates, GHKR22BiasBounties.MeasurableGroup pair.1 ∧ GHKR22BiasBounties.MeasurableModel pair.2
11. parameter: ℝ
12. assumption: 0 ≤ epsilon
13. parameter: GHKR22BiasBounties.Model X Y
14. assumption: GHKR22BiasBounties.MeasurableModel current
15. conclusion: (∃ (group : GHKR22BiasBounties.Group X) (replacement : GHKR22BiasBounties.Model X Y) (mu : ℝ) (Delta : ℝ),
  (group, replacement) ∈ certificates ∧
  GHKR22BiasBounties.MeasureCertificateOfSuboptimality law loss current group replacement mu Delta ∧
  epsilon < mu * Delta) ↔
  ¬GHKR22BiasBounties.MeasureApproxBayesOptimal law loss certificates epsilon current
```

Lean proof endpoint (built separately)

GHKR22BiasBounties.source_theorem8

Recorded source-to-Spec screening Verdict: matches

Reason: The transparent target states that a pair in C has positive certificate parameters with epsilon strictly below μ times Delta if and only if the current model is not epsilon-approximately Bayes optimal relative to C. This is exactly Theorem 8's certificate characterization.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

Review row 3

Source locator: cited publication, lines 426-433

Verbatim source input

```
\begin{restatable}{theorem}{progress}
\label{thm:progress}
Algorithm \ref{alg:basic} (ListUpdate) has the following properties. If  $(g_{t+1}, h_{t+1})$  form a  $(\mu, \Delta)$ -certificate of sub-optimality for  $f_t$ ,
 $\hookrightarrow$  and  $f_{t+1} = \text{ListUpdate}(f_t, (g_{t+1}, h_{t+1}))$  then:
\begin{enumerate}
\item The new model matches the performance of  $h_{t+1}$  on group  $g_{t+1}$ :  $L(\mathcal{D}, f_{t+1}, g_{t+1}) = L(\mathcal{D}, h_{t+1}, g_{t+1})$ , and
\item The overall performance of the model is improved by at least  $\mu \cdot \Delta$ :  $L(\mathcal{D}, f_{t+1}) \leq L(\mathcal{D}, f_t) - \mu \cdot \Delta$ .
\end{enumerate}
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {X : Type u_1} {Y : Type u_2} [inst : MeasurableSpace X] [inst_1 : MeasurableSpace Y]
{law : MeasureTheory.Measure (X × Y)} [MeasureTheory.IsProbabilityMeasure law]
{loss : GHKR22BiasBounties.BoundedLoss Y},
GHKR22BiasBounties.MeasurableBoundedLoss loss →
  ∀ {current replacement : GHKR22BiasBounties.Model X Y},
  GHKR22BiasBounties.MeasurableModel current →
  GHKR22BiasBounties.MeasurableModel replacement →
  ∀ {group : GHKR22BiasBounties.Group X},
  GHKR22BiasBounties.MeasurableGroup group →
  ∀ {mu Delta : ℝ},
  GHKR22BiasBounties.MeasureCertificateOfSuboptimality law loss current group replacement mu Delta →
  GHKR22BiasBounties.measureGroupLoss law loss (GHKR22BiasBounties.listUpdate current group replacement)
    group =
  GHKR22BiasBounties.measureGroupLoss law loss replacement group ∧
  GHKR22BiasBounties.measureModelLoss law loss
    (GHKR22BiasBounties.listUpdate current group replacement) ≤
  GHKR22BiasBounties.measureModelLoss law loss current - mu * Delta
```

Retained governing declarations

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasurableBoundedLoss](#)
- [GHKR22BiasBounties.MeasurableGroup](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.MeasureCertificateOfSuboptimality](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.listUpdate](#)
- [GHKR22BiasBounties.measureGroupLoss](#)
- [GHKR22BiasBounties.measureModelLoss](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: MeasurableSpace X
4. parameter: MeasurableSpace Y
5. parameter: MeasureTheory.Measure (X × Y)
6. assumption: MeasureTheory.IsProbabilityMeasure law
7. parameter: GHKR22BiasBounties.BoundedLoss Y
8. assumption: GHKR22BiasBounties.MeasurableBoundedLoss loss
9. parameter: GHKR22BiasBounties.Model X Y
10. parameter: GHKR22BiasBounties.Model X Y
11. assumption: GHKR22BiasBounties.MeasurableModel current
12. assumption: GHKR22BiasBounties.MeasurableModel replacement
13. parameter: GHKR22BiasBounties.Group X
14. assumption: GHKR22BiasBounties.MeasurableGroup group
15. parameter: ℝ
16. parameter: ℝ
17. assumption: GHKR22BiasBounties.MeasureCertificateOfSuboptimality law loss current group replacement mu Delta
18. conclusion: GHKR22BiasBounties.measureGroupLoss law loss (GHKR22BiasBounties.listUpdate current group replacement) group =
GHKR22BiasBounties.measureGroupLoss law loss replacement group ∧
GHKR22BiasBounties.measureModelLoss law loss (GHKR22BiasBounties.listUpdate current group replacement) ≤
GHKR22BiasBounties.measureModelLoss law loss current - mu * Delta

Lean proof endpoint (built separately)

GHKR22BiasBounties.source_theorem9

Recorded source-to-Spec screening Verdict: matches

Reason: The transparent target proves that ListUpdate matches the replacement model's group loss on the submitted group and decreases total loss by at least μ times Δ whenever the submitted pair is a certificate. These are exactly Theorem 9's two conclusions.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 461-464

Verbatim source input

```
\begin{restatable}{theorem}{updatebound}
\label{thm:updatebound}
Fix any  $\epsilon > 0$ . For any initial model  $f_0$  with loss  $L(\mathcal{D}, f_0) = \ell_0$  and any sequence of models  $f_1, \dots, f_T$ , such that  $f_i =$ 
 $\rightarrow \text{ListUpdate}(f_{i-1}, (g_i, h_i))$  and each pair  $(g_i, h_i)$  forms a  $(\mu, \Delta)$ -certificate of suboptimality for  $f_{i-1}$  for some
 $\rightarrow \mu, \Delta$  such that  $\mu \cdot \Delta \geq \epsilon$ , the length of the update sequence must be at most  $T \leq \frac{\ell_0}{\epsilon}$ 
 $\rightarrow \frac{1}{\epsilon}$ .
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {X : Type u_1} {Y : Type u_2} [inst : MeasurableSpace X] [inst_1 : MeasurableSpace Y]
(law : MeasureTheory.Measure (X × Y)) [MeasureTheory.IsProbabilityMeasure law]
{loss : GHKR22BiasBounties.BoundedLoss Y},
GHKR22BiasBounties.MeasurableBoundedLoss loss →
  ∀ (epsilon : ℝ),
  0 < epsilon →
    ∀ (initial : GHKR22BiasBounties.Model X Y) (updates : List (GHKR22BiasBounties.Update X Y)),
      GHKR22BiasBounties.MeasureValidUpdateSequence law loss epsilon initial updates →
        ↑ updates.length ≤ GHKR22BiasBounties.measureModelLoss law loss initial / epsilon ∧
          GHKR22BiasBounties.measureModelLoss law loss initial / epsilon ≤ 1 / epsilon
```

Retained governing declarations

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.MeasurableBoundedLoss](#)
- [GHKR22BiasBounties.MeasureValidUpdateSequence](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.Update](#)
- [GHKR22BiasBounties.measureModelLoss](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: MeasurableSpace X
4. parameter: MeasurableSpace Y
5. parameter: MeasureTheory.Measure (X × Y)
6. assumption: MeasureTheory.IsProbabilityMeasure law
7. parameter: GHKR22BiasBounties.BoundedLoss Y
8. assumption: GHKR22BiasBounties.MeasurableBoundedLoss loss
9. parameter: ℝ
10. assumption: 0 < epsilon
11. parameter: GHKR22BiasBounties.Model X Y
12. parameter: List (GHKR22BiasBounties.Update X Y)
13. assumption: GHKR22BiasBounties.MeasureValidUpdateSequence law loss epsilon initial updates
14. conclusion: ↑ updates.length ≤ GHKR22BiasBounties.measureModelLoss law loss initial / epsilon ∧ GHKR22BiasBounties.measureModelLoss law loss initial / epsilon ≤ 1 / epsilon

Lean proof endpoint (built separately)

GHKR22BiasBounties.source_theorem10

Recorded source-to-Spec screening Verdict: matches

Reason: The transparent target assumes every accepted update improves loss by at least epsilon and proves both length at most initial loss divided by epsilon and initial loss divided by epsilon at most 1/epsilon. This is exactly Theorem 10's two displayed update-count bounds.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 549-561

Verbatim source input

```
\begin{restatable}{theorem}{certchecker}
\label{thm:adaptive}
Let  $\mathcal{D}$  in  $\Delta(\mathcal{X} \times \mathcal{Y})$  be any distribution over labelled examples, and let  $D \sim \mathcal{D}^n$  be a holdout dataset consisting of  $n$  i.i.d. samples
 $\hookrightarrow$  from  $\mathcal{D}$ . Suppose:
\ifdraft
 $\$n \geq \frac{65 \ln(2U/\delta)}{\epsilon^3}$ 
\else  $\$n \geq \frac{65 \ln(2U/\delta)}{\epsilon^3}$ . \fi
Let  $\pi$  be the output stream generated by CertificateChecker $(\epsilon, D, (f_1, g_1, h_1), \dots)$  (Algorithm \ref{alg:checker}).
Then for any possibly adaptive process generating a stream of up to  $U$  submissions  $(f_i, g_i, h_i)$  as a function of the output stream  $\pi$  in
 $\hookrightarrow \bot, \top$ , with probability  $1 - \delta$  over the randomness of  $D$ :
\begin{enumerate}
\item For every round  $i$  such that  $\pi_i = \bot$  (the submission is rejected), we have that  $(g_i, h_i)$  is not a  $(\mu, \Delta)$  certificate of
 $\hookrightarrow$  sub-optimality for  $f_i$  for any  $(\mu, \Delta)$  with  $\mu \cdot \Delta \geq \epsilon$ . And:
\item For every round  $i$  such that  $\pi_i = \top$  (the submission is accepted), we have that  $(g_i, h_i)$  is a  $(\mu, \Delta)$ -certificate of sub-optimality
 $\hookrightarrow$  for  $f_i$  for  $\mu \cdot \Delta \geq \epsilon/2$ .
\end{enumerate}
\end{restatable}
```

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For every adaptive strategy, the probability that any issued checker decision violates the epsilon-rejection or epsilon/2-acceptance semantics is at most 2
 $\hookrightarrow U(U+1)^{\lfloor 2/\epsilon \rfloor} \exp(-n \epsilon^2/32)$.

Archival source anchor: cited publication, lines 549-561

Expanded PaperInterface specification

```
∀ {X : Type u_1} {Y : Type u_2} [inst : MeasurableSpace X] [inst_1 : MeasurableSpace Y]
(law : MeasureTheory.Measure (X × Y)) [MeasureTheory.IsProbabilityMeasure law]
{loss : GHKR22BiasBounties.BoundedLoss Y},
GHKR22BiasBounties.MeasurableBoundedLoss loss →
  ∀ (epsilon : ℝ),
  0 < epsilon →
  ∀ (n U : ℕ),
  0 < n →
  ∀ (strategy : GHKR22BiasBounties.AdaptiveSubmissionStrategy X Y),
  (∀ (transcript : List GHKR22BiasBounties.CertificateDecision),
    GHKR22BiasBounties.MeasureSubmissionMeasurable (strategy transcript)) →
    GHKR22BiasBounties.measureCertificateCheckerRunGuaranteeFailure law loss epsilon n U strategy ≤
    ↑(U * (U + 1) ^ GHKR22BiasBounties.checkerAcceptanceBudget epsilon) * 2 *
    Real.exp (-↑n * epsilon ^ 2 / 32)
```

Retained governing declarations

- [GHKR22BiasBounties.AdaptiveSubmissionStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.MeasurableBoundedLoss](#)
- [GHKR22BiasBounties.MeasureSubmissionMeasurable](#)
- [GHKR22BiasBounties.checkerAcceptanceBudget](#)
- [GHKR22BiasBounties.measureCertificateCheckerRunGuaranteeFailure](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: MeasurableSpace X
4. parameter: MeasurableSpace Y
5. parameter: MeasureTheory.Measure (X × Y)
6. assumption: MeasureTheory.IsProbabilityMeasure law
7. parameter: GHKR22BiasBounties.BoundedLoss Y
8. assumption: GHKR22BiasBounties.MeasurableBoundedLoss loss
9. parameter: ℝ
10. assumption: 0 < epsilon
11. parameter: ℕ
12. parameter: ℕ
13. assumption: 0 < n
14. parameter: GHKR22BiasBounties.AdaptiveSubmissionStrategy X Y
15. assumption: ∀ (transcript : List GHKR22BiasBounties.CertificateDecision),
GHKR22BiasBounties.MeasureSubmissionMeasurable (strategy transcript)
16. conclusion: GHKR22BiasBounties.measureCertificateCheckerRunGuaranteeFailure law loss epsilon n U strategy ≤
↑(U * (U + 1) ^ GHKR22BiasBounties.checkerAcceptanceBudget epsilon) * 2 * Real.exp (-↑n * epsilon ^ 2 / 32)

Lean proof endpoint (built separately)

GHKR22BiasBounties.source_theorem11

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: The transparent target quantifies over every adaptive submission strategy and bounds the probability of any issued accept/reject semantic error by $2 U (U+1)^{\lfloor 2/\epsilon \rfloor} \exp(-n \epsilon^2/32)$. This exactly matches the approved correction replacing the undefined confidence symbol and invalid transcript count in the printed theorem.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 6

Source locator: cited publication, lines 633-645

Verbatim source input

```
\begin{restatable}{theorem}{bountySC}
\label{thm:bounty-SC}
Fix any  $\epsilon, \delta > 0$ . Let  $\mathcal{D} \in \Delta(\mathcal{X} \times \mathcal{Y})$  be any distribution over labelled examples, and let  $\mathcal{D} \sim \mathcal{D}^n$  be a holdout dataset
 $\hookrightarrow$  consisting of  $n$  i.i.d. samples from  $\mathcal{D}$ . Suppose:
\ifdraft
 $\$n \geq \frac{65 \ln(2U/\delta)}{\epsilon^3}$ .
\else  $\$n \geq \frac{65 \ln(2U/\delta)}{\epsilon^3}$ .
\fi
Then for any (possibly adaptive) process generating a sequence of at most  $U$  submissions  $\{(g_i, h_i)\}_{i=1}^U$ , with probability at least  $1 - \delta$ ,
 $\hookrightarrow$  we have that FalsifyAndUpdate( $\epsilon, \mathcal{D}$ ) satisfies:
\begin{enumerate}
\item If  $(g_i, h_i)$  is rejected, then  $(g_i, h_i)$  is not a  $(\mu, \Delta)$ -certificate of sub-optimality for  $f_t$ , where  $f_t$  is the current model at the time
 $\hookrightarrow$  of submission  $i$ , for any  $\mu, \Delta$  such that  $\mu \cdot \Delta \geq \epsilon$ .
\item If  $(g_i, h_i)$  is accepted, then  $(g_i, h_i)$  is a  $(\mu, \Delta)$ -certificate of sub-optimality for  $f_t$ , where  $f_t$  is the current model at the time of
 $\hookrightarrow$  submission  $i$ , for some  $\mu, \Delta$  such that  $\mu \cdot \Delta \geq \frac{\epsilon}{2}$ . Moreover, the new model  $f_{t+1}$  output satisfies
 $\hookrightarrow$   $L(\mathcal{D}, f_{t+1}, g_i) = L(\mathcal{D}, h_i, g_i)$  and  $L(\mathcal{D}, f_{t+1}) \leq L(\mathcal{D}, f_t) - \frac{\epsilon}{2}$ .
\item FalsifyAndUpdate does not halt before receiving all  $U$  submissions.
\end{enumerate}
\end{restatable}
```

Formalized review target The archival source above is not asserted equivalent to this formalized target.

The complete adaptive `FalsifyAndUpdate` failure event, including incorrect decisions, premature termination, and an excess update count, has the same
 $\hookrightarrow$ explicit exact sparse-transcript tail as the checker.

Archival source anchor: cited publication, lines 633-645

Expanded PaperInterface specification

```
∀ {X : Type u_1} {Y : Type u_2} [inst : MeasurableSpace X] [inst_1 : MeasurableSpace Y]
(law : MeasureTheory.Measure (X × Y)) [MeasureTheory.IsProbabilityMeasure law]
{loss : GHKR22BiasBounties.BoundedLoss Y},
GHKR22BiasBounties.MeasurableBoundedLoss loss →
  ∀ (epsilon : ℝ),
  0 < epsilon →
  ∀ (n : ℕ),
  0 < n →
  ∀ (initial : GHKR22BiasBounties.Model X Y),
  GHKR22BiasBounties.MeasurableModel initial →
  ∀ (strategy : GHKR22BiasBounties.AdaptiveProposalStrategy X Y),
  (∀ (transcript : List GHKR22BiasBounties.CertificateDecision),
    GHKR22BiasBounties.MeasureProposalMeasurable (strategy transcript)) →
  GHKR22BiasBounties.measureAdaptiveFalsifyAndUpdateFailure law loss epsilon n initial strategy ≤
    ↑ (U * (U + 1) ^ GHKR22BiasBounties.checkerAcceptanceBudget epsilon) * 2 *
    Real.exp (-↑ n * epsilon ^ 2 / 32)
```

Retained governing declarations

- [GHKR22BiasBounties.AdaptiveProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.MeasurableBoundedLoss](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.MeasureProposalMeasurable](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.checkerAcceptanceBudget](#)
- [GHKR22BiasBounties.measureAdaptiveFalsifyAndUpdateFailure](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: MeasurableSpace X
4. parameter: MeasurableSpace Y
5. parameter: MeasureTheory.Measure (X × Y)
6. assumption: MeasureTheory.IsProbabilityMeasure law
7. parameter: GHKR22BiasBounties.BoundedLoss Y
8. assumption: GHKR22BiasBounties.MeasurableBoundedLoss loss
9. parameter: ℝ
10. assumption: 0 < epsilon

11. parameter: $\mathbb{N}$
 12. parameter: $\mathbb{N}$
 13. assumption: $0 < n$
 14. parameter: `GHKR22BiasBounties.Model X Y`
 15. assumption: `GHKR22BiasBounties.MeasurableModel initial`
 16. parameter: `GHKR22BiasBounties.AdaptiveProposalStrategy X Y`
 17. assumption: $\forall$ (transcript : List `GHKR22BiasBounties.CertificateDecision`),
 `GHKR22BiasBounties.MeasureProposalMeasurable (strategy transcript)`
 18. conclusion: `GHKR22BiasBounties.measureAdaptiveFalsifyAndUpdateFailure law loss epsilon n U initial strategy` $\leq$
 $\uparrow (U * (U + 1) ^ \text{GHKR22BiasBounties.checkerAcceptanceBudget epsilon}) * 2 * \text{Real.exp } (- \uparrow n * \text{epsilon} ^ 2 / 32)$

Lean proof endpoint (built separately)

`GHKR22BiasBounties.source_theorem12`

Recorded source-to-Spec screening Verdict: matches approved corrected target

Reason: The transparent target bounds the complete adaptive `FalsifyAndUpdate` failure event by the same exact sparse-transcript tail as the checker. That event includes incorrect decisions, premature termination, and violation of the update budget, exactly matching the approved corrected endpoint for Theorem 12.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 7

Source locator: cited publication, lines 659-661

Verbatim source input

```
\begin{remark}
Note that FalsifyAndUpdate has sample complexity scaling only logarithmically with the total number of submissions $U$ that we can accept, and no
↪ dependence on the complexity of the submissions. This means that a relatively small holdout dataset $D$ is sufficient to run an extremely
↪ long-running bias bounty program (i.e. handling a number of submissions that is exponentially large in the size of the holdout set) that automatically
↪ updates the current model whenever submissions are accepted and bounties are awarded.
\end{remark}
```

Expanded PaperInterface specification

```
∀ {X : Type u_1} {Y : Type u_2} [inst : MeasurableSpace X] [inst_1 : MeasurableSpace Y]
(law : MeasureTheory.Measure (X × Y)) [MeasureTheory.IsProbabilityMeasure law]
{loss : GHKR22BiasBounties.BoundedLoss Y},
GHKR22BiasBounties.MeasurableBoundedLoss loss →
  ∀ (U K : ℕ) (family : GHKR22BiasBounties.AdaptiveQueryIndex U K → GHKR22BiasBounties.Submission X Y),
  (∀ (query : GHKR22BiasBounties.AdaptiveQueryIndex U K),
    GHKR22BiasBounties.MeasureSubmissionMeasurable (family query)) →
    ∀ (epsilon : ℝ),
      0 ≤ epsilon →
        ∀ (n : ℕ),
          0 < n →
            GHKR22BiasBounties.measureAdaptiveCheckerDeviationFailure law loss family epsilon n ≤
              ↑(U * (U + 1) ^ K) * 2 * Real.exp (-↑n * epsilon ^ 2 / 32)
```

Retained governing declarations

- [GHKR22BiasBounties.AdaptiveQueryIndex](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.MeasurableBoundedLoss](#)
- [GHKR22BiasBounties.MeasureSubmissionMeasurable](#)
- [GHKR22BiasBounties.Submission](#)
- [GHKR22BiasBounties.measureAdaptiveCheckerDeviationFailure](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: MeasurableSpace X
4. parameter: MeasurableSpace Y
5. parameter: MeasureTheory.Measure (X × Y)
6. assumption: MeasureTheory.IsProbabilityMeasure law
7. parameter: GHKR22BiasBounties.BoundedLoss Y
8. assumption: GHKR22BiasBounties.MeasurableBoundedLoss loss
9. parameter: ℕ
10. parameter: ℕ
11. parameter: GHKR22BiasBounties.AdaptiveQueryIndex U K → GHKR22BiasBounties.Submission X Y
12. assumption: ∀ (query : GHKR22BiasBounties.AdaptiveQueryIndex U K), GHKR22BiasBounties.MeasureSubmissionMeasurable (family query)
13. parameter: ℝ
14. assumption: 0 ≤ epsilon
15. parameter: ℕ
16. assumption: 0 < n
17. conclusion: GHKR22BiasBounties.measureAdaptiveCheckerDeviationFailure law loss family epsilon n ≤
  ↑(U * (U + 1) ^ K) * 2 * Real.exp (-↑n * epsilon ^ 2 / 32)
```

Lean proof endpoint (built separately)

GHKR22BiasBounties.source_remark13

Recorded source-to-Spec screening Verdict: matches

Reason: The transparent target bounds the adaptive checker failure probability by $2 U (U+1)^K \exp(-n \epsilon^2/32)$. Solving this bound for n yields logarithmic dependence on the submission horizon U and no dependence on the syntactic complexity of submitted models, which is precisely Remark 13's claim.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 8

Source locator: cited publication, lines 717-726

Verbatim source input

```
\begin{restatable}{theorem}{bountymonotone}
\label{thm:bounty-monotone-SC}
Fix any  $\epsilon, \delta > 0$ . Let  $\mathcal{D}$  in  $\Delta(\mathcal{X} \times \mathcal{Y})$  be any distribution over labelled examples, and let  $\mathcal{D}^n$  be a holdout dataset
 $\hookrightarrow$  consisting of  $n$  i.i.d. samples from  $\mathcal{D}$ . Suppose: \ifdraft
 $n \geq \frac{65}{\ln \left( \frac{2(U + \frac{8}{\epsilon^3})}{\delta} \right) \epsilon^3}$ .
\else  $n \geq \frac{65}{\ln \left( \frac{2(U + \frac{8}{\epsilon^3})}{\delta} \right) \epsilon^3}$ . \fi
Then for any (possibly adaptive) process generating a sequence of at most  $U$  submissions  $\{(g_i, h_i)\}_{i=1}^U$ , with probability at least  $1 - \delta$ ,
 $\hookrightarrow$  we have that  $\text{MonotoneFalsifyAndUpdates}(\epsilon, \mathcal{D}, \cdot)$  satisfies all of the properties proven in Theorem \ref{thm:bounty-SC} for
 $\hookrightarrow$   $\text{FalsifyAndUpdate}$ , and additionally satisfies the following error monotonicity property. Consider any model  $f_t$  that is output, and any group  $g_j$  in
 $\hookrightarrow$   $\mathcal{G}(f_t)$ . Then:
\ifdraft
 $\$L(\mathcal{D}, f_t, g_j) \leq \min_{\ell < t} L(\mathcal{D}, f_\ell, g_j) + \frac{\epsilon}{\mu_{\mathcal{D}}(g_j)}$ 
\else  $\$L(\mathcal{D}, f_t, g_j) \leq \min_{\ell < t} L(\mathcal{D}, f_\ell, g_j) + \frac{\epsilon}{\mu_{\mathcal{D}}(g_j)}$ . \fi
\end{restatable}
```

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For the shared-state restarting monotone process, the complete failure event has the exact sparse-transcript tail at query horizon $U + \lfloor 2/\epsilon \rfloor^3$,
 $\hookrightarrow$ and outside it every published model satisfies the advertised historical-group monotonicity bound.

Archival source anchor: cited publication, lines 717-726

Expanded PaperInterface specification

```
∀ {X : Type u_1} {Y : Type u_2} [inst : MeasurableSpace X] [inst_1 : MeasurableSpace Y]
(law : MeasureTheory.Measure (X × Y)) [MeasureTheory.IsProbabilityMeasure law]
{loss : GHKR22BiasBounties.BoundedLoss Y},
GHKR22BiasBounties.MeasurableBoundedLoss loss →
  ∀ (epsilon : ℝ),
    0 < epsilon →
      ∀ (n U : ℕ),
        0 < n →
          ∀ (initial : GHKR22BiasBounties.Model X Y),
            GHKR22BiasBounties.MeasurableModel initial →
              ∀ (strategy : GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y),
                (∀ (transcript : List GHKR22BiasBounties.CertificateDecision),
                  GHKR22BiasBounties.MeasureProposalMeasurable (strategy transcript) →
                    have Q : ℕ := GHKR22BiasBounties.monotoneBountyQueryBudget epsilon U;
                    GHKR22BiasBounties.measureMonotoneBountyFailure law loss epsilon n U initial strategy ≤
                      ↑(Q * (Q + 1) ^ GHKR22BiasBounties.checkerAcceptanceBudget epsilon) * 2 *
                      Real.exp (-↑n * epsilon ^ 2 / 32))
```

Retained governing declarations

- [GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy](#)
- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CertificateDecision](#)
- [GHKR22BiasBounties.MeasurableBoundedLoss](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.MeasureProposalMeasurable](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.checkerAcceptanceBudget](#)
- [GHKR22BiasBounties.measureMonotoneBountyFailure](#)
- [GHKR22BiasBounties.monotoneBountyQueryBudget](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: MeasurableSpace X
4. parameter: MeasurableSpace Y
5. parameter: MeasureTheory.Measure (X × Y)
6. assumption: MeasureTheory.IsProbabilityMeasure law
7. parameter: GHKR22BiasBounties.BoundedLoss Y
8. assumption: GHKR22BiasBounties.MeasurableBoundedLoss loss
9. parameter: ℝ
10. assumption: 0 < epsilon
11. parameter: ℕ

12. parameter: $\mathbb{N}$
 13. assumption: $0 < n$
 14. parameter: GHKR22BiasBounties.Model X Y
 15. assumption: GHKR22BiasBounties.MeasurableModel initial
 16. parameter: GHKR22BiasBounties.AdaptiveMonotoneProposalStrategy X Y
 17. assumption: $\forall$ (transcript : List GHKR22BiasBounties.CertificateDecision),
 GHKR22BiasBounties.MeasureProposalMeasurable (strategy transcript)
 18. conclusion: GHKR22BiasBounties.measureMonotoneBountyFailure law loss epsilon n U initial strategy $\leq$
 $\uparrow$ (GHKR22BiasBounties.monotoneBountyQueryBudget epsilon U *
 (GHKR22BiasBounties.monotoneBountyQueryBudget epsilon U + 1) ^
 GHKR22BiasBounties.checkerAcceptanceBudget epsilon) *
 2 *
 Real.exp (- $\uparrow$ n * epsilon ^ 2 / 32)

Lean proof endpoint (built separately)

GHKR22BiasBounties.source_theorem14

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: The transparent target analyzes the single shared-state, restart-after-acceptance monotone process at query horizon $U + \text{floor}(2/\text{epsilon})^3$ and bounds its complete failure event by the exact sparse-transcript tail. The failure event includes the advertised historical-group monotonicity guarantee, so the target matches the approved correction to Theorem 14 and Algorithm 4's state semantics.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 9

Source locator: cited publication, lines 766-770

Verbatim source input

The algorithmic problem we need to solve at each round t is, given an existing model f_{t-1} , find a (μ, Δ) -certificate of optimality (g, h) in $\mathcal{C}$ for f_{t-1} that maximizes $\mu \cdot \Delta$ as computed on the empirical data D --- i.e. to solve:

$$\begin{aligned} & \text{\\begin{equation}} \\ & \text{\\label{eq:optproblem}} \\ & (g_t, h_t) = \arg\max_{(g, h) \in \mathcal{C}} \mu_D(g) \cdot (L(D, f_{t-1}, g) - L(D, h, g)) \\ & \text{\\end{equation}} \end{aligned}$$

Expanded PaperInterface specification

```
∀ {X : Type u_1} {Y : Type u_2} {n : ℕ} (loss : GHKR22BiasBounties.BoundedLoss Y) (sample : Fin n → X × Y)
(current : GHKR22BiasBounties.Model X Y) (candidates : Set (GHKR22BiasBounties.CandidatePair X Y))
(chosen : GHKR22BiasBounties.CandidatePair X Y),
GHKR22BiasBounties.EmpiricalCandidateMaximizer loss sample current candidates chosen ↔
  chosen ∈ candidates ∧
    ∀ candidate ∈ candidates,
      GHKR22BiasBounties.empiricalCandidateObjective loss sample current candidate ≤
        GHKR22BiasBounties.empiricalCandidateObjective loss sample current chosen
```

Retained governing declarations

- [GHKR22BiasBounties.BoundedLoss](#)
- [GHKR22BiasBounties.CandidatePair](#)
- [GHKR22BiasBounties.EmpiricalCandidateMaximizer](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.empiricalCandidateObjective](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Type u_2
3. parameter: ℕ
4. parameter: GHKR22BiasBounties.BoundedLoss Y
5. parameter: Fin n → X × Y
6. parameter: GHKR22BiasBounties.Model X Y
7. parameter: Set (GHKR22BiasBounties.CandidatePair X Y)
8. parameter: GHKR22BiasBounties.CandidatePair X Y
9. conclusion: GHKR22BiasBounties.EmpiricalCandidateMaximizer loss sample current candidates chosen ↔
  chosen ∈ candidates ∧
    ∀ candidate ∈ candidates,
      GHKR22BiasBounties.empiricalCandidateObjective loss sample current candidate ≤
        GHKR22BiasBounties.empiricalCandidateObjective loss sample current chosen
```

Lean proof endpoint (built separately)

GHKR22BiasBounties.certificateOptimizationObjective

Recorded source-to-Spec screening Verdict: matches

Reason: The expanded target states that the chosen group-model pair belongs to the candidate class and has empirical mass-weighted loss improvement at least that of every other candidate. This is exactly the relational form of the source equation's argmax; the explicit finite-sample sum expands the source objective rather than relying on the Lean name.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 10

Source locator: cited publication, lines 773-779

Verbatim source input

```
\begin{lemma}[Adapted from \citep{gerrymandering}]
\label{lem:samplecomplexity}
Let  $\mathcal{C}_G$  denote a class of group indicator functions with VC-dimension  $d_G$  and  $\mathcal{C}_H$  denote a class of binary models with VC-dimension  $d_H$ . Let
 $\hookrightarrow$   $f$  be an arbitrary binary model. Then if:
 $\$n \geq \tilde{O}(\frac{d_H + d_G}{\log(1/\delta)}) \cdot \{\eta^2\}$ 
we have that with probability  $1 - \delta$  over the draw of a dataset  $D \sim \mathcal{C}^n$ , for every  $g \in \mathcal{C}_G$  and  $h \in \mathcal{C}_H$ :
 $\$|\mu_{\mathcal{C}(D)}(g) - \mu_{\mathcal{C}(D,h)}(g) - \mu_D(g) + \mu_{\mathcal{C}(D,h)}(g) - \mu_D(h)| \leq \eta$ 
\end{lemma}
```

Formalized review target The archival source above is not asserted equivalent to this formalized target.

For every quantified g in G and h in H , the absolute difference between the population certificate objective for (g,h) and the empirical certificate objective
 $\hookrightarrow$ for the same (g,h) is at most η at the stated VC sample scale.

Archival source anchor: cited publication, lines 773-779

Expanded PaperInterface specification

```
∀ {X : Type u_1} [inst : MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
[MeasureTheory.IsProbabilityMeasure law] {current : GHKR22BiasBounties.Model X Bool},
GHKR22BiasBounties.MeasurableModel current →
  ∀ (G : Set (GHKR22BiasBounties.Group X)) (H : Set (GHKR22BiasBounties.Model X Bool)),
  (∀ g ∈ G, GHKR22BiasBounties.MeasurableGroup g) →
  (∀ h ∈ H, GHKR22BiasBounties.MeasurableModel h) →
  ∀ (dG dH : ℕ),
    AppliedModelingLib.Statistics.VCDimensionAtMost G dG →
    AppliedModelingLib.Statistics.VCDimensionAtMost H dH →
    ∀ (n : ℕ),
      0 < n →
      GHKR22BiasBounties.listUpdateVCDimensionBound dG dH ≤ n + n →
      ∀ (delta : ℝ),
        0 < delta →
        GHKR22BiasBounties.VCSourceEventsMeasurable law
          (GHKR22BiasBounties.ListUpdateClass current G H)
          (GHKR22BiasBounties.listUpdateVCDimensionBound dG dH) n (delta / 2) →
        GHKR22BiasBounties.VCSourceEventsMeasurable law {current} 1 n (delta / 2) →
        GHKR22BiasBounties.MeasureCertificateVCDeviationFailure law current G H
          (GHKR22BiasBounties.certificateVCConfidenceRadius dG dH n delta) n ≤
          delta
```

Retained governing declarations

- [AppliedModelingLib.Statistics.BinaryClassifier](#)
- [AppliedModelingLib.Statistics.VCDimensionAtMost](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.ListUpdateClass](#)
- [GHKR22BiasBounties.MeasurableGroup](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.MeasureCertificateVCDeviationFailure](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.VCSourceEventsMeasurable](#)
- [GHKR22BiasBounties.certificateVCConfidenceRadius](#)
- [GHKR22BiasBounties.listUpdateVCDimensionBound](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: MeasurableSpace X
3. parameter: MeasureTheory.Measure (X × Bool)
4. assumption: MeasureTheory.IsProbabilityMeasure law
5. parameter: GHKR22BiasBounties.Model X Bool
6. assumption: GHKR22BiasBounties.MeasurableModel current
7. parameter: Set (GHKR22BiasBounties.Group X)
8. parameter: Set (GHKR22BiasBounties.Model X Bool)
9. assumption: $\forall g \in G, \text{GHKR22BiasBounties.MeasurableGroup } g$
10. assumption: $\forall h \in H, \text{GHKR22BiasBounties.MeasurableModel } h$
11. parameter: $\mathbb{N}$

12. parameter: $\mathbb{N}$
 13. assumption: `AppliedModelingLib.Statistics.VCDimensionAtMost` G dG
 14. assumption: `AppliedModelingLib.Statistics.VCDimensionAtMost` H dH
 15. parameter: $\mathbb{N}$
 16. assumption: $0 < n$
 17. assumption: `GHKR22BiasBounties.listUpdateVCDimensionBound` dG $dH \leq n + n$
 18. parameter: $\mathbb{R}$
 19. assumption: $0 < \delta$
 20. assumption: `GHKR22BiasBounties.VCSourceEventsMeasurable` law `(GHKR22BiasBounties.ListUpdateClass` $\text{current } G$ H)
`(GHKR22BiasBounties.listUpdateVCDimensionBound` dG dH) n $(\delta / 2)$
 21. assumption: `GHKR22BiasBounties.VCSourceEventsMeasurable` law `{current}` 1 n $(\delta / 2)$
 22. conclusion: `GHKR22BiasBounties.MeasureCertificateVCDeviationFailure` law $\text{current } G$ H
`(GHKR22BiasBounties.certificateVCConfidenceRadius` dG dH n δ) $n \leq$
 δ

Lean proof endpoint (built separately)

`GHKR22BiasBounties.source_lemma15`

Recorded source-to-Spec screening Verdict: `matches_approved_corrected_target`

Reason: The transparent target gives a uniform high-probability bound for the same population-minus-empirical mass-weighted certificate objective over every g in G and h in H , using an explicit VC radius with the stated soft-linear $d_G + d_H$ and inverse-square accuracy scale. It matches the approved correction that replaces the source's undefined g_p by the quantified group g .

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 11

Source locator: cited publication, lines 816-822

Verbatim source input

`\begin{restatable}{theorem}{samplecomplexity}`
Fix an arbitrary distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$, a class of group indicator functions $\mathcal{G}$ of VC-dimension d_G and a class of binary models $\mathcal{H}$ of VC-dimension d_H . Let $\mathcal{C} \subseteq \mathcal{G} \times \mathcal{H}$ and $\epsilon > 0$ be arbitrary. If: `\ifdraft`
 $n \geq O\left(\frac{d_H + d_G}{\epsilon} + \log(1/\delta)\right) \epsilon^{-3}$
 $n \geq O\left(\frac{d_H + d_G}{\epsilon} + \log(1/\delta)\right) \epsilon^{-3}$ `\fi`
and $D \lesssim \mathcal{D}^n$,
then with probability $1 - \delta$, `TrainByOpt` $(D, \mathcal{C}, \epsilon)$ (Algorithm `\ref{alg:TrainByOpt}`) returns a model f that is $(\epsilon, \mathcal{C})$ -Bayes optimal,
`\using` at most $2/\epsilon$ calls to a sub-routine for solving the optimization problem over (g, h) in $\mathcal{C}$ given in `\eqref{eq:optproblem}`.
`\end{restatable}`

Expanded PaperInterface specification

```

V {X : Type u_1} [inst : MeasureableSpace X] (law : MeasureTheory.Measure (X × Bool))
[inst_1 : MeasureTheory.IsProbabilityMeasure law] (G : Set (GHKR22BiasBounties.Group X))
(H : Set (GHKR22BiasBounties.Model X Bool)),
(∀ g ∈ G, GHKR22BiasBounties.MeasurableGroup g) →
(∀ h ∈ H, GHKR22BiasBounties.MeasurableModel h) →
  ∀ (C : Set (GHKR22BiasBounties.CandidatePair X Bool)),
    GHKR22BiasBounties.CandidateClassContainedIn C G H →
      GHKR22BiasBounties.MeasureCandidateClassMeasurable C →
        ∀ (dG dH : ℕ),
          AppliedModelingLib.Statistics.VCDimensionAtMost G dG →
            AppliedModelingLib.Statistics.VCDimensionAtMost H dH →
              ∀ (epsilon : ℝ),
                0 < epsilon →
                  ∀ {blockSize : ℕ},
                    0 < blockSize →
                      GHKR22BiasBounties.listUpdateVCDimensionBound dG dH ≤ blockSize + blockSize →
                        ∀ (oracle : GHKR22BiasBounties.TrainingOracle X Bool blockSize),
                          GHKR22BiasBounties.ExactTrainingOracle GHKR22BiasBounties.binaryZeroOneLoss C oracle →
                            ∀ {initial : GHKR22BiasBounties.Model X Bool},
                              GHKR22BiasBounties.MeasurableModel initial →
                                ∀ (delta : ℝ),
                                  0 < delta →
                                    GHKR22BiasBounties.certificateVCConfidenceRadius dG dH blockSize
                                      (delta / ↑(GHKR22BiasBounties.trainByOptRoundBudget epsilon)) ≤
                                        epsilon / 4 →
                                          (∀ (current : GHKR22BiasBounties.Model X Bool),
                                            GHKR22BiasBounties.MeasurableModel current →
                                              GHKR22BiasBounties.VCSourceEventsMeasurable law
                                                (GHKR22BiasBounties.listUpdateClass current G H)
                                                (GHKR22BiasBounties.listUpdateVCDimensionBound dG dH) blockSize
                                                  (delta / ↑(GHKR22BiasBounties.trainByOptRoundBudget epsilon) / 2)) →
                                                (∀ (current : GHKR22BiasBounties.Model X Bool),
                                                  GHKR22BiasBounties.MeasurableModel current →
                                                    GHKR22BiasBounties.VCSourceEventsMeasurable law {current} 1 blockSize
                                                      (delta / ↑(GHKR22BiasBounties.trainByOptRoundBudget epsilon) / 2)) →
                                                    GHKR22BiasBounties.MeasureTrainByOptVCBadEventPairMeasurable law G H
                                                        epsilon
                                                          (GHKR22BiasBounties.certificateVCConfidenceRadius dG dH blockSize
                                                            (delta / ↑(GHKR22BiasBounties.trainByOptRoundBudget epsilon)))
                                                            oracle initial →
                                                              GHKR22BiasBounties.MeasureTrainByOptOutputFailureProbability law C
                                                                epsilon oracle initial
                                                                  (GHKR22BiasBounties.trainByOptRoundBudget epsilon) ≤
                                                                    delta ∧
                                                                      (∀
                                                                          (trace :
                                                                              Fin (GHKR22BiasBounties.trainByOptRoundBudget epsilon) →
                                                                                Fin blockSize → X × Bool),
                                                                              (GHKR22BiasBounties.trainByOptRun
                                                                                  GHKR22BiasBounties.binaryZeroOneLoss epsilon oracle initial
                                                                                    (GHKR22BiasBounties.trainByOptRoundBudget epsilon)
                                                                                    trace).calls ≤
                                                                                  GHKR22BiasBounties.trainByOptRoundBudget epsilon) ∧
                                                                                Fintype.card
                                                                                  (Fin (GHKR22BiasBounties.trainByOptRoundBudget epsilon) ×
                                                                                    Fin blockSize) =
                                                                                  GHKR22BiasBounties.trainByOptRoundBudget epsilon * blockSize

```

Retained governing declarations

- `AppliedModelingLib.Statistics.BinaryClassifier`
- `AppliedModelingLib.Statistics.VCDimensionAtMost`
- `GHKR22BiasBounties.CandidateClassContainedIn`
- `GHKR22BiasBounties.CandidatePair`
- `GHKR22BiasBounties.ExactTrainingOracle`
- `GHKR22BiasBounties.Group`

- [GHKR22BiasBounties.ListUpdateClass](#)
- [GHKR22BiasBounties.MeasurableGroup](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.MeasureCandidateClassMeasurable](#)
- [GHKR22BiasBounties.MeasureTrainByOptOutputFailureProbability](#)
- [GHKR22BiasBounties.MeasureTrainByOptVCBadEventPairMeasurable](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TrainByOptOutcome.calls](#)
- [GHKR22BiasBounties.TrainingOracle](#)
- [GHKR22BiasBounties.VCSourceEventsMeasurable](#)
- [GHKR22BiasBounties.binaryZeroOneLoss](#)
- [GHKR22BiasBounties.certificateVCConfidenceRadius](#)
- [GHKR22BiasBounties.listUpdateVCDimensionBound](#)
- [GHKR22BiasBounties.trainByOptRoundBudget](#)
- [GHKR22BiasBounties.trainByOptRun](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: MeasurableSpace X
3. parameter: MeasureTheory.Measure $(X \times \text{Bool})$
4. assumption: MeasureTheory.IsProbabilityMeasure law
5. parameter: Set (GHKR22BiasBounties.Group X)
6. parameter: Set (GHKR22BiasBounties.Model X Bool)
7. assumption: $\forall g \in G, \text{GHKR22BiasBounties.MeasurableGroup } g$
8. assumption: $\forall h \in H, \text{GHKR22BiasBounties.MeasurableModel } h$
9. parameter: Set (GHKR22BiasBounties.CandidatePair X Bool)
10. assumption: GHKR22BiasBounties.CandidateClassContainedIn $C \ G \ H$
11. assumption: GHKR22BiasBounties.MeasureCandidateClassMeasurable C
12. parameter: $\mathbb{N}$
13. parameter: $\mathbb{N}$
14. assumption: AppliedModelingLib.Statistics.VCDimensionAtMost $G \ dG$
15. assumption: AppliedModelingLib.Statistics.VCDimensionAtMost $H \ dH$
16. parameter: $\mathbb{R}$
17. assumption: $0 < \text{epsilon}$
18. parameter: $\mathbb{N}$
19. assumption: $0 < \text{blockSize}$
20. assumption: GHKR22BiasBounties.listUpdateVCDimensionBound $dG \ dH \leq \text{blockSize} + \text{blockSize}$
21. parameter: GHKR22BiasBounties.TrainingOracle X Bool blockSize
22. assumption: GHKR22BiasBounties.ExactTrainingOracle GHKR22BiasBounties.binaryZeroOneLoss C oracle
23. parameter: GHKR22BiasBounties.Model X Bool
24. assumption: GHKR22BiasBounties.MeasurableModel initial
25. parameter: $\mathbb{R}$
26. assumption: $0 < \text{delta}$
27. assumption: GHKR22BiasBounties.certificateVCConfidenceRadius $dG \ dH \ \text{blockSize}$
 $(\text{delta} / \uparrow(\text{GHKR22BiasBounties.trainByOptRoundBudget } \text{epsilon})) \leq \text{epsilon} / 4$
28. assumption: $\forall (\text{current} : \text{GHKR22BiasBounties.Model } X \ \text{Bool}),$
 $\text{GHKR22BiasBounties.MeasurableModel } \text{current} \rightarrow$
 $\text{GHKR22BiasBounties.VCSourceEventsMeasurable law } (\text{GHKR22BiasBounties.ListUpdateClass } \text{current } G \ H)$
 $(\text{GHKR22BiasBounties.listUpdateVCDimensionBound } dG \ dH) \ \text{blockSize}$
 $(\text{delta} / \uparrow(\text{GHKR22BiasBounties.trainByOptRoundBudget } \text{epsilon}) / 2)$
29. assumption: $\forall (\text{current} : \text{GHKR22BiasBounties.Model } X \ \text{Bool}),$
 $\text{GHKR22BiasBounties.MeasurableModel } \text{current} \rightarrow$
 $\text{GHKR22BiasBounties.VCSourceEventsMeasurable law } \{\text{current}\} \ 1 \ \text{blockSize}$
 $(\text{delta} / \uparrow(\text{GHKR22BiasBounties.trainByOptRoundBudget } \text{epsilon}) / 2)$
30. assumption: GHKR22BiasBounties.MeasureTrainByOptVCBadEventPairMeasurable law $G \ H \ \text{epsilon}$
 $(\text{GHKR22BiasBounties.certificateVCConfidenceRadius } dG \ dH \ \text{blockSize})$
 $(\text{delta} / \uparrow(\text{GHKR22BiasBounties.trainByOptRoundBudget } \text{epsilon}))$
 oracle initial
31. conclusion: GHKR22BiasBounties.MeasureTrainByOptOutputFailureProbability law $C \ \text{epsilon} \ \text{oracle initial}$
 $(\text{GHKR22BiasBounties.trainByOptRoundBudget } \text{epsilon}) \leq \text{delta} \wedge$
 $(\forall (\text{trace} : \text{Fin } (\text{GHKR22BiasBounties.trainByOptRoundBudget } \text{epsilon}) \rightarrow \text{Fin } \text{blockSize} \rightarrow X \times \text{Bool}),$
 $(\text{GHKR22BiasBounties.trainByOptRun } \text{GHKR22BiasBounties.binaryZeroOneLoss } \text{epsilon} \ \text{oracle initial}$
 $(\text{GHKR22BiasBounties.trainByOptRoundBudget } \text{epsilon}) \ \text{trace}).\text{calls} \leq$
 $\text{GHKR22BiasBounties.trainByOptRoundBudget } \text{epsilon} \wedge$
 $\text{Fintype.card } (\text{Fin } (\text{GHKR22BiasBounties.trainByOptRoundBudget } \text{epsilon}) \times \text{Fin } \text{blockSize}) =$
 $\text{GHKR22BiasBounties.trainByOptRoundBudget } \text{epsilon} * \text{blockSize}$

Lean proof endpoint (built separately)

GHKR22BiasBounties.source_theorem16

Recorded source-to-Spec screening Verdict: matches

Reason: The transparent target proves that TrainByOpt with fresh blocks returns an epsilon-approximately Bayes-optimal model except with probability delta, makes at most $\text{ceil}(2/\epsilon)$ optimization calls, and uses exactly the corresponding number of blocks. Its explicit VC-radius premise instantiates the source's soft-O sample bound and integer-round interpretation without weakening the advertised guarantee.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 12

Source locator: cited publication, lines 893-900

Verbatim source input

```
\begin{restatable}{theorem}{lemCSC}
\label{lem:CSC}
Fix an arbitrary distribution  $\mathcal{D}$  over  $\mathcal{X} \times \mathcal{Y}$ , let  $\mathcal{K}$  be a class of ternary valued functions, and let  $f: \mathcal{X} \rightarrow \{0,1\}$  be any binary
 $\hookrightarrow$  valued model. Let  $p^*$  be a solution to the cost-sensitive classification problem  $(\mathcal{D}, K, \{c_{\cdot}(x,y)\}^f)$ , where  $\{c_{\cdot}(x,y)\}^f$  are the induced
 $\hookrightarrow$  costs of  $f$ . We have that:
\ifdraft

$$(g_{\{p^*\}}, h_{\{p^*\}}) \in \arg\max_{\{(g,h) \in \mathcal{C}_K\}} \mu_{\mathcal{D}}(g) \cdot (L(\mathcal{D}, f, g) - L(\mathcal{D}, h, g))$$

\else 
$$(g_{\{p^*\}}, h_{\{p^*\}}) \in \arg\max_{\{(g,h) \in \mathcal{C}_K\}} \mu_{\mathcal{D}}(g) \cdot (L(\mathcal{D}, f, g) - L(\mathcal{D}, h, g))$$
. \fi
In other words, when  $\mathcal{D}$  is the empirical distribution over  $\mathcal{D}$ ,  $(g_{\{p^*\}}, h_{\{p^*\}})$  form a solution to optimization problem \eqref{eq:optproblem}.
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {X : Type u_1} [inst : MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
[MeasureTheory.IsProbabilityMeasure law] {current : GHKR22BiasBounties.Model X Bool},
GHKR22BiasBounties.MeasurableModel current →
  ∀ (K : Set (GHKR22BiasBounties.TernaryPredictor X)) (pStar : GHKR22BiasBounties.TernaryPredictor X),
  (∀ p ∈ K, Measurable p) →
    GHKR22BiasBounties.MeasureCostSensitiveMinimizer law current K pStar →
      GHKR22BiasBounties.MeasureDerivedCertificateMaximizer law current K pStar
```

Retained governing declarations

- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.MeasureCostSensitiveMinimizer](#)
- [GHKR22BiasBounties.MeasureDerivedCertificateMaximizer](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.TernaryLabel](#)
- [GHKR22BiasBounties.TernaryPredictor](#)
- [GHKR22BiasBounties.paperInterfaceTernaryLabelMeasurableSpace](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: MeasurableSpace X
3. parameter: MeasureTheory.Measure (X × Bool)
4. assumption: MeasureTheory.IsProbabilityMeasure law
5. parameter: GHKR22BiasBounties.Model X Bool
6. assumption: GHKR22BiasBounties.MeasurableModel current
7. parameter: Set (GHKR22BiasBounties.TernaryPredictor X)
8. parameter: GHKR22BiasBounties.TernaryPredictor X
9. assumption: ∀ p ∈ K, Measurable p
10. assumption: GHKR22BiasBounties.MeasureCostSensitiveMinimizer law current K pStar
11. conclusion: GHKR22BiasBounties.MeasureDerivedCertificateMaximizer law current K pStar

Lean proof endpoint (built separately)

GHKR22BiasBounties.source_theorem20

Recorded source-to-Spec screening Verdict: matches

Reason: The transparent target assumes pStar minimizes the induced ternary expected cost over K and concludes that its derived group/model pair maximizes the binary certificate objective over the induced class. This is exactly Theorem 20’s cost-sensitive-classification reduction.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 13

Source locator: cited publication, lines 939-942

Verbatim source input

```
\begin{restatable}{lemma}{lemhopt}
\label{lem:hopt}
Fix any  $g_t$  in  $\mathcal{C}_g$  and dataset  $\mathcal{D}$  in  $\mathcal{C}_X^n$ . Let  $\mathcal{D}_{g_t} = \{(x,y) \in \mathcal{C}_D : g_t(x) = 1\}$  be the subset of  $\mathcal{D}$  consisting of members of group  $g$ .
 $\hookrightarrow$  Let  $h^* = \arg\min_{h \in \mathcal{C}_H} L(\mathcal{D}_{g_t}, h)$ . Then  $h^*$  is a solution to optimization problem \ref{eq:hmin}.
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {X : Type u_1} [inst : MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
[MeasureTheory.IsProbabilityMeasure law] {current : GHKR22BiasBounties.Model X Bool},
GHKR22BiasBounties.MeasurableModel current →
  ∀ {group : GHKR22BiasBounties.Group X},
  GHKR22BiasBounties.MeasurableGroup group →
    ∀ (H : Set (GHKR22BiasBounties.Model X Bool)),
      (∀ h ∈ H, GHKR22BiasBounties.MeasurableModel h) →
        ∀ (hStar : GHKR22BiasBounties.Model X Bool),
          GHKR22BiasBounties.MeasureGroupRestrictedERM law group H hStar →
            hStar ∈ H ∧
              ∀ h ∈ H,
                GHKR22BiasBounties.MeasureBinaryCertificateObjective law current group h ≤
                  GHKR22BiasBounties.MeasureBinaryCertificateObjective law current group hStar
```

Retained governing declarations

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasurableGroup](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.MeasureBinaryCertificateObjective](#)
- [GHKR22BiasBounties.MeasureGroupRestrictedERM](#)
- [GHKR22BiasBounties.Model](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: MeasurableSpace X
3. parameter: MeasureTheory.Measure (X × Bool)
4. assumption: MeasureTheory.IsProbabilityMeasure law
5. parameter: GHKR22BiasBounties.Model X Bool
6. assumption: GHKR22BiasBounties.MeasurableModel current
7. parameter: GHKR22BiasBounties.Group X
8. assumption: GHKR22BiasBounties.MeasurableGroup group
9. parameter: Set (GHKR22BiasBounties.Model X Bool)
10. assumption: ∀ h ∈ H, GHKR22BiasBounties.MeasurableModel h
11. parameter: GHKR22BiasBounties.Model X Bool
12. assumption: GHKR22BiasBounties.MeasureGroupRestrictedERM law group H hStar
13. conclusion: hStar ∈ H ∧
  ∀ h ∈ H,
    GHKR22BiasBounties.MeasureBinaryCertificateObjective law current group h ≤
      GHKR22BiasBounties.MeasureBinaryCertificateObjective law current group hStar
```

Lean proof endpoint (built separately)

GHKR22BiasBounties.source_lemma21

Recorded source-to-Spec screening Verdict: matches

Reason: For a fixed group, the transparent target assumes hStar minimizes the group-restricted loss numerator over H and concludes that hStar maximizes the same fixed-group certificate objective over H. The current-model term is constant in h, exactly matching Lemma 21's ERM reduction.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 14

Source locator: cited publication, lines 957-966

Verbatim source input

```
\begin{restatable}{\lemma}{\lemgopt}
\label{lem:gopt}
Fix any  $h_t$  in  $\mathcal{H}$  and dataset  $D$  in  $\mathcal{X}^n$ . Let:
\ifdraft

$$D_{h_t}^1 = \{(x,y) \in D : h_t(x) = y \neq f_{t-1}(x)\} \cup \{(x,y) \in D : h_t(x) \neq y = f_{t-1}(x)\}$$


$$D_{h_t}^0 = \{(x,y) \in D : h_t(x) = y \neq f_{t-1}(x)\} \cup \{(x,y) \in D : h_t(x) \neq y = f_{t-1}(x)\}$$

\else

$$D_{h_t}^1 = \{(x,y) \in D : h_t(x) = y \neq f_{t-1}(x)\}$$
,  $D_{h_t}^0 = \{(x,y) \in D : h_t(x) \neq y = f_{t-1}(x)\}$ , and  $D_{h_t} =$ 

$$\{(x,y) \in D : h_t(x) = y \neq f_{t-1}(x)\} \cup \{(x,y) \in D : h_t(x) \neq y = f_{t-1}(x)\}$$
.
\fi
Let  $g^* = \arg\min_{g \in \mathcal{G}} L(D_{h_t}, g)$ . Then  $g^*$  is a solution to optimization problem \eqref{eq:gmin}.
\end{restatable}
```

Expanded PaperInterface specification

```
∀ {X : Type u_1} [inst : MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
[MeasureTheory.IsProbabilityMeasure law] {current replacement : GHKR22BiasBounties.Model X Bool},
GHKR22BiasBounties.MeasurableModel current →
GHKR22BiasBounties.MeasurableModel replacement →
∀ (G : Set (GHKR22BiasBounties.Group X)),
(∀ g ∈ G, GHKR22BiasBounties.MeasurableGroup g) →
∀ (gStar : GHKR22BiasBounties.Group X),
GHKR22BiasBounties.MeasureDisagreementERM law current replacement G gStar →
gStar ∈ G ∧
∀ g ∈ G,
GHKR22BiasBounties.MeasureBinaryCertificateObjective law current g replacement ≤
GHKR22BiasBounties.MeasureBinaryCertificateObjective law current gStar replacement
```

Retained governing declarations

- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasurableGroup](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.MeasureBinaryCertificateObjective](#)
- [GHKR22BiasBounties.MeasureDisagreementERM](#)
- [GHKR22BiasBounties.Model](#)

Lean-elaborated claim roles

1. parameter: Type u_1
2. parameter: MeasurableSpace X
3. parameter: MeasureTheory.Measure (X × Bool)
4. assumption: MeasureTheory.IsProbabilityMeasure law
5. parameter: GHKR22BiasBounties.Model X Bool
6. parameter: GHKR22BiasBounties.Model X Bool
7. assumption: GHKR22BiasBounties.MeasurableModel current
8. assumption: GHKR22BiasBounties.MeasurableModel replacement
9. parameter: Set (GHKR22BiasBounties.Group X)
10. assumption: ∀ g ∈ G, GHKR22BiasBounties.MeasurableGroup g
11. parameter: GHKR22BiasBounties.Group X
12. assumption: GHKR22BiasBounties.MeasureDisagreementERM law current replacement G gStar
13. conclusion: gStar ∈ G ∧
 ∀ g ∈ G,
 GHKR22BiasBounties.MeasureBinaryCertificateObjective law current g replacement ≤
 GHKR22BiasBounties.MeasureBinaryCertificateObjective law current gStar replacement

Lean proof endpoint (built separately)

GHKR22BiasBounties.source_lemma22

Recorded source-to-Spec screening Verdict: matches

Reason: For a fixed replacement model, the transparent target assumes gStar minimizes the exact disagreement-loss construction over G and concludes that gStar maximizes the certificate objective over G. This is the statement of Lemma 22; the Lean proof uses the correct constant-minus-objective identity rather than the source proof's erroneous normalization.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 15

Source locator: cited publication, lines 1008-1014

Verbatim source input

```
\begin{restatable}{theorem}{localopt}
Let  $D$  in  $(\mathcal{X} \times \mathcal{Y})^n$  be an arbitrary dataset,  $f_{t-1} : \mathcal{X} \rightarrow \mathcal{Y}$  be an arbitrary model,  $\mathcal{C}_G$  and  $\mathcal{C}_H$  be arbitrary group and model
 $\hookrightarrow$  classes, and  $\epsilon > 0$ . Then after solving at most  $2/\epsilon$  empirical risk minimization problems over each of  $\mathcal{C}_G$  and  $\mathcal{C}_H$ ,
 $\hookrightarrow$  AltMinCertificateFinder (Algorithm \ref{alg:altmin}) returns a  $(\mu, \Delta)$ -certificate of sub-optimality  $(g_{t,h})$  for  $f_{t-1}$  that is an
 $\hookrightarrow$   $\epsilon$ -approximate local optimum (or saddle point) in the sense that:
\begin{enumerate}
\item For every  $h$  in  $\mathcal{C}_H$ ,  $(g_{t,h})$  is not a  $(\mu', \Delta')$ -certificate of sub-optimality for  $f_{t-1}$  for any  $\mu' \cdot \Delta' \geq \mu \cdot \Delta$ 
 $\hookrightarrow$  +  $\epsilon$ , and
\item For every  $g$  in  $\mathcal{C}_G$ ,  $(g, h_t)$  is not a  $(\mu', \Delta')$ -certificate of sub-optimality for  $f_{t-1}$  for any  $\mu' \cdot \Delta' \geq \mu \cdot \Delta$ 
 $\hookrightarrow$  +  $\epsilon$ 
\end{enumerate}
\end{restatable}
```

Formalized review target The archival source above is not asserted equivalent to this formalized target.

From an admissible initialization with positive certificate objective, repaired two-gap coordinate ascent returns a positive certificate that is ϵ -locally
 $\hookrightarrow$ optimal in both coordinates; every strict-update trajectory has at most $2/\epsilon$ updates.

Archival source anchor: cited publication, lines 1008-1014

Expanded PaperInterface specification

```
( $\forall$  {X : Type u_1} [inst : MeasurableSpace X] (law : MeasureTheory.Measure (X  $\times$  Bool))
[MeasureTheory.IsProbabilityMeasure law] {current : GHKR22BiasBounties.Model X Bool},
GHKR22BiasBounties.MeasurableModel current  $\rightarrow$ 
 $\forall$  (G : Set (GHKR22BiasBounties.Group X)) (H : Set (GHKR22BiasBounties.Model X Bool)),
( $\forall$  g  $\in$  G, GHKR22BiasBounties.MeasurableGroup g)  $\rightarrow$ 
( $\forall$  h  $\in$  H, GHKR22BiasBounties.MeasurableModel h)  $\rightarrow$ 
 $\forall$  (epsilon :  $\mathbb{R}$ ),
0 < epsilon  $\rightarrow$ 
 $\forall$  (oracle : GHKR22BiasBounties.MeasureCoordinateBestResponses law current G H)
(initial : GHKR22BiasBounties.CertificatePair X),
GHKR22BiasBounties.PairAdmissible G H initial  $\rightarrow$ 
0 <
GHKR22BiasBounties.MeasureBinaryCertificateObjective law current initial.group
initial.replacement  $\rightarrow$ 
 $\exists$  (output : GHKR22BiasBounties.CertificatePair X),
GHKR22BiasBounties.measureCoordinateAscentLoop law current epsilon oracle
(GHKR22BiasBounties.measureCoordinateAscentFuel epsilon) initial =
some output  $\wedge$ 
GHKR22BiasBounties.MeasureEpsilonCoordinatewiseLocal law current G H epsilon output  $\wedge$ 
GHKR22BiasBounties.MeasureCertificateOfSuboptimality law
GHKR22BiasBounties.binaryZeroOneLoss current output.group output.replacement
(GHKR22BiasBounties.measureGroupMass law output.group)
(GHKR22BiasBounties.measureGroupLoss law GHKR22BiasBounties.binaryZeroOneLoss current
output.group -
GHKR22BiasBounties.measureGroupLoss law GHKR22BiasBounties.binaryZeroOneLoss
output.replacement output.group))  $\wedge$ 
( $\forall$  {X : Type u_2} [inst : MeasurableSpace X] (law : MeasureTheory.Measure (X  $\times$  Bool))
[MeasureTheory.IsProbabilityMeasure law] {current : GHKR22BiasBounties.Model X Bool},
GHKR22BiasBounties.MeasurableModel current  $\rightarrow$ 
 $\forall$  (epsilon :  $\mathbb{R}$ ),
0 < epsilon  $\rightarrow$ 
 $\forall$  (initial : GHKR22BiasBounties.CertificatePair X) (updates : List (GHKR22BiasBounties.CertificatePair X)),
GHKR22BiasBounties.MeasureImprovingCoordinateSequence law current epsilon initial updates  $\rightarrow$ 
GHKR22BiasBounties.MeasurableGroup initial.group  $\rightarrow$ 
GHKR22BiasBounties.MeasurableModel initial.replacement  $\rightarrow$ 
GHKR22BiasBounties.MeasurableGroup (updates.getLastD initial).group  $\rightarrow$ 
GHKR22BiasBounties.MeasurableModel (updates.getLastD initial).replacement  $\rightarrow$ 
 $\uparrow$  updates.length  $\leq$  2 / epsilon)  $\wedge$ 
 $\forall$  (epsilon :  $\mathbb{R}$ ), 0 < epsilon  $\rightarrow$  GHKR22BiasBounties.measureCoordinateAscentFuel epsilon + 1 =  $\lceil 2 / \epsilon \rceil$ +
```

Retained governing declarations

- [GHKR22BiasBounties.CertificatePair](#)
- [GHKR22BiasBounties.Group](#)
- [GHKR22BiasBounties.MeasurableGroup](#)
- [GHKR22BiasBounties.MeasurableModel](#)
- [GHKR22BiasBounties.MeasureBinaryCertificateObjective](#)
- [GHKR22BiasBounties.MeasureCertificateOfSuboptimality](#)
- [GHKR22BiasBounties.MeasureCoordinateBestResponses](#)
- [GHKR22BiasBounties.MeasureEpsilonCoordinatewiseLocal](#)

- [GHKR22BiasBounties.MeasureImprovingCoordinateSequence](#)
- [GHKR22BiasBounties.Model](#)
- [GHKR22BiasBounties.PairAdmissible](#)
- [GHKR22BiasBounties.binaryZeroOneLoss](#)
- [GHKR22BiasBounties.measureCoordinateAscentFuel](#)
- [GHKR22BiasBounties.measureCoordinateAscentLoop](#)
- [GHKR22BiasBounties.measureGroupLoss](#)
- [GHKR22BiasBounties.measureGroupMass](#)

Lean-elaborated claim roles

```

1. conclusion: (∀ {X : Type u_1} [inst : MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
[MeasureTheory.IsProbabilityMeasure law] {current : GHKR22BiasBounties.Model X Bool},
GHKR22BiasBounties.MeasurableModel current →
  ∀ (G : Set (GHKR22BiasBounties.Group X)) (H : Set (GHKR22BiasBounties.Model X Bool)),
  (∀ g ∈ G, GHKR22BiasBounties.MeasurableGroup g) →
  (∀ h ∈ H, GHKR22BiasBounties.MeasurableModel h) →
  ∀ (epsilon : ℝ),
  0 < epsilon →
  ∀ (oracle : GHKR22BiasBounties.MeasureCoordinateBestResponses law current G H)
  (initial : GHKR22BiasBounties.CertificatePair X),
  GHKR22BiasBounties.PairAdmissible G H initial →
  0 <
  GHKR22BiasBounties.MeasureBinaryCertificateObjective law current initial.group
  initial.replacement →
  ∃ (output : GHKR22BiasBounties.CertificatePair X),
  GHKR22BiasBounties.measureCoordinateAscentLoop law current epsilon oracle
  (GHKR22BiasBounties.measureCoordinateAscentFuel epsilon) initial =
  some output ∧
  GHKR22BiasBounties.MeasureEpsilonCoordinatewiseLocal law current G H epsilon output ∧
  GHKR22BiasBounties.MeasureCertificateOfSuboptimality law
  GHKR22BiasBounties.binaryZeroOneLoss current output.group output.replacement
  (GHKR22BiasBounties.measureGroupMass law output.group)
  (GHKR22BiasBounties.measureGroupLoss law GHKR22BiasBounties.binaryZeroOneLoss current
  output.group -
  GHKR22BiasBounties.measureGroupLoss law GHKR22BiasBounties.binaryZeroOneLoss
  output.replacement output.group) ∧
  (∀ {X : Type u_2} [inst : MeasurableSpace X] (law : MeasureTheory.Measure (X × Bool))
[MeasureTheory.IsProbabilityMeasure law] {current : GHKR22BiasBounties.Model X Bool},
GHKR22BiasBounties.MeasurableModel current →
  ∀ (epsilon : ℝ),
  0 < epsilon →
  ∀ (initial : GHKR22BiasBounties.CertificatePair X) (updates : List (GHKR22BiasBounties.CertificatePair X)),
  GHKR22BiasBounties.MeasureImprovingCoordinateSequence law current epsilon initial updates →
  GHKR22BiasBounties.MeasurableGroup initial.group →
  GHKR22BiasBounties.MeasurableModel initial.replacement →
  GHKR22BiasBounties.MeasurableGroup (updates.getLastD initial).group →
  GHKR22BiasBounties.MeasurableModel (updates.getLastD initial).replacement →
  ↑updates.length ≤ 2 / epsilon) ∧
  ∀ (epsilon : ℝ), 0 < epsilon → GHKR22BiasBounties.measureCoordinateAscentFuel epsilon + 1 = ⌈2 / epsilon⌉+

```

Lean proof endpoint (built separately)

GHKR22BiasBounties.source_theorem23

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: The transparent target starts from an admissible pair with positive certificate objective, returns an epsilon-local optimum in both coordinates that is a genuine positive certificate, proves every strict-improvement trajectory has at most 2/epsilon updates, and records the exact rounded response-round count. These are exactly the approved two-gap and positive-initialization repairs to Theorem 23.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation